

Test Automation Interview Preparation

250 real interview questions with model answers, explanations, code and diagrams — built for a **4–6 year experience** SDET / Automation Engineer profile, but starting from the basics so a newcomer can follow every line.

It covers everything a product or services company actually asks a mid-level automation candidate: **manual testing thinking** (they still ask, and it is where most people lose marks), **Java fundamentals**, **Selenium WebDriver + framework design**, and **Playwright with TypeScript** — the stack most teams migrate to today — plus **Appium**, **performance and security basics**, and a **live-coding section** of exercises to type out by hand.

250

QUESTIONS

9

SECTIONS

14

DIAGRAMS

210+

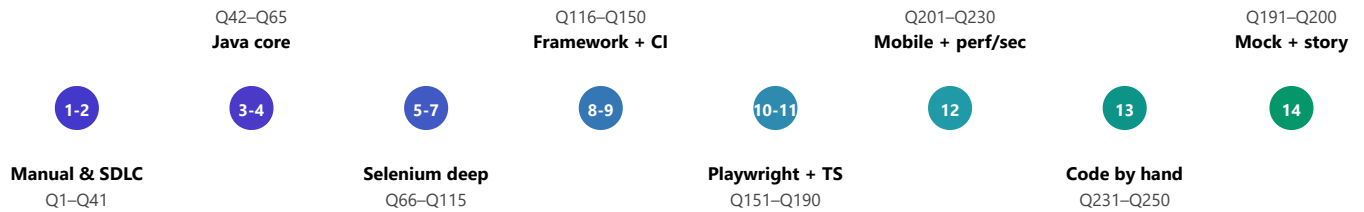
CODE SAMPLES

How to use it

- **Study mode** (default) — answers are open. Read a section end-to-end, out loud.
- **Quiz mode** — click the button in the header to collapse every answer. Read the question, *say your answer aloud in 60–90 seconds*, then click to reveal. Speaking is the skill being tested, not reading.
- **Search** — press and type a keyword (e.g. `StaleElement`, `fixture`, `severity`) to filter instantly.
- **PDF** — click **Print / PDF** and choose "Save as PDF". All answers are automatically expanded and diagrams are preserved.
- Tags tell you what a question is: **CORE** must know cold, **ADVANCED** senior differentiator, **TRAP** commonly answered wrong.

The single biggest scoring tip: for every technical answer, add one sentence of "on my project we...". Interviewers grade experience, not definitions. A textbook-perfect answer with no project story scores lower than an average answer anchored in real work. As you read, write your own project example next to each topic.

14-Day Interview Sprint — all 250 questions



Every day: 2 hours reading in study mode + 30 min speaking answers aloud in quiz mode + 1 hour writing code by hand.
Day 14: rehearse your 3-minute project introduction until it needs no thinking.

Figure 0 — A realistic two-week plan. If you have only a weekend, do Sections 1, 3, 5 and 9, and memorise your project story.

Study one category at a time

This page holds all 250 questions. If you'd rather read a single topic — better on a phone, and faster to load — each category is also a standalone page with its own search and quiz mode.

Q1–Q41 · 41 questions

1 · Manual Testing & QA Fundamentals

Test design techniques, STLC, defect life cycle, severity vs priority, Agile, risk-based testing.

Q42–Q65 · 24 questions

2 · Java Essentials for Automation

OOP, collections, exceptions, Java 8 streams, generics, ThreadLocal, coding-round questions.

Q66–Q115 · 50 questions

3 · Selenium WebDriver with Java

Architecture, locators, waits, frames, windows, Actions, JS executor, Grid, CDP, flakiness.

Q116–Q150 · 35 questions

4 · TestNG, Framework, Maven, Git & CI/CD

Annotations, data providers, listeners, Page Object Model, design patterns, Jenkins, Docker, BDD.

Q151–Q190 · 40 questions

5 · Playwright with TypeScript / JS

Auto-waiting, locators, fixtures, tracing, network mocking, parallelism, migration from Selenium.

Q191–Q200 · 10 questions

6 · API, Database, Scenario & Behavioural

HTTP methods and status codes, auth, SQL for testers, and the questions that decide offers.

Q201–Q215 · 15 questions

7 · Appium & Mobile Automation

Q216–Q230 · 15 questions

8 · Performance, Security & Non-Functional

Appium 2 architecture, mobile locators, gestures, context switching, ADB, device strategy.

JMeter, load test design, bottleneck analysis, OWASP Top 10, accessibility, contract testing.

Q231–Q250 · 20 questions

9 · Practical Coding Exercises

Twenty write-the-code tasks with full solutions, for the live-coding round.

SECTION 1

Manual Testing & QA Fundamentals

Q1 – Q41 · Even for a pure automation role, the first 15 minutes are usually manual/process questions. This is where interviewers decide whether you *think* like a tester or only *code* like one.

The V-Model — every development phase has a matching test phase

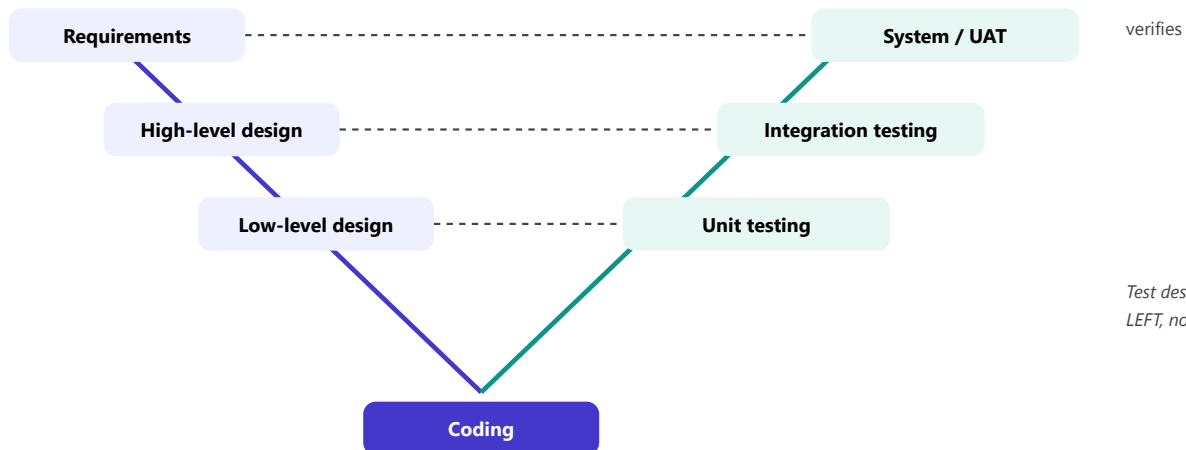


Figure 1 — The V-model. Use it to explain "shift-left": test artefacts are written in parallel with design, so defects are caught in documents (cheap) instead of in production (expensive).

What is software testing, and what is its real objective?

Software testing is the process of evaluating a system to find the difference between **expected** and **actual** behaviour, and to give stakeholders information about the quality and risk of releasing it.

The objective is not "to find all bugs"

Exhaustive testing is impossible — a single form with 10 text fields has effectively infinite input combinations. The real objectives are:

- **Find defects early**, when they are cheapest to fix (a requirement defect costs ~1x to fix in analysis, ~100x in production).
- **Reduce risk** to an acceptable level and provide evidence for a go/no-go decision.
- **Prevent defects** — reviewing a requirement and asking "what happens if the amount is negative?" prevents a bug from ever being coded.
- **Build confidence** that the product meets business and user needs.

Say this in the interview: "Testing shows the presence of defects, not their absence." It is one of the seven ISTQB testing principles and instantly signals you know the fundamentals.

CORE

OPENING QUESTION

Q2

Difference between Verification and Validation?

ASPECT	VERIFICATION	VALIDATION
Question answered	"Are we building the product right ?"	"Are we building the right product?"
Focus	Conformance to specification / design / standards	Fitness for the user's actual need
Techniques	Reviews, walkthroughs, inspections, static analysis	Functional, system, UAT, exploratory testing
Code executed?	No (static)	Yes (dynamic)
Who	Author, peers, QA, architects	QA team and end users / product owner

Concrete example

A requirement says "the OTP must expire in 30 seconds". **Verification** = reviewing the design doc and confirming the timer is specified as 30s and the code implements 30s. **Validation** = actually waiting 31 seconds in the app and confirming the OTP is rejected — and also raising that users on a slow 2G network can never complete it in 30 seconds, so the *requirement itself* is wrong. That last part is validation catching a specification defect.

CORE

TRAP

QA vs QC vs Testing — how are they different?

- **Quality Assurance (QA)** — *process oriented, preventive*. Defining and improving the process so defects do not get created: definition of done, code review standards, branching strategy, CI gates, test entry/exit criteria, audits. QA is proactive.
- **Quality Control (QC)** — *product oriented, corrective*. Checking the actual built product against requirements to detect defects. Reactive.
- **Testing** — a *subset of QC*: the concrete activity of executing the system with inputs to observe behaviour.

Relationship: **QA** $\supset$ **QC** $\supset$ **Testing** .

Interview angle: "In my team QA is when I insist that no story is 'done' without automated regression coverage and a passing pipeline; QC is when I execute that regression suite on the release build."

Explain SDLC and the common models (Waterfall, V-Model, Agile).

SDLC = Software Development Life Cycle: Requirement gathering → Analysis → Design → Implementation → Testing → Deployment → Maintenance.

MODEL	HOW IT WORKS	USE WHEN	WEAKNESS
Waterfall	Strictly sequential; each phase completes before the next starts	Requirements are frozen and well understood (e.g. regulated hardware firmware)	Testing happens at the very end; change is expensive
V-Model	Same sequence, but each dev phase is paired with a test phase planned in parallel	Safety-critical / compliance projects (medical, avionics)	Still rigid about changing requirements
Iterative / Spiral	Build in repeated cycles with risk analysis each loop	Large, high-risk, evolving systems	Heavy process overhead
Agile / Scrum	Short sprints (1–3 weeks) delivering potentially shippable increments; continuous feedback	Most modern product development	Needs discipline; regression load grows every sprint → automation becomes mandatory
DevOps / CD	Agile extended into automated build-test-deploy pipelines, many releases per day	Cloud / SaaS products	Requires very high automation maturity and observability

Close with the automation link: *"In Agile the regression suite grows every sprint, so without automation the team's testing capacity becomes the release bottleneck by about sprint 6. That's the business case for my automation framework."*

Q5

What is STLC? Walk me through its phases and deliverables.

STLC (Software Testing Life Cycle) is the testing-side counterpart of SDLC. Six phases:

#	PHASE	ACTIVITY	DELIVERABLE
1	Requirement analysis	Study requirements, identify testable requirements, raise ambiguities, decide automation feasibility	RTM (draft), list of clarifications
2	Test planning	Scope, approach, effort estimation, resources, schedule, risk, tools	Test Plan, effort estimate
3	Test case design	Write cases and scripts, prepare test data, peer review	Test cases, test data, automation scripts, updated RTM
4	Environment setup	Set up test env, builds, DB, stubs/mocks, smoke check	Ready environment + smoke test results
5	Test execution	Execute, log defects, retest, regress, track	Execution report, defect reports
6	Test closure	Exit criteria evaluation, metrics, lessons learned, archive artefacts	Test Summary Report, RCA/retrospective notes

Each phase has its own **entry criteria**, **activities**, and **exit criteria** — say that sentence, it is exactly what the interviewer is listening for.

CORE

Q6

What are entry and exit criteria? Give real examples.

Entry criteria = conditions that must be true before a test phase can start. **Exit criteria** = conditions that must be true before it can be declared complete.

Example — System Test phase

- **Entry:** requirements signed off; test cases reviewed and approved; test environment available and stable; build deployed and smoke test passed; unit & integration tests green.
- **Exit:** 100% of planned test cases executed; $\geq 95\%$ pass rate; zero open Critical/High defects; all Medium defects triaged with an agreed fix or deferral; regression suite green; test summary report signed off.

Trap follow-up: "What if the deadline arrives and exit criteria are not met?" Correct answer: you do *not* silently pass it. You escalate with data — which criteria failed, the open defect list with severity, the risk of releasing anyway — and the business owner takes the go/no-go decision. QA's job is to expose risk, not to own the release decision alone.

CORE

TRAP

Q7

Test Plan vs Test Strategy — what is the difference, and what goes into a test plan?

	TEST STRATEGY	TEST PLAN
Level	Organisation / programme level	Project or release level
Content	Overall approach, standards, tools, test levels, automation policy, defect management process	Scope, schedule, resources, deliverables, environment, entry/exit, risks for <i>this</i> release
Changes	Rarely (static)	Frequently (living document)
Author	QA Manager / Head of Quality	Test Lead / senior QA

IEEE 829 Test Plan sections (know at least 8)

1. Test plan identifier & introduction
2. Scope — features to be tested / **not** to be tested (this one is asked a lot)
3. Test approach & levels
4. Item pass/fail criteria, entry/exit & suspension-resumption criteria
5. Test deliverables
6. Environment needs
7. Roles, responsibilities, staffing & training
8. Schedule & estimation
9. Risks and mitigations
10. Approvals

CORE

Test Scenario vs Test Case vs Test Script?

- **Test Scenario** — a one-line statement of *what* to test, from the user's point of view. "Verify user can log in." It answers "what areas must we cover?"
- **Test Case** — a detailed, repeatable set of steps with preconditions, test data, and an expected result for *one* condition. "Login with valid username and invalid password."
- **Test Script** — the automated/coded implementation of a test case.

One scenario typically explodes into 8–15 test cases (valid, invalid, empty, locked account, case sensitivity, SQL injection, session behaviour, remember-me, etc.).

Why interviewers ask: they want to see that you go from a vague requirement to a structured set of conditions. If they say "give me test scenarios for a login page", answer in *categories*: functional positive, functional negative, boundary, security, UI/usability, performance, compatibility, accessibility. Categories score far higher than a random list.

Q9

What makes a good test case? Write one for a login page.

A good test case is **atomic** (tests one thing), **independent** (does not depend on another case having run), **repeatable** (same result every time), **traceable** (linked to a requirement), and written so that **anyone** can execute it without asking you questions.

FIELD	VALUE
Test Case ID	TC_LOGIN_007
Requirement ID	REQ-AUTH-03 (account lockout)
Title	Account locks after 3 consecutive invalid password attempts
Priority / Severity	High
Preconditions	An active user <code>qa.user@test.com</code> exists, is not currently locked, failed-attempt counter = 0
Test data	Valid user, wrong password <code>Wrong@123</code>
Steps	1. Open <code>/login</code> 2. Enter valid email + wrong password, click Sign in 3. Repeat step 2 two more times 4. Enter valid email + correct password, click Sign in
Expected result	Steps 2–3 show "Invalid credentials" with attempts remaining. After the 3rd failure the account is locked; step 4 shows "Your account is locked. Try after 30 minutes or reset your password" and the user is not logged in. An audit entry is written.
Post-condition	Unlock the account / reset counter (cleanup)

Notice the expected result covers UI message **and** back-end state (audit entry, not logged in). Mentioning back-end verification separates a 5-year candidate from a fresher.

CORE

HANDS-ON

Q10**Explain Equivalence Partitioning with an example.**

Divide the input domain into **classes where the system is expected to behave identically**, then test one representative value from each class. Rationale: if one value in a class fails, all of them will — so testing 1 value instead of 1000 gives the same defect-finding power for 0.1% of the effort.

Example — "Age must be between 18 and 60"

PARTITION	RANGE	REPRESENTATIVE	EXPECTED
Invalid (low)	< 18	10	Reject
Valid	18 – 60	35	Accept
Invalid (high)	> 60	75	Reject
Invalid (type)	non-numeric / empty / negative / decimal	"abc", "", -5, 22.5	Reject with proper message

4 tests instead of hundreds. It applies to outputs and to non-numeric data too (e.g. user roles: admin / manager / guest are three partitions).

CORE

Q11**Explain Boundary Value Analysis. Why is it more effective than EP alone?**

Defects cluster at the **edges** of partitions because that is where developers write `>` instead of `>=` or use `<= size` instead of `< size`. BVA tests the values *at and around* each boundary.

For "Age 18 to 60"

- **2-value BVA** (each boundary ± 1): 17, 18 | 60, 61
- **3-value BVA** (min-1, min, min+1, max-1, max, max+1): 17, 18, 19, 59, 60, 61

EP and BVA are used **together**: EP tells you which classes exist, BVA tells you which specific values inside/around them to pick.

Other boundaries people forget: string length limits (0, 1, max, max+1 characters), file upload size, pagination (page 0, page 1, last page, last+1), dates (leap year 29-Feb, month-end 31st, year rollover 31-Dec 23:59:59), and empty/one/many for lists.

CORE

Q12

What is Decision Table testing? When do you use it?

Used when the output depends on a **combination of conditions** (business rules), not on a single input range. You enumerate all condition combinations and their expected actions, which guarantees no rule is missed.

Example — discount rule: "Premium members get 20% off; orders above ₹5000 get 10% off; they do not stack — the higher applies."

RULE	R1	R2	R3	R4
Premium member?	Y	Y	N	N
Order > ₹5000?	Y	N	Y	N
Expected discount	20%	20%	10%	0%

With n binary conditions you get 2^n rules. For large tables you collapse impossible/irrelevant combinations using "-" (don't care) entries.

This is the technique to name when asked "how would you test a loan eligibility / insurance premium / pricing engine?" — combination logic is exactly what decision tables are for.

CORE

Q13**What is State Transition testing?**

Used when the system's response depends on its **current state and history**, not just the current input. You model states, events, transitions and actions, then design tests that cover valid transitions — and importantly, attempt **invalid** ones.

Example — ATM card / login states

Active → (1 wrong PIN) → Active(1 fail) → (2nd wrong) → Active(2 fails) → (3rd wrong) → Blocked → (admin unblock) → Active

- **Valid transition tests:** each arrow above.
- **Invalid transition tests (high value):** can a Blocked card be used with the *correct* PIN? Can a cancelled order be shipped? Can a submitted-and-approved leave request be edited? These find the most bugs.
- **Coverage levels:** 0-switch = every single transition once; 1-switch = every valid pair of consecutive transitions.

Great real-world fits: order lifecycle (Created → Paid → Shipped → Delivered → Returned), ticket workflow, subscription (Trial → Active → Past-due → Cancelled).

CORE

ADVANCED

Q14

What is pairwise (all-pairs) / orthogonal array testing?

A combinatorial reduction technique. Research shows most defects are triggered by a single parameter or an interaction between just **two** parameters. So instead of testing all combinations, you generate the smallest set of tests in which **every pair of parameter values appears at least once**.

Example

Browser (Chrome, Firefox, Safari) × OS (Windows, macOS, Linux) × Payment (Card, UPI, Wallet) × Language (EN, HI) = $3 \times 3 \times 3 \times 2 = 54$ full combinations. Pairwise reduces this to roughly **9–10** tests while still covering every browser-OS, browser-payment, OS-payment and language pair.

Tools: **PICT** (Microsoft), **ACTS** (NIST), [pairwise](#) npm/pip libraries, or Hexawise.

Caveat to mention: pairwise will not catch a bug that only appears with a specific 3-way combination, and it is blind to business-critical combinations. So you always *seed* the must-test combinations (e.g. Chrome + Windows + Card) and let the tool fill in the rest.

ADVANCED

No. Both are **experience-based** techniques with structure.

- **Error guessing** — deliberately trying inputs that historically break systems: empty submit, whitespace-only, very long strings, special characters `<script>` / `'--`, 0 and negative numbers, duplicate submit (double click), browser back after submit, session timeout mid-flow, network drop mid-payment, copy-paste with trailing spaces, emoji in a name field.
- **Exploratory testing** — simultaneous learning, test design and execution, time-boxed and charter-driven. This is *Session-Based Test Management*: a 60–90 minute session with a written charter ("Explore checkout with expired coupons to discover pricing errors"), notes taken during the session, and a debrief afterwards. That is what makes it auditable rather than random.
- **Monkey/random testing** is the truly unstructured one — random inputs, no oracle beyond "did it crash".

Strong line: "Automation checks what we already know; exploratory testing discovers what we didn't think of. I reserve one timeboxed exploratory session per sprint for the newly built feature and it consistently finds defects my scripted cases miss."

CORE

ADVANCED

Q16

Black box vs White box vs Grey box testing.

	BLACK BOX	WHITE BOX	GREY BOX
Knowledge of code	None — works from requirements	Full internal structure	Partial (schema, APIs, architecture)
Done by	QA / users	Developers / SDET	SDET
Techniques	EP, BVA, decision table, state transition, use case	Statement, branch/decision, condition, path coverage, mutation testing	Matrix testing, DB validation, API + UI combined
Example	Submitting the order form and checking the confirmation page	Unit test that forces the <code>catch</code> branch of a payment gateway timeout	Placing an order in the UI then asserting the <code>orders</code> table row and the Kafka event

White box coverage types worth naming

- **Statement coverage** — every line executed at least once (weakest).
- **Branch/decision coverage** — every `if` takes both true and false.
- **Condition coverage** — every boolean sub-expression takes both values. 100% branch coverage does *not* imply 100% condition coverage.
- **Path coverage** — every independent path (usually impractical; measured via cyclomatic complexity).

CORE

Q17**List the main functional and non-functional testing types.****Functional (does it do the right thing?)**

Unit → Integration → System → Acceptance (UAT/alpha/beta); plus Smoke, Sanity, Regression, Retesting, Interface/API, Database, Localisation/Internationalisation.

Non-functional (how well does it do it?)

TYPE	CHECKS	TYPICAL TOOL
Performance / Load	Response time & throughput at expected load	JMeter, k6, Gatling
Stress	Behaviour beyond capacity; does it fail gracefully?	JMeter, Locust
Scalability	Does adding resources increase capacity linearly?	k6 + cloud autoscaling
Security	OWASP Top 10, authZ/authN, injection, XSS	OWASP ZAP, Burp Suite
Usability	Learnability, efficiency, error recovery	Heuristic review, user studies
Accessibility	WCAG 2.1 AA, screen readers, keyboard-only	axe-core, Lighthouse, NVDA
Compatibility	Browsers, OS, devices, resolutions	BrowserStack, LambdaTest
Reliability / Recovery	Failover, restart, data integrity after crash	Chaos tooling
Maintainability / Portability	Ease of change, install/uninstall/migrate	Static analysis (SonarQube)

CORE

Q18

Smoke vs Sanity testing.

	SMOKE	SANITY
Purpose	Is this build stable enough to test ?	Is this specific fix/change working ?
Breadth vs depth	Wide and shallow — all critical paths	Narrow and deep — one module/feature
When	On every new build, before the QA cycle starts	After a bug fix or minor change on a stable build
Scripted?	Usually scripted & automated (build verification test)	Often unscripted / ad-hoc
Failure means	Build rejected, sent back to dev	Fix rejected, defect reopened

Memory hook: **Smoke** = "**does the machine turn on without catching fire?**" (from hardware — power it up and see if smoke comes out). **Sanity** = "**is this one change sane?**"

Automation connection: your smoke suite should be tagged (@smoke in TestNG groups or @smoke tag in Playwright) and run on every deployment in under 5–10 minutes. Say that number — interviewers like concrete SLAs.

CORE

TRAP

Regression vs Retesting (confirmation testing).

- **Retesting** — re-executing the *exact failed test case* with the *same data* on a new build to confirm the defect is fixed. Always uses the defect's reproduction steps. Cannot be planned in advance (you don't know which tests will fail). Priority: done first.
- **Regression** — re-executing *previously passing* tests to confirm the fix or new feature did not break anything else. Can be planned and is the #1 candidate for automation.

Regression selection strategy (asked as a follow-up)

You rarely run everything. Selection approaches:

1. **Full regression** — before a major release or after a framework/infra change.
2. **Regional / impact-based** — analyse the changed module and its dependencies (talk to the dev, read the PR diff) and run those areas.
3. **Risk-based** — always include revenue-critical flows (login, checkout, payment) regardless of what changed.
4. **Test-impact analysis** — modern tooling maps code changes to the tests that cover them and runs only those.

CORE

TRAP

Explain the integration testing approaches.

- **Big Bang** — integrate everything at once, then test. Simple but fault isolation is a nightmare. Only for very small systems.
- **Top-down** — test high-level modules first, using **stubs** (dummy called-modules) for the not-yet-built lower ones. Advantage: the main control flow / UI is validated early.
- **Bottom-up** — test low-level modules first using **drivers** (dummy calling-modules). Advantage: core utilities are proven early; disadvantage: no working product to demo until late.
- **Sandwich / Hybrid** — both directions meeting in the middle. Most common in practice.
- **Incremental / continuous** — in modern microservices, integration is verified continuously using **contract testing** (Pact) and service virtualisation (WireMock, MockServer).

Stub vs Driver in one line: a *stub* replaces the module being **called** (used in top-down); a *driver* replaces the module that **calls** (used in bottom-up). Mnemonic: **S**tab is **S**ubordinate, **D**river **D**rives from above.

CORE

TRAP

System testing vs UAT. What are alpha and beta testing?

- **System testing** — QA validates the complete, integrated system against the **specified requirements** in a QA environment. Includes functional + non-functional.
- **UAT** — the business/end user validates against **business needs and real-world workflows** in a UAT/pre-prod environment with production-like data. Passing system tests does not guarantee passing UAT: the software can match the spec and still not fit how the business actually works.
- **Alpha testing** — done by internal staff (often non-QA employees) at the developer's site, in a controlled environment, before release.
- **Beta testing** — done by real external users in their own environment, uncontrolled. Feedback is collected, not scripted. Also called field testing.

Related modern practices worth mentioning: **canary release** (roll out to 1% of production traffic), **feature flags / dark launch**, and **A/B testing** — these have largely replaced long beta phases in SaaS.

CORE

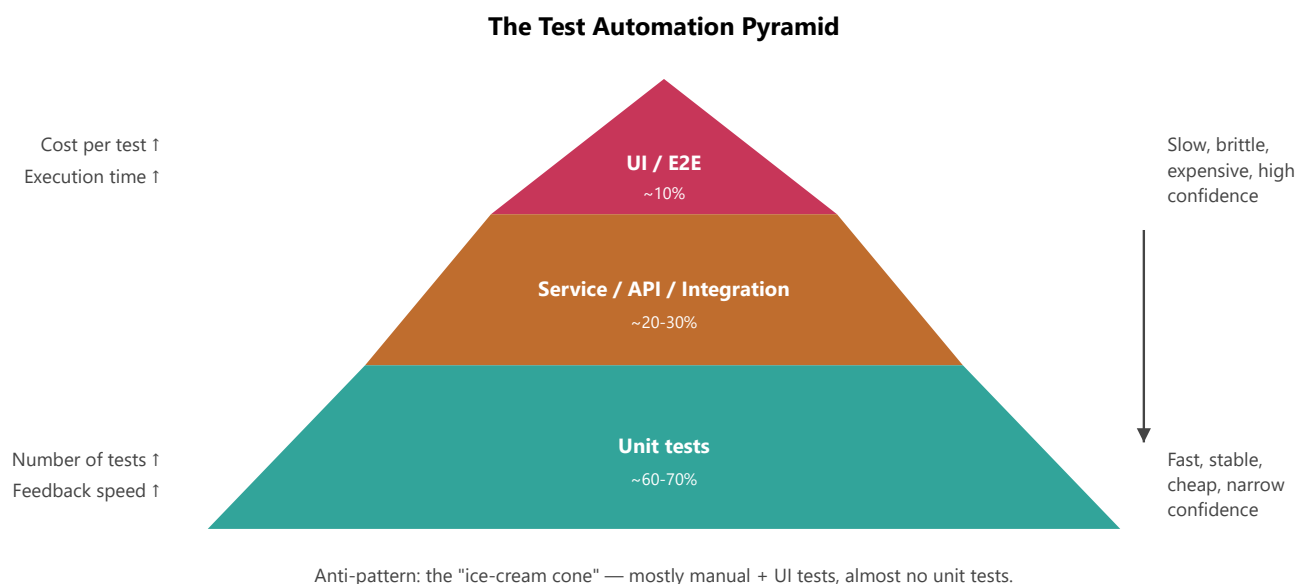


Figure 2 — Push tests down to the cheapest level that can still find the defect. If a bug can be caught by an API test, do not write a UI test for it.

Q22**Explain the test automation pyramid. Do you agree with it?**

Mike Cohn's pyramid says the bulk of automated tests should be fast unit tests, a smaller layer of service/API tests, and a thin layer of UI end-to-end tests. The reasoning is economics: as you go up, tests get slower, more brittle, more expensive to maintain, and harder to debug — but each one gives more end-user confidence.

The right answer to "do you agree?"

"Directionally yes — the principle is **push every check to the lowest layer that can still catch the defect**. But I treat the ratios as guidance, not law. On a micro-services back-end I lean heavily on contract and API tests, and the 'trophy' shape (heavy integration layer) fits better. What I do defend strictly is: the UI layer only covers **critical user journeys** — login, search, add-to-cart, checkout, payment — not field-level validations, which belong in unit/component tests."

Anti-patterns to name: the **ice-cream cone** (huge manual + UI layer, no unit tests → 4-hour suites, constant flakiness) and the **cupcake** (dev, QA and business each build their own duplicated layers).

CORE**ADVANCED**

What is shift-left testing? What about shift-right?

Shift-left = moving testing activities earlier in the lifecycle instead of treating testing as a phase after coding.

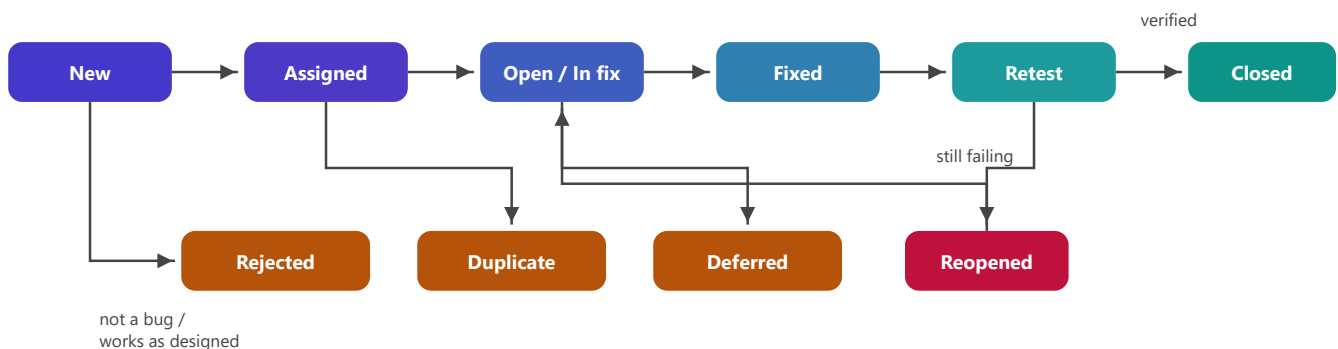
- QA joins requirement refinement and challenges acceptance criteria (the "three amigos": BA + Dev + QA).
- Static testing: requirement reviews, design reviews, code reviews, linting, SAST.
- Developers write unit & component tests; QA writes API tests before UI exists (against contracts/mocks).
- TDD / BDD / ATDD — tests written before the code.
- CI gates: no merge without passing tests and coverage thresholds.

Shift-right = continuing to test in production, because no pre-prod environment fully models reality: synthetic monitoring, canary releases, feature flags, A/B tests, real-user monitoring, chaos engineering, and observability-based alerting on error rates.

A modern QA answer covers both: "Shift left to prevent defects, shift right to detect what escaped. I owned the synthetic monitoring checks that re-used our Playwright login and checkout tests against production every 15 minutes."

ADVANCED

Defect / Bug Life Cycle



Only the **tester who raised it** should close a defect — never the developer.

Figure 3 — Defect life cycle. Learn the terminal states (Closed, Rejected, Duplicate, Deferred) and who moves the bug at each step.

Walk me through the defect life cycle.

1. **New** — tester logs the defect.
2. **Assigned** — triage/lead assigns it to a developer.
3. **Open / In Progress** — developer accepts and starts working. From here it can also go to Rejected, Duplicate or Deferred.
4. **Fixed** — code change done and merged to the test branch.
5. **Retest / Ready for QA** — deployed to QA; tester re-executes the original steps.
6. **Closed** — verified fixed by the tester. If it still fails → **Reopened** and back to the developer.

Terminal / side states

- **Rejected / Not a bug** — behaviour matches the requirement. Your response: re-read the spec; if you still disagree, take it to the BA/PO as a possible *requirement* defect.
- **Duplicate** — already reported; link to the original.
- **Deferred / Postponed** — genuine bug, but fixed in a later release (low severity, low priority, or high fix risk near a release).
- **Cannot reproduce** — usually missing environment/data/version detail in the report. Add logs, video, network HAR, exact build number and re-open.

Q25

Severity vs Priority — with examples of all four combinations.

Severity = technical impact of the defect on the system's functionality. Set by the **tester**.

Priority = urgency of fixing it from a business perspective. Set by the **PO / product manager / triage** (though the tester proposes it).

COMBINATION	EXAMPLE
High severity, High priority	Payment gateway fails for all users — the app crashes on checkout. Blocks revenue entirely.
High severity, Low priority	The application crashes when a user sets the system date to the year 2099, or on a legacy browser used by 0.01% of users. Technically fatal, commercially irrelevant.
Low severity, High priority	The company name is misspelt on the home page banner, or the logo is a competitor's. Nothing breaks, but it must ship today — brand/legal impact.
Low severity, Low priority	Tooltip text on a rarely-used admin screen has a typo or wrong alignment.

Typical severity scale

S1 Critical (system down / data loss, no workaround) · S2 Major (major feature broken, workaround is painful) · S3 Minor (feature partially works, easy workaround) · S4 Trivial (cosmetic).

The trap: many candidates say "severity is set by the tester and priority by the developer."
Priority is a **business** decision — PO/manager/triage board, not the developer.

CORE

TRAP

What does a good defect report contain?

The goal is that a developer can reproduce it in under 2 minutes without messaging you.

- **ID** and a **title** that states the problem, not the symptom vaguely: ✓ "Checkout: order total excludes GST when a coupon is applied" ✗ "Total wrong".
- **Environment**: build/version number, environment (QA2), browser + version, OS, device, user role, tenant.
- **Preconditions** and **test data** (the actual account/order ID used).
- **Steps to reproduce** — numbered, minimal, deterministic. Strip out irrelevant steps.
- **Expected result** — with the requirement/AC reference that says so.
- **Actual result**.
- **Evidence**: screenshot with the issue highlighted, screen recording for timing bugs, browser console errors, network HAR, application/server log snippet with timestamp and correlation ID.
- **Severity**, proposed **priority**, **reproducibility** (always / 3 out of 5 / once), and whether it is a **regression** (worked in build N-1).
- Link to the user story / requirement.

Saying "I always state whether it is a regression and which build last worked" is a strong senior signal — it immediately tells the developer where to bisect.

What is defect triage, defect leakage, defect age and DRE?

- **Defect triage** — a recurring meeting (QA lead, dev lead, PO, sometimes support) to review new defects and agree severity, priority, owner and target release. Its purpose is to make sure limited dev capacity is spent on the highest-risk defects.
- **Defect leakage** — defects that escaped a phase and were found later, especially in production. $\text{Leakage \%} = (\text{defects found in prod} / \text{total defects}) \times 100$.
- **Defect removal efficiency (DRE)** — $\text{DRE} = \text{defects found before release} / (\text{defects found before release} + \text{defects found after release}) \times 100$. Above 90% is generally considered healthy. This is *the* metric to quote when asked "how do you measure your testing effectiveness?"
- **Defect age** — time between defect detection and closure (or between injection and detection). High age points to triage/process bottlenecks.
- **Defect density** — defects per KLOC or per module; used to find the risky modules that deserve more testing.
- **Defect clustering** — the Pareto observation that ~80% of defects come from ~20% of modules. Use it to target regression effort.

ADVANCED

METRICS

A critical bug escaped to production. Walk me through what you do.

This is a behavioural + process question. Answer in a structured order and never blame a person.

1. **Contain first.** Confirm and reproduce it on prod, assess blast radius (how many users, is data being corrupted, is money at risk), and push for the fastest mitigation — feature flag off, rollback, or hotfix. Communicate to stakeholders immediately with facts.
2. **Fix and verify.** Write the failing test *first*, then verify the hotfix on staging with production-like data, then verify on prod after deployment.
3. **Root cause analysis (blameless).** Use the **5 Whys**. Why did it happen (code)? Why wasn't it caught (missing test / missing requirement / environment difference / test data gap)? Why wasn't it caught in review?
4. **Close the gap permanently.** Add the regression test to the automated suite, tag it, and make sure it runs in CI. If it was an environment/data difference, fix the environment parity. If a requirement was ambiguous, add it to the acceptance criteria checklist.
5. **Share the learning.** Post-mortem doc, and update the risk-based regression selection so that area gets more coverage.

The closing sentence that scores: *"Every production escape becomes a permanent automated test. That's how a suite earns its value — it encodes every mistake the team has already made once."*

CORE

BEHAVIOURAL

Q29

What is a Requirement Traceability Matrix (RTM) and why does it matter?

A document mapping each requirement to the test cases (and defects) that validate it.

REQ ID	REQUIREMENT	TEST CASES	STATUS	DEFECTS
REQ-AUTH-01	User logs in with email + password	TC_01, TC_02, TC_03	Pass	—
REQ-AUTH-03	Lock account after 3 failures	TC_07, TC_08	Fail	BUG-412

Three directions of traceability

- **Forward** — requirement → test case. Proves nothing is untested (**coverage gaps**).
- **Backward** — test case → requirement. Exposes tests that map to nothing (**scope creep / gold plating**).
- **Bi-directional** — both, which is what teams actually maintain.

Practical value: instant impact analysis ("requirement 12 changed — which tests do I re-run?"), audit/compliance evidence, and objective coverage reporting to management. In modern tools this lives in Jira + Xray/Zephyr rather than in Excel.

CORE

Q30

Test coverage vs code coverage. Is 100% coverage a good goal?

- **Code coverage** — a white-box, tool-measured metric: what percentage of lines/branches were executed by tests (JaCoCo, Istanbul/nyc). It measures *execution*, not *verification*.
- **Test coverage** — a broader, qualitative measure of how much of the requirements, risks, platforms, and user journeys are covered.

Why 100% code coverage is a misleading goal: a test can execute every line and assert nothing. Coverage tells you what is *definitely untested*; it never tells you what is well tested. I use it as a **gap-finder** (which files have 0%?) and a **ratchet** (coverage must not drop in a PR), not as a target to chase. The technique that actually measures assertion quality is **mutation testing** (PIT for Java, Stryker for JS/TS): it changes the code and checks whether your tests fail.

ADVANCED

TRAP

Q31

Describe Agile/Scrum ceremonies and the QA role in each.

CEREMONY	PURPOSE	WHAT QA DOES
Backlog refinement / grooming	Clarify and size upcoming stories	Challenge acceptance criteria, ask edge-case questions, flag testability and test-data needs, estimate test effort
Sprint planning	Commit to sprint scope	Confirm each story has AC + a definition of done that includes automation; raise environment/dependency risks
Daily stand-up	Sync & unblock	Report progress, raise blockers early (build not deployed, data missing), highlight risky areas
Sprint review / demo	Show the increment to stakeholders	Demo tested functionality, report known limitations honestly
Retrospective	Improve the process	Bring data: escaped defects, flaky tests, build failures, and propose one concrete improvement

Also mention the **Three Amigos** session (BA/PO + Dev + QA before development starts) — it is the single highest-ROI shift-left practice and interviewers love hearing it.

CORE

User story — "As a <role>, I want <goal>, so that <benefit>." It captures intent, not implementation. Good stories follow **INVEST**: Independent, Negotiable, Valuable, Estimable, Small, Testable. As QA, the "T" is your veto: if you cannot describe how you'd verify it, it is not ready.

Acceptance criteria — the conditions that make the story acceptable, often in Given/When/Then form:

```
Given I am a registered user on the login page
When I enter a valid email and an incorrect password three times
Then my account is locked
And I see "Your account is locked. Try again after 30 minutes"
```

Definition of Ready — a checklist a story must satisfy before it enters a sprint: AC written, dependencies known, designs attached, test data identified, estimable.

Definition of Done — a checklist before it is "done": code merged, unit tests written, code reviewed, functional testing passed, automation added to the regression suite, no open critical/high defects, documentation updated, deployed to QA/staging.

Say: "On my team, 'automated regression test added and green in CI' is part of the Definition of Done — that's how we prevented the suite from lagging behind development."

Q33

How do you estimate testing effort?

Name at least three techniques and say which you use:

- **Work Breakdown Structure (WBS)** — decompose into tasks (analysis, design, data setup, execution, automation, reporting) and estimate each. Most defensible.
- **Three-point / PERT** — $E = (\text{Optimistic} + 4 \times \text{Most-likely} + \text{Pessimistic}) / 6$. Good when uncertainty is high; it gives you a number you can justify.
- **Planning poker / story points** — relative sizing in Scrum, using Fibonacci (1,2,3,5,8,13). The team estimates the *whole* story including testing, not dev and QA separately.
- **Test case point / function point** — formal, used in fixed-bid services projects.
- **Analogy-based** — "this module is like the one we did last release, which took 8 days."

Always add a **buffer for retesting and regression** (a defect isn't done when it's found — it must be retested, often twice) and a contingency of 10–20% for environment downtime.

ADVANCED

Q34

What is risk-based testing? How do you prioritise when there is no time to test everything?

Risk-based testing allocates test effort proportional to risk, where $\text{Risk} = \text{Likelihood of failure} \times \text{Business impact of failure}$.

How I do it practically

1. List the features/modules in scope.
2. Score **likelihood** (1–5): code churn in this area, complexity, new vs mature, number of past defects, whether a new/junior dev wrote it, number of integrations.
3. Score **impact** (1–5): revenue, legal/compliance, data integrity, number of users affected, reputational damage.
4. Multiply → risk score → sort. Test the top band deeply (all techniques), the middle band normally, the bottom band with a smoke check only.
5. Explicitly write down what you are **not** testing and get the PO to acknowledge it. That converts a hidden gamble into an accepted business decision.

When they ask "you have 2 days to test a release that needs 5 — what do you do?", this is the answer. Never say "I'll work overtime and test everything." Say: risk-ranking, critical path first, communicate residual risk in writing, propose a phased/canary release.

CORE

ADVANCED

Automate

- Repetitive regression tests that run every release.
- Critical business flows (login, search, cart, checkout, payment) — the smoke suite.
- Data-driven tests with many input combinations.
- Tests that are tedious/error-prone manually (long calculations, large data verification).
- Cross-browser / cross-device matrices.
- Tests needed as CI gates for fast feedback (API + unit level especially).
- Stable features whose requirements are not changing.

Do not automate

- Tests that run once (one-off migration verification).
- Features still under heavy UI/requirement churn — the script will be rewritten every sprint.
- Exploratory, usability, look-and-feel and "does this feel right" testing — human judgement.
- Tests needing physical interaction (scanning, printing, biometric hardware) unless real device farms exist.
- Anything with very low risk and very high automation cost — check the ROI first.
- CAPTCHA / OTP flows (bypass them in test environments instead — that is the correct answer, not "automate the OTP email").

How do you calculate and justify the ROI of test automation?

Simple, defensible model:

Manual cost per cycle = T_m (hours to run the suite manually)
 Automation build cost = T_a (one-time hours to script it)
 Automation maintenance = T_{mt} (hours per cycle to keep it green)
 Automated execution cost = T_e (near zero, runs on CI)

Break-even cycles $N = T_a / (T_m - T_{mt} - T_e)$
 ROI after n cycles = $(n \times T_m - (T_a + n \times (T_{mt} + T_e))) / (T_a + n \times (T_{mt} + T_e))$

Worked example: the regression suite takes 40 hours manually. Scripting it costs 200 hours, maintenance is 4 hours per cycle. Break-even = $200 / (40 - 4) \approx 5.6$ cycles. With fortnightly releases, it pays for itself in about 3 months.

Benefits you cannot put in that formula — mention them

- Feedback in 20 minutes instead of 3 days → cheaper defect fixes.
- Enables CI/CD and more frequent releases (a business capability, not a cost saving).
- Frees manual testers for exploratory testing, which finds different (often more severe) defects.
- Consistency — no human step-skipping at 6pm on a Friday.

Do **not** claim automation reduces headcount. Experienced interviewers see that as naive. Claim it changes what the team spends its time on.

ADVANCED

Q37

How do you manage test data?

Test data is the #1 cause of flaky and unmaintainable suites, so this question separates real practitioners from theory.

- **Self-sufficient tests** — each test creates the data it needs (preferably through an **API** in setup, which is 10–50× faster and more reliable than doing it through the UI) and cleans it up afterwards. This is my default.
- **Unique data generation** — never hardcode `test@test.com`. Use `user_" + System.currentTimeMillis()` or a Faker library so parallel runs don't collide.
- **Externalised static data** — reference data (product codes, tax rates) in JSON/CSV/Excel, read by the data provider, never in the code.
- **Seeded/reset environments** — a known DB snapshot restored before a suite run, or a Docker/Testcontainers database per run.
- **Masked production data** — realistic volumes and shapes for performance/UAT, with PII anonymised. Mention **GDPR/DPDP compliance** here; it is a strong senior signal.
- **Never depend on data another test created** — that creates execution-order coupling and kills parallelism.

CORE**ADVANCED**

Q38

What is a test environment, and what problems have you faced with environments?

A test environment = application build + configuration + database + dependent services + network + test data + access credentials, mirroring production as closely as budget allows.

Common real problems (talk about these — it shows experience)

- **Environment drift** — QA has a different config/version than prod, so "works in QA, fails in prod". Fix: infrastructure as code, containerisation, config in version control.
- **Shared environment contention** — two teams testing on the same env corrupt each other's data. Fix: ephemeral per-branch environments, or namespaced/tenant-isolated data.
- **Third-party dependency downtime** — the payment sandbox is down. Fix: service virtualisation with WireMock/MockServer plus a small suite of real-integration tests run less often.
- **Data reset breaking overnight runs** — schedule a data seed step as part of the pipeline, not manually.

CORE

Q39

Explain performance test types: load, stress, soak, spike, volume.

TYPE	WHAT YOU DO	WHAT IT FINDS
Load	Apply the expected concurrent user load for a normal period	Whether response-time SLAs hold at expected traffic
Stress	Increase load beyond capacity until the system breaks	The breaking point and whether it degrades gracefully or corrupts data
Soak / Endurance	Sustain a moderate load for 8–72 hours	Memory leaks, connection-pool exhaustion, log-disk filling, gradual degradation
Spike	Sudden jump from 100 to 10,000 users and back	Auto-scaling behaviour, queue overflow, recovery time
Volume	Very large data sets (10M rows, 500MB uploads)	Query timeouts, pagination, index problems, batch job runtimes
Scalability	Measure capacity as you add nodes/resources	Whether scaling is linear; where the bottleneck moves to

Key metrics to name: **response time percentiles (p90/p95/p99, not average)**, throughput (req/s), error rate, concurrent users, CPU/memory/GC, DB connection pool usage.

Saying "I look at p95, not average" is a small phrase that signals real performance exposure — averages hide the users who are actually suffering.

ADVANCED

Accessibility (WCAG 2.1, level AA is the usual legal target)

Four principles — **POUR**: Perceivable, Operable, Understandable, Robust. Practical checks:

- All images have meaningful `alt` text; decorative images have `alt=""`.
- Colour contrast ratio $\geq 4.5:1$ for normal text.
- Full keyboard navigation, visible focus indicator, logical tab order, no keyboard traps.
- Form inputs have associated `<label>`; errors are announced, not only shown in red.
- Correct heading hierarchy and ARIA roles; screen-reader testing with NVDA/VoiceOver.
- Automatable with **axe-core** — integrates into both Selenium (`axe-selenium-java`) and Playwright (`@axe-core/playwright`). Note that automated tools catch only ~30–40% of issues; the rest needs manual checking.

Security — OWASP Top 10 awareness

Broken access control (test by changing an ID in the URL — IDOR), injection (SQL/NoSQL/command), XSS (stored and reflected), broken authentication (session fixation, weak password policy, token not invalidated on logout), security misconfiguration, sensitive data exposure (PII in URLs, logs or local storage), CSRF, SSRF, vulnerable dependencies (`npm audit` , OWASP Dependency-Check), insufficient logging.

ADVANCED

Answer in **categories**, out loud, and ask clarifying questions first ("is there SSO? MFA? is this web only or mobile too?"). Categories beat a random list.

1. Functional — positive

Valid credentials log in and land on the right page; "Remember me" persists the session; redirect back to the originally requested page after login; SSO/social login; MFA/OTP happy path.

2. Functional — negative

Wrong password, non-existent user, empty fields, only spaces, correct password wrong case, deactivated/locked/expired account, expired password, account lockout after N attempts and its reset, generic error message (must not reveal whether the email exists).

3. Field-level / boundary

Min/max lengths, leading/trailing spaces trimmed, email format validation, special characters, Unicode/emoji, copy-paste, autofill, password masking + show/hide toggle, tab order, Enter key submits.

4. Security

SQL injection `' OR '1'='1`, XSS payload in the username, password never sent in the URL/query string, HTTPS enforced, password not in local storage or logs, session token regenerated after login (session fixation), token invalidated on logout, back button after logout must not restore the session, brute-force rate limiting, CAPTCHA after N failures, concurrent-session policy.

5. UI / usability / accessibility

Labels, placeholder vs label, error message placement, keyboard-only flow, screen reader announces errors, contrast, responsive layout at 320px, zoom to 200%.

6. Compatibility & performance

Chrome/Firefox/Safari/Edge, iOS/Android, slow 3G, login API response time under load, behaviour when the auth service is down (graceful error, not a stack trace).

Finish with: *"I'd automate the positive path plus lockout and session-invalidation as regression, keep the CAPTCHA path manual or bypassed in test config, and run the accessibility scan with axe*

in Cl." That closing line ties manual thinking to automation strategy — exactly the profile they're hiring.

CORE

TRAP

SECTION 2

Java Essentials for Automation

Q42 – Q65 · Interviewers test Java to see whether you can *design* a framework or only *use* one. Every answer below is anchored to an automation use case, because that is what they want to hear.

Do not give the textbook definition alone. Give the definition **plus where it lives in a Selenium framework**.

PILLAR	DEFINITION	WHERE I USE IT
Encapsulation	Bundling data with the methods that operate on it, and hiding internal state behind accessors	Page Objects: <code>WebElement</code> locators are <code>private</code> ; only business methods like <code>login(user, pass)</code> are public. Tests cannot touch locators directly.
Inheritance	A class acquires the fields/behaviour of a parent class (<code>extends</code>)	<code>BasePage</code> holds the driver, waits and common helpers; every page extends it. <code>BaseTest</code> holds setup/teardown; every test class extends it.
Polymorphism	One interface, many implementations — compile-time (overloading) and runtime (overriding)	<code>WebDriver driver = new ChromeDriver();</code> — the reference type is the interface, so a factory can return Chrome, Firefox or Edge and the tests never change. Also <code>click(WebElement)</code> / <code>click(By)</code> overloads.
Abstraction	Exposing <i>what</i> something does while hiding <i>how</i>	A test calls <code>checkoutPage.payWith("UPI")</code> . It has no idea about iframes, waits or retries inside. Implemented with interfaces and abstract classes.

```

public class LoginPage extends BasePage {           // inheritance
    @FindBy(id = "username") private WebElement user; // encapsulation
    @FindBy(id = "password") private WebElement pass;
    @FindBy(css = "button[type='submit']") private WebElement signIn;

    public LoginPage(WebDriver driver) { super(driver); }

    public HomePage login(String u, String p) {      // abstraction
        type(user, u);                               // helper from BasePage
        type(pass, p);
        click(signIn);
        return new HomePage(driver);
    }
}

```

Q43

Abstraction vs Encapsulation — they sound the same. What is the actual difference?

- **Abstraction** solves a problem at **design** level: *what* should be exposed. Achieved with interfaces and abstract classes. It hides **complexity/implementation**.
- **Encapsulation** solves it at **implementation** level: *how* to protect the exposed thing. Achieved with access modifiers and getters/setters. It hides **data**.

One-liner: **abstraction hides the implementation; encapsulation hides the data**. A TV remote is abstraction (you press "volume up", not knowing the circuitry). The sealed plastic casing with screws is encapsulation.

CORE

TRAP

	OVERLOADING	OVERRIDING
Also called	Compile-time / static polymorphism	Runtime / dynamic polymorphism
Where	Same class (or a subclass)	Subclass, method inherited from parent
Signature	Must differ: number, type or order of parameters	Must be identical
Return type	Can differ (but cannot differ <i>only</i> by return type)	Same, or a covariant subtype
Access modifier	Anything	Cannot be more restrictive than the parent
Exceptions	Anything	Cannot throw new/broader checked exceptions (unchecked is fine)
<code>static</code> / <code>private</code> / <code>final</code>	Can be overloaded	Cannot be overridden (static is <i>hidden</i> , not overridden)
Resolved	By the compiler, from the reference type	By the JVM, from the actual object type

```
// Overloading – a reusable click helper
public void click(WebElement el)      { el.click(); }
public void click(By locator)         { driver.findElement(locator).click(); }
public void click(WebElement el, int timeoutSec) { /* wait then click */ }

// Overriding – different wait behaviour for a slow page
class BasePage      { protected void waitForLoad() { /* 10s */ } }
class ReportsPage extends BasePage {
    @Override protected void waitForLoad() { /* 60s, report generation */ }
}
```

Classic trick question: "Can we override a static method?" No. If you declare the same static method in a subclass it is **method hiding**, resolved at compile time by the reference type —

not polymorphism.

CORE

TRAP

Q45

Interface vs Abstract class. When do you choose which?

	INTERFACE	ABSTRACT CLASS
Multiple inheritance	A class can implement many	Only one can be extended
Fields	Only <code>public static final</code> constants	Any instance fields with any modifier
Constructor	No	Yes (called via <code>super()</code>)
Methods	abstract; plus <code>default</code> & <code>static</code> (Java 8) and <code>private</code> (Java 9)	Abstract and concrete, any access level
Represents	A capability / contract ("can do")	A common base identity ("is a") with shared state

The decision rule

"Do the implementations share **state and code**? → abstract class. Do unrelated classes need to share a **contract**? → interface." In practice both are used together: `WebDriver` is an interface (contract), and `RemoteWebDriver` is a class implementing it that `ChromeDriver` extends.

Java 8 follow-up: "If interfaces have default methods now, why keep abstract classes?"

Because interfaces still cannot hold instance state or constructors. And the **diamond problem** with two conflicting default methods must be resolved explicitly:

```
InterfaceA.super.method();
```

CORE

ADVANCED

static members belong to the **class**, not to an instance — one copy shared by everyone, loaded when the class is loaded, accessible without creating an object.

- **static variable** — shared counter, config values.
- **static method** — utility functions (`ConfigReader.get("url")`). Cannot use **this** or instance members directly.
- **static block** — runs once when the class loads; used to initialise static config.
- **static nested class** — used in the Builder pattern.

The important interview answer: a static `WebDriver driver`; works fine until you run tests in **parallel**. Then all threads share one browser instance: thread A navigates while thread B is asserting, and you get random `NoSuchElementExceptions` and impossible failures. The correct solution is `ThreadLocal<WebDriver>` — a static holder that gives each thread its own value (see Q61).

```
// WRONG for parallel execution
public static WebDriver driver;

// RIGHT
private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();
public static WebDriver get()           { return DRIVER.get(); }
public static void set(WebDriver d)     { DRIVER.set(d); }
public static void unload()             { DRIVER.remove(); } // avoid memory leak
```

CORE

ADVANCED

Q47**final vs finally vs finalize.**

- **final** — a modifier. **final** variable = constant (cannot be reassigned; the *object* it points to can still mutate). **final** method = cannot be overridden. **final** class = cannot be extended (e.g. **String**).
- **finally** — a block that always executes after try/catch, whether or not an exception was thrown. Used for cleanup: **driver.quit()** , closing streams and DB connections. It runs even if the try block has a **return** . (It does *not* run if the JVM exits via **System.exit()** or the thread is killed.)
- **finalize()** — a legacy **Object** method the garbage collector *might* call before reclaiming an object. Unpredictable, **deprecated since Java 9**, never rely on it. Use **try-with-resources** / **AutoCloseable** instead.

```
try (BufferedReader br = new BufferedReader(new FileReader("data.csv"))) {  
    return br.readLine();                // resource auto-closed, no finally needed  
} catch (IOException e) {  
    throw new FrameworkException("Could not read test data", e);  
}
```

CORE**TRAP**

	STRING	STRINGBUILDER	STRINGBUFFER
Mutable	No	Yes	Yes
Thread-safe	Yes (because immutable)	No	Yes (synchronized)
Performance	Slow for repeated concatenation	Fastest	Slower than StringBuilder
Use when	Fixed text, map keys	Building strings in a loop (single thread) — the normal choice	Building strings shared across threads (rare)

Why immutable?

- **String pool** — identical literals share one object in the heap's string pool, saving huge memory. That is only safe if nobody can mutate them.
- **Security** — file paths, DB URLs and credentials are passed as Strings; immutability prevents a reference being changed after a security check.
- **Thread safety** for free, and a cached `hashCode()` which makes String an ideal `HashMap` key.

```
String a = "qa";           // goes to the string pool
String b = "qa";           // same pooled object -> a == b is true
String c = new String("qa");// new heap object      -> a == c is FALSE
System.out.println(a.equals(c)); // true - always compare with equals()
```

Q49

== vs equals(), and the equals/hashCode contract.

- `==` compares **references** for objects (do both variables point to the same memory?) and **values** for primitives.
- `equals()` compares **content**, if the class overrides it. The default `Object.equals()` is just `==`.

The contract

If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` **must** be true. The reverse is not required (unequal objects may share a hash — a collision). If you override `equals()` you must override `hashCode()`, otherwise your object will "disappear" inside a `HashMap` / `HashSet`: it is stored in bucket A, and the lookup goes to bucket B.

```
Integer x = 127, y = 127; System.out.println(x == y); // true (Integer cache -128..127)
Integer p = 128, q = 128; System.out.println(p == q); // false (outside the cache!)
```

CORE

TRAP

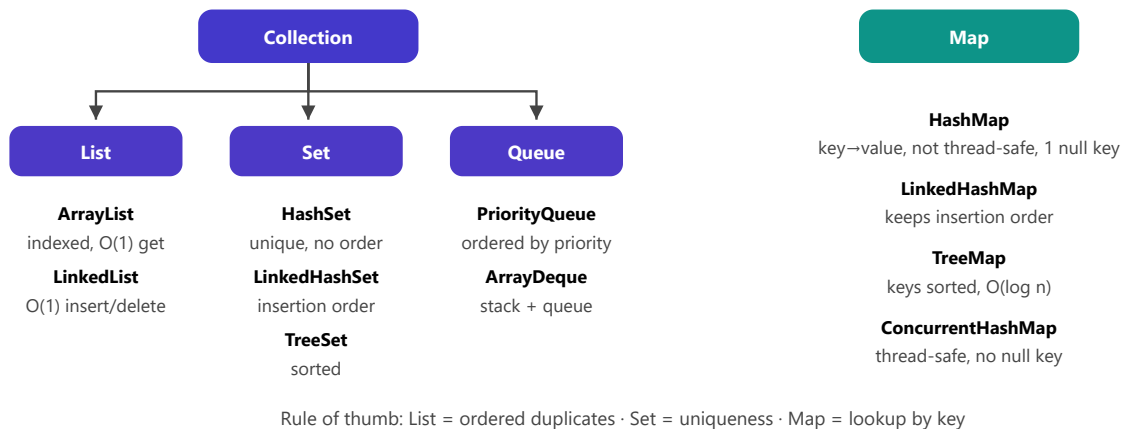
Java Collections — what to pick and why

Figure 4 — Collections cheat sheet. Interviewers usually ask "which one and why", not the full hierarchy.

Q50

Explain the Collections framework — List, Set, Map — and where you use each in automation.

- **List** — ordered, allows duplicates, index access. Automation use: `List<WebElement> rows = driver.findElements(By.cssSelector("table tr"));` and collecting all product names to verify sorting.
- **Set** — no duplicates. Automation use: `Set<String> handles = driver.getWindowHandles();` (window handles are unique), and detecting duplicate entries in a UI list.
- **Map** — key→value. Automation use: test data (`Map<String,String> row` from an Excel/JSON row), request headers, and config properties.

```
// verify a product list is sorted ascending, without mutating the original
List<String> actual = driver.findElements(By.cssSelector(".product-name"))
    .stream().map(WebElement::getText).collect(Collectors.toList());
List<String> expected = new ArrayList<>(actual);
Collections.sort(expected);
Assert.assertEquals(actual, expected, "Products are not sorted A-Z");
```

NEED	PICK
Fast random access, mostly reads	<code>ArrayList</code>
Frequent insert/delete in the middle	<code>LinkedList</code>
Uniqueness, order irrelevant	<code>HashSet</code>
Uniqueness + insertion order	<code>LinkedHashSet</code>
Sorted keys	<code>TreeMap</code> / <code>TreeSet</code>
Shared across threads	<code>ConcurrentHashMap</code> / <code>CopyOnWriteArrayList</code>

CORE

Q51**ArrayList vs LinkedList, and HashMap vs Hashtable vs ConcurrentHashMap.**

	ARRAYLIST	LINKEDLIST
Structure	Dynamic array (grows $\sim 1.5\times$)	Doubly linked list
<code>get(i)</code>	$O(1)$	$O(n)$
Insert/delete in middle	$O(n)$ — shifts elements	$O(1)$ once you are at the node
Memory	Less (just data + spare capacity)	More (each node stores 2 pointers)
Verdict	Default choice $\sim 95\%$ of the time	Only for heavy queue/deque style use

	HASHMAP	HASHTABLE	CONCURRENTHASHMAP
Thread-safe	No	Yes — whole map locked	Yes — fine-grained locking
Performance under threads	Fast but unsafe	Very slow	Fast and safe
null key / values	1 null key, many null values	None	None
Status	Standard	Legacy, avoid	Use for concurrency

CORE

Q52

How does HashMap work internally?

1. A HashMap holds an array of buckets (default capacity **16**, load factor **0.75**).
2. On `put(k,v)` it calls `k.hashCode()`, applies a supplemental hash (spreads the high bits), and computes the index as `hash & (n-1)`.
3. If the bucket is empty → store a new `Node`. If occupied (a **collision**) → walk the chain comparing with `hashCode()` then `equals()`. If an equal key exists, the value is **replaced**; otherwise the node is appended.
4. When a bucket's chain exceeds **8** nodes (and capacity ≥ 64), Java 8+ converts that chain into a **balanced red-black tree**, improving worst-case lookup from $O(n)$ to $O(\log n)$. It reverts to a list below 6 nodes.
5. When `size > capacity × loadFactor` (12 for the default 16), the map **resizes** to double capacity and rehashes all entries — an expensive operation, which is why you pre-size a map when you know the volume.

Follow-up: "What if you use a mutable object as a key and then change it?" Its `hashCode` changes, so it now maps to a different bucket — the entry becomes unreachable (a memory leak that `get()` can never find). That is exactly why `String` is immutable and is the preferred key type.

ADVANCED**TRAP**

Q53**Explain exception handling: checked vs unchecked, throw vs throws, and custom exceptions.**

Hierarchy: `Throwable` → `Error` (JVM problems: `OutOfMemoryError`, `StackOverflowError` — do not catch) and `Exception` → `RuntimeException` (unchecked) and everything else (checked).

- **Checked** — the compiler forces you to handle or declare them. `IOException`, `SQLException`, `InterruptedException`. They represent recoverable, expected external failures.
- **Unchecked** — `NullPointerException`, `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`, and **all Selenium exceptions** (they extend `WebDriverException` → `RuntimeException`). That's why you are not forced to try/catch `findElement`.
- **throw** — actually throws an instance: `throw new FrameworkException("msg");`. **throws** — declares in the method signature that the caller must deal with it.

```
public class FrameworkException extends RuntimeException {
    public FrameworkException(String message, Throwable cause) { super(message, cause); }
}

// Wrap low-level failures with business context – this is what makes
// a failed CI run debuggable at 2 a.m.
public void uploadInvoice(Path file) {
    try {
        driver.findElement(By.id("file")).sendKeys(file.toAbsolutePath().toString());
    } catch (NoSuchElementException e) {
        throw new FrameworkException("Upload control missing on Invoice page. URL="
            + driver.getCurrentUrl(), e); // keep the cause, never swallow it
    }
}
```

Never write an empty catch block in a test framework. A swallowed exception turns a real failure into a silent false PASS — which is worse than no test at all. If you must ignore one, log it and comment why.

CORE

Q54

Can you have try without catch? What is multi-catch and try-with-resources? Does finally always run?

- **try without catch** — yes, if there is a `finally` (or it is try-with-resources). `try { } finally { }` is legal.
- **Multi-catch** (Java 7+): `catch (NoSuchElementException | TimeoutException e)` — the variable is implicitly final. Catch blocks must go from most specific to most general, or the compiler errors.
- **try-with-resources** — any object implementing `AutoCloseable` declared in the parentheses is closed automatically in reverse order, even on exception. Cleaner than a finally block.
- **Does finally always run?** Almost always — including when `try` returns. Exceptions: `System.exit()`, JVM crash, infinite loop or thread kill inside try, or a power failure.

```
public int demo() {  
    try { return 1; }           // value 1 is computed and held  
    finally { System.out.println("always runs"); } // prints, then 1 is returned  
}  
  
// Warning: a `return` inside finally would OVERRIDE the try's return and  
// swallow any pending exception - never do that.
```

CORE**TRAP**

Java 8 features you actually use in automation — lambdas, functional interfaces, streams.

- **Lambda expression** — an anonymous implementation of a functional interface: `(a, b) -> a + b`. Selenium's `ExpectedCondition` and `FluentWait` take one.
- **Functional interface** — an interface with exactly one abstract method, marked `@FunctionalInterface`. Built-ins: `Predicate<T>` (`T→boolean`), `Function<T,R>`, `Consumer<T>` (`T→void`), `Supplier<T>` (`→T`), `BiFunction`.
- **Stream API** — declarative processing of collections. **Intermediate** operations (`filter`, `map`, `sorted`, `distinct`, `limit`) are lazy; **terminal** operations (`collect`, `forEach`, `count`, `anyMatch`, `reduce`) trigger execution.
- **Method reference** — `WebElement::getText` instead of `e -> e.getText()`.
- **Optional, default methods, new Date/Time API** (`LocalDate`, `Duration` — used everywhere in modern Selenium waits).

```
// Real framework usage: extract, filter and assert in one readable block
List<String> outOfStock = driver.findElements(By.cssSelector(".product-card"))
    .stream()
    .filter(card -> card.findElement(By.className("stock")).getText().contains("Out"))
    .map(card -> card.findElement(By.className("title")).getText())
    .sorted()
    .collect(Collectors.toList());

// Custom wait with a Lambda
new WebDriverWait(driver, Duration.ofSeconds(15))
    .until(d -> d.findElements(By.cssSelector(".spinner")).isEmpty());

// Grouping test data
Map<String, List<Order>> byStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::getStatus));
```

If asked "difference between map and flatMap": map transforms 1→1; flatMap transforms 1→many and flattens the nested streams into one (useful for a list of lists).

CORE

ADVANCED

Q56

What is Optional and how does it help?

A container that may or may not hold a non-null value. It makes "this might be absent" part of the **type signature** instead of an undocumented surprise, which reduces

`NullPointerException` S.

```
Optional<WebElement> banner = driver.findElements(By.id("promo")).stream().findFirst();
banner.ifPresent(WebElement::click); // do only if present
String text = banner.map(WebElement::getText).orElse(""); // safe default
WebElement el = banner.orElseThrow(() -> new FrameworkException("Promo banner missing"));
```

Do not call `.get()` without checking `isPresent()` — that just recreates the NPE with extra steps. And do not use `Optional` for method parameters or entity fields; it is designed for **return types**.

ADVANCED

Q57

What are Generics and why do they matter in a framework?

Generics give **compile-time type safety** and remove casting. `List<String>` cannot accidentally receive an `Integer`. Java implements them via **type erasure** — the type parameter is removed at compile time, which is why you cannot do `new T[]` or `instanceof List<String>`.

```
// A generic page-navigation helper: returns the page object you asked for
public <T extends BasePage> T navigateTo(Class<T> pageClass) {
    try {
        return pageClass.getDeclaredConstructor(WebDriver.class).newInstance(driver);
    } catch (Exception e) {
        throw new FrameworkException("Cannot instantiate " + pageClass.getSimpleName(), e);
    }
}
// Usage - fully type-safe, no cast:
LoginPage login = navigateTo(LoginPage.class);
```

Wildcards: `<? extends T>` = read-only "producer"; `<? super T>` = write "consumer".

Mnemonic **PECS** — Producer Extends, Consumer Super.

ADVANCED

Q58

Access modifiers, and what happens if a constructor is private?

MODIFIER	SAME CLASS	SAME PACKAGE	SUBCLASS (OTHER PKG)	ANYWHERE
<code>private</code>	✓	✗	✗	✗
<i>default</i> (no keyword)	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

A **private constructor** prevents external instantiation. Used for: **singletons** (DriverManager, ConfigReader), **static utility classes** (so nobody writes `new StringUtils()`), and the **builder** pattern.

```
public final class ConfigReader {
    private static final Properties PROPS = new Properties();
    static {
        // runs once at class load
        try (InputStream in = ConfigReader.class.getResourceAsStream("/config.properties")) {
            PROPS.load(in);
        } catch (IOException e) { throw new ExceptionInInitializerError(e); }
    }
    private ConfigReader() { } // no instances allowed
    public static String get(String key) {
        return System.getProperty(key, PROPS.getProperty(key)); // -D overrides file
    }
}
```

That `System.getProperty(key, fileValue)` line is worth pointing out in an interview: it lets you override any config from the command line (`mvn test -Dbrowser=firefox`) without touching the file — essential for CI.

CORE

Q59**Constructor rules, this vs super, and constructor chaining.**

- A constructor has the class's name, **no return type**, and runs on object creation. If you write none, Java gives a no-arg default — but only if you declared **no** constructor at all.
- Constructors can be **overloaded** but never overridden or inherited.
- `this(...)` calls another constructor of the same class; `super(...)` calls the parent's. Either must be the **first statement**, and you cannot use both.
- If you don't write `super(...)`, the compiler inserts an implicit `super()`. So the parent is always constructed first.

```
public class BasePage {
    protected WebDriver driver;
    protected WebDriverWait wait;
    public BasePage(WebDriver driver) { this(driver, 20); }           // this()
    public BasePage(WebDriver driver, int timeoutSec) {
        this.driver = driver;                                         // shadowing resolved by this.
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(timeoutSec));
        PageFactory.initElements(driver, this);
    }
}

public class CartPage extends BasePage {
    public CartPage(WebDriver driver) { super(driver, 45); }         // super()
}
```

CORE

Implement a thread-safe Singleton. Where do you use it in a framework?

Singleton = exactly one instance for the whole JVM, with a global access point. In frameworks: config reader, logger, extent-report manager, DB connection pool.

```
// 1. Bill Pugh (holder idiom) – lazy, thread-safe, no synchronization cost
public class ExtentManager {
    private ExtentManager() { }
    private static class Holder {
        static final ExtentReports INSTANCE = build();
    }
    public static ExtentReports get() { return Holder.INSTANCE; }
    private static ExtentReports build() { /* configure reporter */ return new ExtentReports(); }
}

// 2. Double-checked locking – needs `volatile`, or another thread can see
//    a half-constructed object due to instruction reordering
public class Config {
    private static volatile Config instance;
    private Config() { }
    public static Config get() {
        if (instance == null) {
            synchronized (Config.class) {
                if (instance == null) instance = new Config();
            }
        }
        return instance;
    }
}

// 3. Enum singleton – simplest, serialization- and reflection-safe (Joshua Bloch's pick)
public enum Env { INSTANCE; public String baseUrl() { return "https://qa.example.com"; } }
```

Very common follow-up: "Should the WebDriver be a singleton?" No. A singleton means one driver for the whole JVM, which breaks parallel execution. Use a **ThreadLocal-backed factory** instead — conceptually "one instance per thread", not "one per JVM".

ADVANCED

TRAP

What is ThreadLocal and why is it essential for parallel Selenium execution?

`ThreadLocal<T>` gives every thread its own independently initialised copy of a variable. Internally each `Thread` object carries a `ThreadLocalMap` keyed by the `ThreadLocal` instance — so `get()` returns the calling thread's value.

In TestNG parallel mode, each test method runs on its own thread. Each thread must drive its own browser, but the page objects and utilities need to reach "the driver" without passing it through every method signature. `ThreadLocal` solves exactly that.

```
public final class DriverFactory {
    private static final ThreadLocal<WebDriver> TL = new ThreadLocal<>();

    public static void init(String browser) {
        WebDriver driver = switch (browser.toLowerCase()) {
            case "firefox" -> new FirefoxDriver();
            case "edge"    -> new EdgeDriver();
            default        -> new ChromeDriver();
        };
        driver.manage().window().maximize();
        TL.set(driver);
    }

    public static WebDriver get() {
        WebDriver d = TL.get();
        if (d == null) throw new IllegalStateException("Driver not initialised for thread "
            + Thread.currentThread().getName());
        return d;
    }

    public static void quit() {
        WebDriver d = TL.get();
        if (d != null) { d.quit(); TL.remove(); } // remove() prevents a memory leak
    }
}
```

Always call `TL.remove()` in teardown. Thread pools reuse threads; if you don't remove, the stale driver reference stays attached to the pooled thread and leaks memory (and the next test may grab a dead driver).

ADVANCED

CORE

Q62

Basic multithreading: thread lifecycle, synchronized, and why `Thread.sleep()` is bad in tests.

Lifecycle: New → Runnable → Running → (Blocked / Waiting / Timed-waiting) → Terminated.

- **Creating a thread:** extend `Thread`, or implement `Runnable` / `Callable` (preferred — leaves your inheritance slot free) and submit to an `ExecutorService`.
- `synchronized` — a lock ensuring only one thread executes a block/method on that monitor at a time. Prevents race conditions on shared mutable state (e.g. a shared report or a counter).
- `wait()/notify()` vs `sleep()` : `wait()` releases the lock, `sleep()` does not.

Why `Thread.sleep()` is an anti-pattern in tests: it is a fixed unconditional pause. On a fast run it wastes seconds (a 200-test suite with 3s sleeps wastes 10 minutes per run); on a slow run 3s isn't enough and the test fails anyway. It converts a deterministic wait into a gamble. Use explicit/fluent waits (Selenium) or auto-waiting assertions (Playwright). The only defensible use is a documented workaround for an animation with no observable end state.

CORE


```
// ---- Excel with Apache POI ----
public static List<Map<String,String>> readSheet(String path, String sheetName) {
    List<Map<String,String>> rows = new ArrayList<>();
    try (FileInputStream fis = new FileInputStream(path);
        Workbook wb = new XSSFWorkbook(fis)) { // XSSF = .xlsx, HSSF = .xls
        Sheet sheet = wb.getSheet(sheetName);
        Row header = sheet.getRow(0);
        DataFormatter fmt = new DataFormatter(); // reads any cell type as String
        for (int r = 1; r <= sheet.getLastRowNum(); r++) {
            Row row = sheet.getRow(r);
            if (row == null) continue;
            Map<String,String> map = new LinkedHashMap<>();
            for (int c = 0; c < header.getLastCellNum(); c++) {
                map.put(fmt.formatCellValue(header.getCell(c)),
                        fmt.formatCellValue(row.getCell(c)));
            }
            rows.add(map);
        }
    } catch (IOException e) { throw new FrameworkException("Excel read failed: " + path, e); }
    return rows;
}

// ---- JSON with Jackson ----
ObjectMapper mapper = new ObjectMapper();
User user = mapper.readValue(new File("src/test/resources/user.json"), User.class);
List<User> users = mapper.readValue(file, new TypeReference<List<User>>() {});

// ---- Properties ----
Properties p = new Properties();
p.load(new FileInputStream("config.properties"));
String url = p.getProperty("base.url");
```

DataFormatter is the detail that shows experience: without it, a numeric mobile number cell comes back as 9.87654321E9 and your test types garbage into the field.

Coding round: string questions they actually ask (reverse, palindrome, duplicates, vowels).

```
// 1. Reverse a string WITHOUT StringBuilder.reverse()
public static String reverse(String s) {
    char[] c = s.toCharArray();
    for (int i = 0, j = c.length - 1; i < j; i++, j--) {
        char t = c[i]; c[i] = c[j]; c[j] = t;
    }
    return new String(c);
}

// 2. Palindrome check, ignoring case and non-alphanumerics
public static boolean isPalindrome(String s) {
    String clean = s.replaceAll("[^A-Za-z0-9]", "").toLowerCase();
    return clean.equals(new StringBuilder(clean).reverse().toString());
}

// 3. Reverse the WORDS of a sentence ("I Love QA" -> "QA Love I")
public static String reverseWords(String s) {
    String[] w = s.trim().split("\\s+");
    Collections.reverse(Arrays.asList(w));
    return String.join(" ", w);
}

// 4. First non-repeating character
public static Character firstUnique(String s) {
    Map<Character, Integer> count = new LinkedHashMap<>();
    for (char c : s.toCharArray()) count.merge(c, 1, Integer::sum);
    return count.entrySet().stream().filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey).findFirst().orElse(null);
}

// 5. Count duplicate characters
public static Map<Character, Long> duplicates(String s) {
    return s.chars().mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(c -> c, Collectors.counting()))
        .entrySet().stream().filter(e -> e.getValue() > 1)
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
}

// 6. Count vowels and consonants
long vowels = s.toLowerCase().chars().filter(c -> "aeiou".indexOf(c) >= 0).count();

// 7. Are two strings anagrams?
public static boolean isAnagram(String a, String b) {
    char[] x = a.replaceAll("\\s", "").toLowerCase().toCharArray();
    char[] y = b.replaceAll("\\s", "").toLowerCase().toCharArray();
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}
```

In a coding round, **talk while you code**: state the approach, the time complexity ($O(n)$, $O(n \log n)$), and the edge cases you're handling (null, empty, single char, mixed case, spaces). Interviewers score the reasoning at least as much as the code.

CORE

CODING ROUND

```
// 1. Second largest element in an array (single pass, no sorting)
public static int secondLargest(int[] a) {
    int max = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
    for (int n : a) {
        if (n > max) { second = max; max = n; }
        else if (n > second && n != max) { second = n; }
    }
    if (second == Integer.MIN_VALUE) throw new IllegalArgumentException("No second largest");
    return second;
}

// 2. Remove duplicates from a List, preserving order
List<String> unique = new ArrayList<>(new LinkedHashSet<>(items));
// or: items.stream().distinct().collect(Collectors.toList());

// 3. Sort a Map by value (descending)
Map<String,Integer> sorted = map.entrySet().stream()
    .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
        (a,b) -> a, LinkedHashMap::new)); // LinkedHashMap keeps the order!

// 4. Find duplicate elements in an array
Set<Integer> seen = new HashSet<>(), dups = new HashSet<>();
for (int n : arr) if (!seen.add(n)) dups.add(n);

// 5. Fibonacci (iterative – mention recursion is  $O(2^n)$  without memoisation)
public static long fib(int n) {
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}

// 6. Factorial, prime check, swap without a temp variable
public static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) if (n % i == 0) return false; // only up to sqrt(n)
    return true;
}

a = a + b; b = a - b; a = a - b; // swap without temp

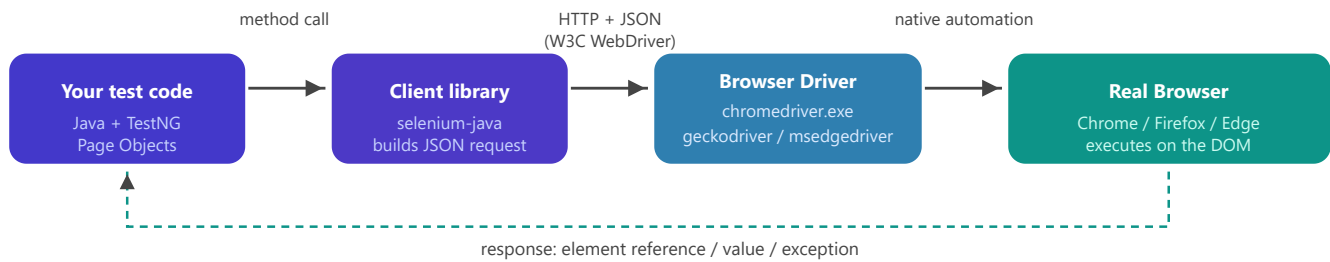
// 7. Sort a List of objects by a field
employees.sort(Comparator.comparing(Employee::getSalary)
    .thenComparing(Employee::getName).reversed());
```

SECTION 3

Selenium WebDriver with Java

Q66 – Q115 · The core of the interview. Expect to be asked to **write code on a whiteboard or shared editor** for waits, frames, windows, tables and the Actions class — so type these out at least once by hand.

Selenium 4 (W3C) Architecture — what happens on `driver.click()`



Selenium 3 vs 4:

Selenium 3 sent JSON Wire Protocol requests that the driver had to *translate* to W3C.
Selenium 4 speaks W3C natively — no encoding/decoding layer, so it is faster and less flaky.

Figure 5 — Draw this on the whiteboard when asked "how does Selenium work internally". Mention that every command is an HTTP request to a local server the browser driver runs.

What is Selenium and what are its components?

Selenium is a free, open-source suite for automating **web browsers**. It is not a testing tool by itself — it is a browser-driving library; you add a test framework (TestNG/JUnit), assertions and reporting around it.

- **Selenium IDE** — a browser extension for record-and-playback. Good for quick prototypes and for non-programmers; not used for production suites.
- **Selenium WebDriver** — the programmable API (Java, Python, C#, JS, Ruby, Kotlin) that drives a real browser via the browser's native driver. This is what "Selenium" means in practice.
- **Selenium Grid** — distributes tests across multiple machines/browsers in parallel.
- **Selenium RC** — the deprecated predecessor (proxy/JS-injection based), removed in Selenium 3+. Mention it only historically.

Supported languages: Java, Python, C#, Ruby, JavaScript, Kotlin. Supported browsers: Chrome, Firefox, Edge, Safari, Opera, and headless variants.

Q67

Explain the Selenium WebDriver architecture — what happens when you call `driver.findElement(...).click()`?

1. Your test calls a method on the `WebDriver` interface (implemented by `ChromeDriver` → `RemoteWebDriver`).
2. The Selenium client library serialises that command into an **HTTP request with a JSON body** following the **W3C WebDriver protocol**, e.g. `POST /session/{id}/element` with `{"using":"css selector","value":"#login"}`.
3. The request goes over HTTP to the **browser driver** (chromedriver / geckodriver / msedgedriver), which is a small HTTP server running locally on a port.
4. The driver translates it into the browser's own native automation interface (DevTools Protocol for Chrome, Marionette for Firefox) and executes it.
5. The browser performs the action on the real DOM and returns a result. The driver wraps it in an HTTP response; the client deserialises it into a Java object (a `WebElement` reference, a String, or an exception).

Key consequence to state: **every single Selenium command is a network round trip**. That is why a loop calling `getText()` on 500 rows is slow, and why chaining fewer, smarter commands (or one `executeScript`) is faster.

Also mention: since Selenium 4.6, **Selenium Manager** is built in — it auto-downloads the matching driver binary, so `WebDriverManager` and manual `System.setProperty("webdriver.chrome.driver", ...)` are no longer required.

CORE**ADVANCED**

AREA	SELENIUM 3	SELENIUM 4
Protocol	JSON Wire Protocol + W3C translation layer	Native W3C WebDriver — faster, more stable
Driver setup	Manual <code>System.setProperty</code> or WebDriverManager	Selenium Manager auto-resolves drivers (4.6+)
Capabilities	<code>DesiredCapabilities</code>	<code>ChromeOptions</code> / <code>FirefoxOptions</code> ; DesiredCapabilities deprecated
Waits	new <code>WebDriverWait(driver, 20)</code> (long seconds)	<code>Duration.ofSeconds(20)</code>
Locators	8 standard locators	+ Relative locators : <code>above()</code> , <code>below()</code> , <code>toLeftOf()</code> , <code>toRightOf()</code> , <code>near()</code>
Windows/tabs	Open via JavaScript hack	<code>driver.switchTo().newWindow(WindowType.TAB / WINDOW)</code>
Screenshots	Viewport only (element screenshot limited)	Element screenshots natively; full-page via CDP/Firefox
DevTools	Not available	Chrome DevTools Protocol access: network interception, mock geolocation, console logs, basic auth, performance metrics
Grid	Hub + Node only	Standalone / Hub-Node / fully distributed ; Docker & Kubernetes ready; built-in UI & observability
Standard	—	Groundwork for WebDriver BiDi (bi-directional, event-based) — the future direction

What are Selenium's limitations?

- **Web only** — no desktop apps (use WinAppDriver/Appium for desktop, Appium for mobile native).
- **No built-in reporting, assertions, or test runner** — you supply TestNG/JUnit + Extent/Allure.
- **Cannot read non-web content:** captcha, OTP, barcodes, PDF content (needs PDFBox), images (needs OCR/visual tools).
- **No native API/DB testing** — combine with RestAssured / JDBC.
- **Windows-level dialogs** (Windows file picker, print dialog, basic-auth popups, browser notification prompts) are outside the DOM — need Robot class, AutoIT, or browser options.
- **Flakiness** from timing and dynamic DOMs — needs a disciplined wait strategy.
- **Performance / load testing** is not its purpose (spinning up 1000 browsers is not a load test — use JMeter/k6).
- **No official support** — it is a community-driven open-source project.

CORE

Q70

List all Selenium locators. Which do you prefer and why?

Eight standard locators: `id`, `name`, `className`, `tagName`, `linkText`, `partialLinkText`, `cssSelector`, `xpath`. Selenium 4 adds **relative locators**.

My preference order

1. **Custom test attribute** — `[data-testid='login-btn']`. Ask the developers to add these; they never change with styling. This answer alone impresses interviewers.
2. `id` — fastest and unique... *if* it is static. Watch out for generated ids like `input_28471`.
3. `name`, then `cssSelector` — concise, fast, readable.
4. `linkText` / `partialLinkText` — only for anchors; breaks with localisation.
5. `xpath` — last resort, but unavoidable when you need to traverse *up* the DOM or match by visible text.
6. Avoid `tagName` and bare `className` alone — almost never unique.

Never use absolute XPath (`/html/body/div[2]/div[3]/form/input[1]`). One extra wrapper div from a designer and every test breaks.

```
driver.findElement(By.id("username"));
driver.findElement(By.cssSelector("[data-testid='submit']"));
driver.findElement(By.xpath("//button[normalize-space()='Sign in']"));
// Selenium 4 relative locator:
driver.findElement(with(By.tagName("input")).below(By.id("password-label")));
```

CORE

Absolute starts at the root with a single slash `/html/body/div/...` — extremely brittle.

Relative starts with `//` and searches anywhere in the DOM.

```
//tagname[@attribute='value']           // basic
//input[@id='user' and @type='text']     // multiple conditions
//input[@id='user' or @name='user']
//*[id='user']                           // any tag
//button[text()='Submit']                // exact visible text
//button[normalize-space()='Submit']     // trims & collapses whitespace ← use this one
//div[contains(@class,'alert')]          // partial attribute match
//a[starts-with(@href,'/orders/')]
//input[ends-with(@id,'_email')]         // XPath 2.0 – NOT supported by browsers!
//td[contains(text(),'Delivered')]
(//table//tr)[3]                         // indexing – brackets around the whole expression
//table//tr[last()]
//table//tr[position()<=5]
//input[@type='checkbox'][not(@disabled)]
//div[@class='card'][./span[text()='In stock']] // predicate with nested condition
```

Two very common traps: (1) `ends-with()` is XPath 2.0 and does **not** work in browsers — use `contains()` or `substring()`. (2) `//tr[1]` means "the first tr *within each parent*", while `(//tr)[1]` means "the first tr in the whole document". Interviewers love this one.

CORE

TRAP

Axes let you navigate **relative to a known element** — the killer feature CSS does not have (CSS cannot go upwards).

AXIS	SELECTS	EXAMPLE
<code>parent</code>	Immediate parent	<code>//span[text()='Price']/parent::div</code>
<code>ancestor</code>	All ancestors up to root	<code>//td[text()='ORD-991']/ancestor::tr</code>
<code>child</code>	Direct children	<code>//ul[@id='menu']/child::li</code>
<code>descendant</code>	All nested elements	<code>//div[@id='cart']/descendant::input</code>
<code>following</code>	Everything after the closing tag	<code>//label[text()='Email']/following::input[1]</code>
<code>following-sibling</code>	Siblings after it	<code>//td[text()='ORD-991']/following-sibling::td[2]</code>
<code>preceding</code>	Everything before it	<code>//h2[text()='Payment']/preceding::input[1]</code>
<code>preceding-sibling</code>	Siblings before it	<code>//td[@class='status']/preceding-sibling::td[1]</code>
<code>self</code>	The node itself	<code>//div/self::div</code>

The classic table interview problem

```
// "Click the Delete button in the row whose Order ID is ORD-991"
By deleteBtn = By.xpath("//td[normalize-space()='ORD-991']"
                    + "/ancestor::tr//button[@aria-label='Delete']");
driver.findElement(deleteBtn).click();
```

If you can write that one XPath fluently, you will clear most Selenium locator rounds. Practise it.

CORE

ADVANCED

```
#login           /* id */
.btn-primary     /* class */
input#user.form-ctl /* tag + id + class */
[data-testid='cart'] /* attribute */
[href^='/orders'] /* starts with */
[href$='.pdf']     /* ends with */
[class*='alert']  /* contains */
div > input       /* direct child */
div input         /* any descendant */
h2 + p           /* adjacent sibling */
h2 ~ p           /* all following siblings */
ul li:nth-child(3) /* 3rd child */
tr:nth-of-type(odd)
li:first-child, li:last-child
input:not([disabled])
input[type='checkbox']:checked
```

	CSS	XPATH
Speed	Slightly faster (native browser engine)	Slightly slower, especially in older IE
Readability	Shorter, cleaner	Verbose
Traverse upward	✗ not possible	✓ <code>parent::</code> , <code>ancestor::</code>
Match by visible text	✗ (<code>:contains()</code> is jQuery, not CSS)	✓ <code>text()</code> , <code>contains(text(),...)</code>
Indexing	<code>:nth-child()</code> only	Full <code>[n]</code> , <code>last()</code> , <code>position()</code>

Balanced answer: "I default to CSS for its speed and readability, and switch to XPath when I need text matching or upward traversal — typically for tables and dynamic content. The performance difference is negligible in modern browsers, so I optimise for *maintainability*, not microseconds."

Strategies, in order of preference:

1. **Anchor to something stable** — a label, heading or visible text near it, then traverse with XPath axes.
2. **Partial matching** on the stable part of the attribute:

```
//input[starts-with(@id, 'txt_email_')]  
//div[contains(@class, 'ng-tns-') and @role='dialog']  
//*[contains(@id, '_submit')]
```

3. **Match on a non-changing attribute** — `name`, `type`, `placeholder`, `aria-label`, `data-*`.
4. **Relative position** — Selenium 4 relative locators, or `following::input[1]` after a label.
5. **Ask the dev team for** `data-testid` — the real long-term fix. Frame it as a team practice, not a personal preference.
6. **Parameterised locators** in the page object so one locator serves many values:

```
private static final String ROW_ACTION =  
    "//td[normalize-space()='%s']/ancestor::tr//button[@data-action='%s']";  
  
public void rowAction(String orderId, String action) {  
    driver.findElement(By.xpath(String.format(ROW_ACTION, orderId,  
        action))).click();  
}
```

Q75

findElement vs findElements — what does each return when nothing matches?

	FINDELEMENT	FINDELEMENTS
Returns	A single <code>WebElement</code> — the first match	<code>List<WebElement></code> of all matches
No match	Throws <code>NoSuchElementException</code>	Returns an empty list (size 0) — no exception
With implicit wait	Polls until timeout, then throws	Polls until timeout, then returns empty (so it costs you the full timeout)

```
// Standard idiom for "is this element present?" – no try/catch needed
public boolean isPresent(By locator) {
    return !driver.findElements(locator).isEmpty();
}

// And chaining a search INSIDE an element (scoped search – very useful)
WebElement row = driver.findElement(By.id("order-991"));
List<WebElement> cells = row.findElements(By.tagName("td")); // searches only within row
```

Follow-up they often add: "how do you check an element is **not** present without a 10-second penalty?" Answer: temporarily reduce the implicit wait, or better, use an explicit wait with `ExpectedConditions.invisibilityOfElementLocated / numberOfElementsToBe(locator, 0)`.

CORE

TRAP

Q76

Name the common Selenium exceptions and their causes.

EXCEPTION	CAUSE	TYPICAL FIX
<code>NoSuchElementException</code>	Locator matched nothing at that moment	Fix locator; add explicit wait; check you're in the right frame/window
<code>StaleElementReferenceException</code>	The element reference points to a DOM node that was re-rendered/removed	Re-find the element; use PageFactory refresh or a retry (see Q77)
<code>ElementNotInteractableException</code>	Element exists but is hidden, zero-sized, disabled or animating	Wait for visibility/enabled; scroll into view
<code>ElementClickInterceptedException</code>	Another element (overlay, sticky header, cookie banner) covers it	Dismiss the overlay, scroll, or JS click as a last resort
<code>TimeoutException</code>	An explicit wait condition never became true	Verify the condition is right; increase timeout; check the app is actually working
<code>NoSuchFrameException</code> / <code>NoSuchWindowException</code>	Switching to a frame/window that doesn't exist	Wait for frame availability; re-read window handles
<code>NoAlertPresentException</code>	<code>switchTo().alert()</code> with no alert open	<code>wait.until(alertIsPresent())</code>
<code>InvalidSelectorException</code>	Malformed XPath/CSS syntax	Validate the selector in DevTools console
<code>SessionNotCreatedException</code>	Driver/browser version mismatch, or browser not installed	Selenium Manager / matching driver version
<code>WebDriverException: chrome not reachable</code>	Browser crashed or was closed manually	Check resources; ensure only one <code>quit()</code>

UnhandledAlertException

An unexpected JS alert
blocked the command

Handle/dismiss alerts, or set
unhandledPromptBehavior

CORE

When you call `findElement`, Selenium returns a **reference** (an internal element id) to a node in the browser's DOM at that instant. If the page then re-renders — an AJAX update, a React/Angular re-render, a page refresh, a sort/filter — the old node is discarded. Your reference now points to something that no longer exists in the DOM, so the next use throws `StaleElementReferenceException`.

Two flavours

- The element was **deleted** from the DOM.
- The element is **no longer attached** to the current document (navigation happened).

Fixes, best to worst

1. **Don't cache element references.** Store `By` locators and find the element at the moment of use. This is why `By`-based page objects are more robust than eagerly-cached `WebElement` fields.
2. **Wait for the update to finish first** — wait for the spinner to disappear or for a specific expected condition before interacting.
3. **Re-find inside a loop** when iterating a list that changes:

```
int size = driver.findElements(By.cssSelector(".row")).size();
for (int i = 0; i < size; i++) {
    // re-find EVERY iteration – the list is rebuilt after each click
    WebElement row = driver.findElements(By.cssSelector(".row")).get(i);
    row.findElement(By.cssSelector(".open")).click();
    driver.navigate().back();
}
```

4. **Retry wrapper** for genuinely racy UIs:

```
public void safeClick(By locator) {
    int attempts = 0;
    while (attempts < 3) {
        try {
            wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
            return;
        } catch (StaleElementReferenceException e) {
            attempts++;
            // element re-rendered, look it up
            again
        }
    }
}
```

```
    }  
    throw new FrameworkException("Element still stale after 3 attempts: " +  
locator);  
}
```

5. `ExpectedConditions.refreshed(ExpectedConditions.elementToBeClickable(locator))` — built-in staleness-tolerant wait.

Why @FindBy is less prone to this: PageFactory creates a lazy proxy that re-locates the element on each method call. It still goes stale if you hold the element across a re-render inside one method, and @CacheLookup disables that protection entirely.

CORE

ADVANCED

Q78

ElementClickInterceptedException — the click "works manually but fails in automation". How do you debug it?

It means the point Selenium would click is occupied by a **different** element. Debug in this order:

1. **Take a screenshot at the moment of failure** (your listener should do this automatically). 90% of the time you'll see the culprit: a cookie consent banner, a sticky header, a toast notification, a loading overlay, or a modal backdrop that hasn't faded out.
2. **Fix the actual cause** — dismiss the banner in setup (or set the cookie directly), wait for the overlay to become invisible:

```
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.cssSelector(".loading-overlay")));  
wait.until(ExpectedConditions.elementToBeClickable(saveBtn)).click();
```

3. **Scroll it into the centre** of the viewport (a sticky header covers elements scrolled to the top):

```
((JavascriptExecutor) driver).executeScript(  
    "arguments[0].scrollIntoView({block: 'center'})";, element);
```

4. **JavaScript click as a last resort:**

```
((JavascriptExecutor) driver).executeScript("arguments[0].click();", element);
```

Say this and you'll stand out: "A JS click bypasses the browser's real event model — it fires the click even if a human could never click it, so it can hide a genuine UI bug like an overlay that never disappears. I use it only after understanding *why* the normal click failed, and I add a comment saying why."

CORE**ADVANCED**

Selenium Waits — how they behave on a timeline

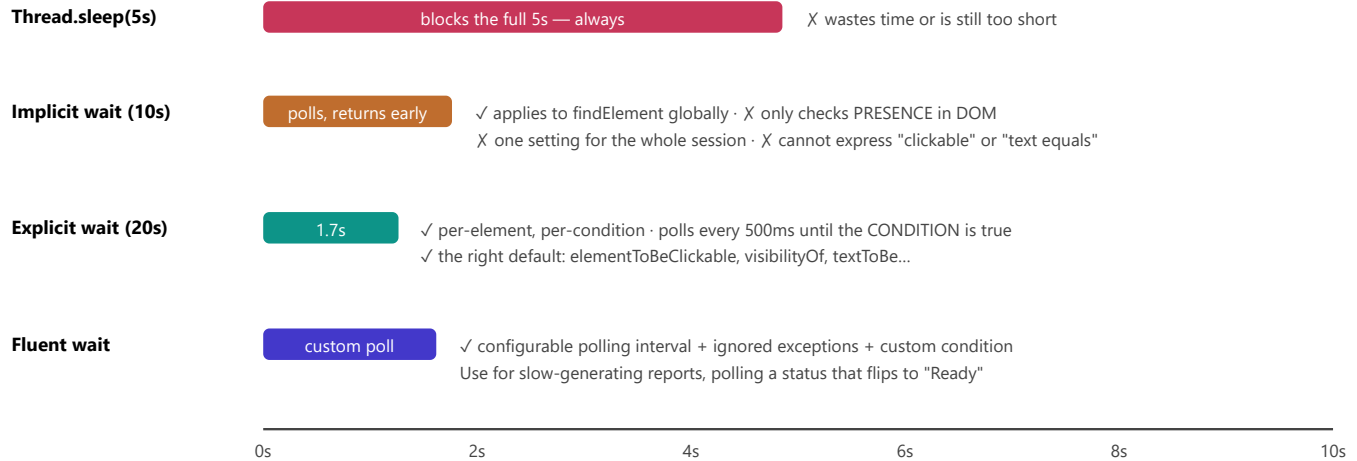


Figure 6 — Explicit waits return as soon as the condition is true, so a suite using them is dramatically faster *and* more stable than one using sleeps.

1. Implicit wait — global, DOM presence only

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

Applies to **every** `findElement` / `findElements` for the life of the driver. Polls until the element is *present in the DOM*, then returns. It cannot express visibility, clickability or text conditions.

2. Explicit wait — per element, per condition (the default choice)

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));
WebElement btn = wait.until(ExpectedConditions.elementToBeClickable(By.id("pay")));
btn.click();
wait.until(ExpectedConditions.textToBe(By.id("status"), "Payment successful"));
```

Polls every 500ms by default and throws `TimeoutException` if the condition never becomes true.

3. Fluent wait — full control

```
Wait<WebDriver> fluent = new FluentWait<>(driver)
    .withTimeout(Duration.ofMinutes(2))           // Long-running report
    .pollingEvery(Duration.ofSeconds(3))          // don't hammer the server
    .ignoring(NoSuchElementException.class)
    .ignoring(StaleElementReferenceException.class)
    .withMessage("Report never reached READY state");

WebElement report = fluent.until(d -> {
    WebElement s = d.findElement(By.id("report-status"));
    return "READY".equals(s.getText()) ? s : null; // null/false = keep polling
});
```

`WebDriverWait` is actually a subclass of `FluentWait` with a fixed 500ms polling interval and `NoSuchElementException` pre-ignored. Saying that shows you've read the source, not just a blog.

Why should you never mix implicit and explicit waits?

Because the two polling mechanisms interfere and produce **unpredictable, non-additive timeouts**. The explicit wait calls `findElement` internally; that call is itself subject to the implicit wait, so each polling attempt can block for the full implicit duration. The official Selenium documentation explicitly warns that this can cause waits to last far longer than either value — and in some driver versions, unexpected timing behaviour.

Example of the confusion: implicit = 10s, explicit = 15s. You might expect a failure at 15s. In practice the first internal lookup blocks for 10s, so you get roughly one or two polls in 15s and a confusing result — or a total closer to 20–30s.

My recommendation (say this): "Set the implicit wait to zero and use explicit waits exclusively, wrapped in reusable helper methods in `BasePage`. That gives one predictable wait strategy, self-documenting conditions, and much better failure messages."

```
// BasePage – one place, used everywhere
protected WebElement visible(By locator) {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
}
protected WebElement clickable(By locator) {
    return wait.until(ExpectedConditions.elementToBeClickable(locator));
}
protected void click(By locator) { clickable(locator).click(); }
protected void type(By locator, String text) {
    WebElement e = visible(locator);
    e.clear();
    e.sendKeys(text);
}
```

CORE

TRAP

Q81**Which ExpectedConditions do you use most? Name at least ten.**

- `presenceOfElementLocated(by)` — in the DOM (may be invisible)
- `visibilityOfElementLocated(by)` / `visibilityOf(element)` — present **and** displayed with non-zero size
- `invisibilityOfElementLocated(by)` / `invisibilityOfElementWithText(by, text)` — for spinners
- `elementToBeClickable(by)` — visible **and** enabled
- `textToBePresentInElement(el, text)` / `textToBe(by, text)` / `textMatches(by, pattern)`
- `attributeToBe(by, attr, value)` / `attributeContains(...)`
- `titleIs(t)` / `titleContains(t)` / `urlToBe(u)` / `urlContains(u)`
- `alertIsPresent()`
- `frameToBeAvailableAndSwitchToIt(by)`
- `numberOfWindowsToBe(2)` — the correct way to wait for a new tab
- `numberOfElementsToBeMoreThan(by, n)` — for lazy-loaded lists
- `stalenessOf(element)` — wait for the old page to be torn down before finding the new one
- `elementSelectionStateToBe(by, true)` , `invisibilityOfAllElements(list)`
- Combinators: `ExpectedConditions.and(c1, c2)` , `or(c1, c2)` , `not(c)` , `refreshed(c)`

CORE


```

// Wait until a jQuery/AJAX-heavy page has finished all in-flight requests
public static ExpectedCondition<Boolean> pageIsQuiet() {
    return driver -> {
        JavascriptExecutor js = (JavascriptExecutor) driver;
        boolean docReady = "complete".equals(js.executeScript("return document.readyState"));
        boolean jqueryDone = (Boolean) js.executeScript(
            "return (typeof jQuery === 'undefined') || jQuery.active === 0");
        return docReady && jqueryDone;
    };
}

// Wait until a table has more than N rows AND the loader is gone
public static ExpectedCondition<Boolean> tableLoaded(By table, int minRows) {
    return new ExpectedCondition<Boolean>() {
        @Override public Boolean apply(WebDriver d) {
            return d.findElements(table).size() >= minRows
                && d.findElements(By.cssSelector(".spinner")).isEmpty();
        }
        @Override public String toString() {           // gives a readable timeout message
            return "table " + table + " to have at least " + minRows + " rows";
        }
    };
}

// usage
wait.until(pageIsQuiet());
wait.until(tableLoaded(By.cssSelector("#orders tbody tr"), 1));

```

Overriding `toString()` is the detail that separates a senior answer: it turns "Timed out after 20 seconds" into "Timed out waiting for table #orders tbody tr to have at least 1 row" — which saves hours of CI debugging.

ADVANCED

Q83

What is `pageLoadTimeout`, `setScriptTimeout` and page load strategy?

```
driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30)); // driver.get() gives up
driver.manage().timeouts().scriptTimeout(Duration.ofSeconds(15));    // async JS execution
```

```
ChromeOptions options = new ChromeOptions();
options.setPageLoadStrategy(PageLoadStrategy.EAGER);
```

STRATEGY	DRIVER.GET() RETURNS WHEN	USE FOR
NORMAL (default)	<code>document.readyState = "complete"</code> — all resources incl. images loaded	Safe default
EAGER	<code>readyState = "interactive"</code> — DOM parsed, images/stylesheets may still load	Big speed win on image-heavy sites; combine with explicit waits
NONE	Immediately after the initial HTML is downloaded	Maximum control; you handle every wait yourself

Exceeding `pageLoadTimeout` throws `TimeoutException`, which can leave the browser in a half-loaded state — always handle it and call `driver.navigate().refresh()` or fail cleanly.

ADVANCED

Standard HTML `<select>` — use the `Select` class

```
Select country = new Select(driver.findElement(By.id("country")));
country.selectByVisibleText("India");
country.selectByValue("IN");
country.selectByIndex(2);

// reading
WebElement current = country.getFirstSelectedOption();
List<WebElement> all = country.getOptions();
List<String> texts = all.stream().map(WebElement::getText).collect(Collectors.toList());

// multi-select
if (country.isMultiple()) {
    country.deselectByVisibleText("India");
    country.deselectAll();
}
```

Select only works on a real `<select>` tag. Passing anything else throws `UnexpectedTagNameException`. That is a favourite interview trap.

Custom dropdowns (div/ul based — React, Angular Material, Bootstrap)

```
// 1. open the dropdown
click(By.cssSelector("[data-testid='country-trigger']"));
// 2. wait for the option list to render
wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector(".dropdown-menu")));
// 3. click the option by its visible text
driver.findElement(By.xpath("//ul[@role='listbox']/li[normalize-space()='India']")).click();
// 4. assert the trigger now shows the selection (don't skip this)
Assert.assertEquals(getText(By.cssSelector("[data-testid='country-trigger']")), "India");
```

Searchable / type-ahead dropdown

```
type(By.id("country-search"), "Indi");
wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector(".suggestion")));
driver.findElements(By.cssSelector(".suggestion")).stream()
    .filter(e -> e.getText().equals("India"))
    .findFirst().orElseThrow(() -> new FrameworkException("India not in suggestions"))
    .click();
```

```
// Always wait – the alert is asynchronous
Alert alert = new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.alertIsPresent());

String message = alert.getText();           // read the text (assert it!)
alert.accept();                             // OK
alert.dismiss();                           // Cancel (confirm boxes)
alert.sendKeys("QA Tester");               // type into a prompt, then accept()
```

Three types: **alert** (OK only), **confirm** (OK/Cancel), **prompt** (text input + OK/Cancel).

While a JS alert is open, **every other WebDriver command throws `UnhandledAlertException`** — the browser is blocked. So handle it immediately. You can also configure default behaviour:

```
ChromeOptions o = new ChromeOptions();
o.setCapability("unhandledPromptBehavior", "accept"); // or "dismiss", "ignore"
```

Important distinction: a modal built with HTML/CSS (Bootstrap modal, custom dialog) is **not** a JS alert — it lives in the DOM and is handled with normal locators. Browser-level auth popups and file dialogs are neither, and need different techniques (Q93, Q105).

```
// Safe helper
public boolean acceptAlertIfPresent() {
    try {
        driver.switchTo().alert().accept();
        return true;
    } catch (NoAlertPresentException e) { return false; }
}
```

An iframe is a separate HTML document embedded in the page. WebDriver can only see elements in the frame it is currently focused on — this is the #1 cause of "the locator is correct in DevTools but Selenium says NoSuchElementException".

```
// switch by index (fragile), name/id (better), or WebElement (best)
driver.switchTo().frame(0);
driver.switchTo().frame("payment-frame");
driver.switchTo().frame(driver.findElement(By.cssSelector("iframe[title='Card details']")));

// safest – waits for the frame to exist AND switches in one step
new WebDriverWait(driver, Duration.ofSeconds(15)).until(
    ExpectedConditions.frameToBeAvailableAndSwitchToIt(By.id("payment-frame")));

// do the work inside the frame
driver.findElement(By.id("cardNumber")).sendKeys("4111111111111111");

// come back out – MANDATORY
driver.switchTo().defaultContent();    // back to the top-level document
driver.switchTo().parentFrame();       // up exactly one level (nested frames)

// how many frames on the page?
int count = driver.findElements(By.tagName("iframe")).size();
```

Nested frames: you must switch level by level — `frame("outer")` then `frame("inner")`. You cannot jump directly. And you cannot switch from one sibling frame to another without going back to `defaultContent()` first.

Good framework practice: wrap it so a test never forgets to switch back.

```
public <T> T inFrame(By frame, Supplier<T> action) {
    wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(frame));
    try { return action.get(); }
    finally { driver.switchTo().defaultContent(); }
}
// usage: inFrame(By.id("pay"), () -> { cardField.sendKeys("4111..."); return null; });
```

```
String parent = driver.getWindowHandle();           // current window's unique id (String)

driver.findElement(By.linkText("Terms & Conditions")).click(); // opens a new tab

// Wait for the new window rather than sleeping
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.numberOfWindowsToBe(2));

Set<String> all = driver.getWindowHandles();         // Set – unique, unordered
for (String handle : all) {
    if (!handle.equals(parent)) {
        driver.switchTo().window(handle);
        break;
    }
}
Assert.assertTrue(driver.getTitle().contains("Terms"));
driver.close();                                     // close only the child
driver.switchTo().window(parent);                   // ALWAYS switch back explicitly
```

Selenium 4 — open a tab/window yourself

```
driver.switchTo().newWindow(WindowType.TAB); // opens AND switches focus
driver.get("https://example.com/help");
driver.close();
driver.switchTo().window(parent);
```

Two traps: (1) after `driver.close()` the driver still points at the dead window — any command throws `NoSuchWindowException` until you switch. (2) `getWindowHandles()` returns a `Set`, so it has **no guaranteed order** — never assume the last element is the newest. Compare against the stored parent handle.

CORE

TRAP

`Actions` builds a chain of low-level input events (the W3C Actions API) and dispatches them with `.perform()`. Used for anything a simple `.click()` cannot do.

```
Actions actions = new Actions(driver);

actions.moveToElement(menu).perform();           // hover
actions.contextClick(row).perform();             // right click
actions.doubleClick(cell).perform();
actions.clickAndHold(source).moveToElement(target).release().perform();
actions.dragAndDrop(source, target).perform();    // convenience method
actions.dragAndDropBy(slider, 120, 0).perform();  // offset drag (sliders)
actions.moveToElement(el, 10, 10).click().perform(); // click at an offset

// Multi-step chain: hover a menu, then click a submenu item
actions.moveToElement(driver.findElement(By.id("products")))
    .pause(Duration.ofMillis(300)) // Let the CSS animation settle
    .moveToElement(driver.findElement(By.linkText("Laptops")))
    .click()
    .build().perform();
```

Very common real-world issue: HTML5 drag-and-drop (using the native dragstart/drop events) frequently does **not** work with `Actions.dragAndDrop`, because the browser's native DnD is not driven by synthetic mouse events. The standard workaround is a JavaScript-based DnD simulation, or the newer `clickAndHold` → `moveByOffset` (small steps) → `release` sequence with pauses. Mentioning this shows real project experience.

`build()` composes the chain into a single action; `perform()` executes it. Calling `perform()` directly implies `build()`.

```
// Simple keys straight into an element
search.sendKeys("selenium" + Keys.ENTER);
input.sendKeys(Keys.TAB);
input.sendKeys(Keys.chord(Keys.CONTROL, "a"), Keys.DELETE); // select all + delete

// Actions class for modifier combinations
new Actions(driver)
    .keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL) // Ctrl+A
    .perform();

new Actions(driver)
    .keyDown(Keys.CONTROL).sendKeys("c").keyUp(Keys.CONTROL).perform(); // copy
new Actions(driver).keyDown(Keys.CONTROL).sendKeys("v").keyUp(Keys.CONTROL).perform();

// Selenium 4 shortcuts
new Actions(driver).sendKeys(Keys.PAGE_DOWN).perform();
new Actions(driver).keyDown(Keys.SHIFT).click(first).click(last).keyUp(Keys.SHIFT).perform();

// Common Keys constants
Keys.ENTER, Keys.RETURN, Keys.TAB, Keys.ESCAPE, Keys.SPACE, Keys.BACK_SPACE,
Keys.ARROW_DOWN, Keys.END, Keys.HOME, Keys.F5, Keys.CONTROL, Keys.ALT, Keys.SHIFT
```

Keyboard-accessibility testing tip: you can automate a basic accessibility check by sending only TAB and ENTER through an entire flow and asserting it completes. That is a great thing to volunteer in an interview.

`JavaScriptExecutor` runs JavaScript directly in the browser context. `arguments[0]` maps to the first extra parameter you pass.

```
JavaScriptExecutor js = (JavaScriptExecutor) driver;

js.executeScript("arguments[0].click();", element);           // click through overlays
js.executeScript("arguments[0].value='QA';", input);          // set value directly
js.executeScript("arguments[0].scrollIntoView({block:'center'});", el);
js.executeScript("window.scrollTo(0,500);");
js.executeScript("window.scrollTo(0, document.body.scrollHeight);"); // scroll to bottom
js.executeScript("arguments[0].style.border='3px solid red';", el); // highlight for reports
js.executeScript("arguments[0].setAttribute('value','2026-03-15');", dateInput);
js.executeScript("arguments[0].removeAttribute('readonly');", el);

String title = (String) js.executeScript("return document.title;");
String state = (String) js.executeScript("return document.readyState;");
Long height = (Long) js.executeScript("return document.body.scrollHeight;");
Boolean done = (Boolean) js.executeScript("return jQuery.active == 0;");

js.executeScript("window.open('https://example.com','_blank');");
js.executeScript("localStorage.setItem('token', arguments[0]);", jwt); // skip UI Login!
```

The judgement question — "so should you always use JS click?" No. JS bypasses the browser's real event pipeline: it will click a disabled, hidden or covered element, and it does not fire the focus/blur/mouseover events a real user triggers. That means your test can pass while the feature is broken for users. Rule: **use JS only for things a user cannot do anyway** (scrolling into view, highlighting for reports, seeding localStorage), and treat a JS click as a documented workaround, never a default.

CORE

ADVANCED

```

JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("window.scrollTo(0, 500);"); // relative
js.executeScript("window.scrollTo(0, document.body.scrollHeight);");// to bottom
js.executeScript("window.scrollTo(0, 0);"); // to top
js.executeScript("arguments[0].scrollIntoView({block:'center', behavior:'instant'})", el);

// Scroll inside a scrollable container (a modal or a virtualised table)
js.executeScript("arguments[0].scrollTop = arguments[0].scrollHeight;", container);

// Selenium 4 – native scroll via the Actions API (no JS)
new Actions(driver).scrollToElement(element).perform();
new Actions(driver).scrollByAmount(0, 600).perform();
new Actions(driver).scrollFromOrigin(
    WheelInput.ScrollOrigin.fromElement(container), 0, 400).perform();

// Infinite scroll: keep scrolling until the height stops changing
long last = 0, current;
while ((current = (Long) js.executeScript("return document.body.scrollHeight")) != last) {
    last = current;
    js.executeScript("window.scrollTo(0, document.body.scrollHeight);");
    new WebDriverWait(driver, Duration.ofSeconds(5))
        .until(d -> (Long) js.executeScript("return document.body.scrollHeight") != last);
}

```

`scrollIntoView({block:'center'})` beats plain `scrollIntoView()`: the plain version aligns the element to the very top of the viewport, where a sticky header usually covers it — the #1 cause of `ElementClickInterceptedException`.

```
// Viewport screenshot
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
Files.copy(src.toPath(), Paths.get("screenshots/fail.png"), REPLACE_EXISTING);

// As Base64 – best for embedding in Extent/Allure reports (no file path issues in CI)
String base64 = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BASE64);

// As bytes – for Allure
byte[] bytes = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES);

// Selenium 4: element-only screenshot (great for visual checks of one component)
File logo = driver.findElement(By.id("logo")).getScreenshotAs(OutputType.FILE);

// Full-page beyond the viewport: Firefox natively, Chrome via CDP
Object result = ((ChromiumDriver) driver)
    .executeCdpCommand("Page.captureScreenshot", Map.of("captureBeyondViewport", true));
```

Automatic capture on failure — the part they actually want to hear

```
public class TestListener implements ITestListener {
    @Override public void onTestFailure(ITestResult result) {
        WebDriver driver = DriverFactory.get();
        String name = result.getMethod().getMethodName() + "_"
            + LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmss"));
        String b64 = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BASE64);
        ExtentManager.getTest().fail(result.getThrowable(),
            MediaEntityBuilder.createScreenCaptureFromBase64String(b64).build());
    }
}
```

Also capture: the current URL, browser console logs, and the page source. A screenshot alone often doesn't explain a CI-only failure; the console log usually does.

1. If it is a real `<input type="file">` — just `sendKeys` (correct 90% of the time)

```
WebElement upload = driver.findElement(By.cssSelector("input[type='file']"));
upload.sendKeys(Paths.get("src/test/resources/invoice.pdf").toAbsolutePath().toString());
```

This bypasses the OS file dialog entirely — no Robot/AutoIT needed. Key points: use an **absolute** path, and the input must exist in the DOM (it may be visually hidden behind a styled button, which is fine — `sendKeys` works on hidden file inputs; if it doesn't, unhide it with JS).

```
// If the input is display:none and sendKeys is rejected
js.executeScript("arguments[0].style.display='block'; arguments[0].style.opacity=1;", upload);
upload.sendKeys(absolutePath);
```

2. Multiple files

```
upload.sendKeys(pathA + "\n" + pathB); // newline-separated, needs the `multiple` attribute
```

3. If the OS dialog genuinely opens (drag-drop zones, non-input widgets)

```
// Robot — pure Java, no external tool, but ties the test to the OS & blocks parallel runs
StringSelection sel = new StringSelection(absolutePath);
Toolkit.getDefaultToolkit().getSystemClipboard().setContents(sel, null);
Robot robot = new Robot();
robot.keyPress(KeyEvent.VK_CONTROL); robot.keyPress(KeyEvent.VK_V);
robot.keyRelease(KeyEvent.VK_V);      robot.keyRelease(KeyEvent.VK_CONTROL);
robot.keyPress(KeyEvent.VK_ENTER);    robot.keyRelease(KeyEvent.VK_ENTER);
```

Alternatives: **AutoIT** (Windows-only .exe), or **CDP** `DOM.setFileInputFiles` in Selenium 4. In Playwright this is trivial (`setInputFiles` / `filechooser` event) — a fair point to raise if comparing tools.

For **Grid / remote** execution the file must exist on the *node* machine. Use `driver.setFileDetector(new LocalFileDetector());` on a `RemoteWebDriver` and Selenium will upload the local file to the node for you. This is a favourite senior-level follow-up.

```
// 1. Point the browser at a known folder and suppress the download prompt
String dir = System.getProperty("user.dir") + File.separator + "downloads";
Map<String,Object> prefs = new HashMap<>();
prefs.put("download.default_directory", dir);
prefs.put("download.prompt_for_download", false);
prefs.put("plugins.always_open_pdf_externally", true); // download PDFs, don't preview
ChromeOptions options = new ChromeOptions();
options.setExperimentalOption("prefs", prefs);
WebDriver driver = new ChromeDriver(options);

// 2. Click download, then WAIT for the file (never Thread.sleep)
public static Path waitForDownload(Path dir, String extension, Duration timeout) {
    long end = System.currentTimeMillis() + timeout.toMillis();
    while (System.currentTimeMillis() < end) {
        try (Stream<Path> files = Files.list(dir)) {
            Optional<Path> done = files
                .filter(p -> p.toString().endsWith(extension))
                .filter(p -> !p.toString().endsWith(".crdownload")) // Chrome partial file
                .max(Comparator.comparingLong(p -> p.toFile().lastModified()));
            if (done.isPresent() && done.get().toFile().length() > 0) return done.get();
        } catch (IOException ignored) { }
    }
    throw new FrameworkException("No " + extension + " downloaded within " + timeout);
}

// 3. Verify CONTENT, not just existence
Path pdf = waitForDownload(Paths.get(dir), ".pdf", Duration.ofSeconds(30));
try (PDDocument doc = PDDocument.load(pdf.toFile())) { // Apache PDFBox
    String text = new PDFTextStripper().getText(doc);
    Assert.assertTrue(text.contains("Invoice #INV-2026-0042"));
}
```

Two senior points: (1) clean the download folder in `@BeforeMethod` so a stale file can't produce a false pass; (2) for headless Chrome in Docker you may need `Page.setDownloadBehavior` via CDP, and on a remote Grid node the file lands on the node, so downloading via an authenticated HTTP call (grab the cookies from the driver and use `RestAssured/HttpClient`) is often more reliable than driving the browser.

```
// ---- 1. Basic dimensions ----
List<WebElement> rows = driver.findElements(By.cssSelector("#orders tbody tr"));
int rowCount = rows.size();
int colCount = driver.findElements(By.cssSelector("#orders thead th")).size();

// ---- 2. Read a specific cell (row 3, column 2) ----
String cell = driver.findElement(
    By.xpath("//table[@id='orders']//tbody/tr[3]/td[2]")).getText();

// ---- 3. Read the whole table into a List<Map> – reusable and assertion-friendly ----
public List<Map<String,String>> readTable(By table) {
    List<String> headers = driver.findElements(
        new ByChained(table, By.cssSelector("thead th")))
        .stream().map(WebElement::getText).collect(Collectors.toList());
    List<Map<String,String>> data = new ArrayList<>();
    for (WebElement row : driver.findElements(new ByChained(table, By.cssSelector("tbody tr")))) {
        List<WebElement> cells = row.findElements(By.tagName("td"));
        Map<String,String> map = new LinkedHashMap<>();
        for (int i = 0; i < headers.size() && i < cells.size(); i++)
            map.put(headers.get(i), cells.get(i).getText().trim());
        data.add(map);
    }
    return data;
}

// ---- 4. Find the row matching a value and act on it (the classic question) ----
driver.findElement(By.xpath(
    "//td[normalize-space()='ORD-991']/ancestor::tr//button[@data-action='cancel']")).click();

// ---- 5. Verify a column is sorted ----
List<String> dates = readTable(By.id("orders")).stream()
    .map(r -> r.get("Order Date")).collect(Collectors.toList());
Assert.assertEquals(dates, dates.stream().sorted(Comparator.reverseOrder())
    .collect(Collectors.toList()), "Orders not sorted newest first");

// ---- 6. Pagination: collect across all pages ----
List<Map<String,String>> all = new ArrayList<>();
while (true) {
    all.addAll(readTable(By.id("orders")));
    List<WebElement> next = driver.findElements(By.cssSelector("a.next:not(.disabled)"));
    if (next.isEmpty()) break;
    WebElement firstRow = driver.findElement(By.cssSelector("#orders tbody tr"));
    next.get(0).click();
    wait.until(ExpectedConditions.stalenessOf(firstRow));    // wait for the page to change
}
```

The `stalenessOf(firstRow)` trick for pagination is a great thing to show: it waits for the *old* content to be replaced, which is far more reliable than waiting for a row to be present (rows are

present on both pages).

CORE

ADVANCED

Approach in order of preference:

1. **Type into the input if it is editable** — fastest and most stable.

```
WebElement date = driver.findElement(By.id("checkin"));
date.clear();
date.sendKeys("15/03/2026");
date.sendKeys(Keys.ESCAPE);           // close the popup calendar
```

2. If `readonly`, **remove the attribute with JS** (acceptable when you're testing downstream logic, not the picker itself) — but note this bypasses the widget, so also keep one test that uses the real UI.
3. **Navigate the widget properly** — the answer they want:

```
public void pickDate(LocalDate target) {
    click(By.id("checkin"));           // open the calendar
    // navigate month/year until the header matches
    String wanted = target.format(DateTimeFormatter.ofPattern("MMMM yyyy"));
    for (int guard = 0; guard < 36; guard++) {           // guard against
infinite loops
        String shown = getText(By.cssSelector(".dp-header"));
        if (shown.equalsIgnoreCase(wanted)) break;
        boolean forward = YearMonth.parse(shown,
MONTH_YEAR).isBefore(YearMonth.from(target));
        click(By.cssSelector(forward ? ".dp-next" : ".dp-prev"));
    }
    // click the day, scoped to the current month (avoid greyed-out neighbour
days)
    driver.findElement(By.xpath("//td[not(contains(@class,'other-month'))]"
        + "[normalize-space()=' " + target.getDayOfMonth() + "']")).click();
}
```

4. **Set the value directly with JS** plus dispatch a `change` event so the framework (React/Angular) notices:

```
js.executeScript(
    "arguments[0].value=arguments[1];" +
    "arguments[0].dispatchEvent(new Event('input',{bubbles:true}));" +
```



```
"arguments[0].dispatchEvent(new Event('change',{bubbles:true}));", input, "2026-03-15");
```

Test data tip: never hardcode "15/03/2026" — it will be in the past next year and your booking test will fail. Compute relative dates: `LocalDate.now().plusDays(7)`. Interviewers notice this.

CORE

Q97

How do you handle auto-suggest / type-ahead search boxes?

```
WebElement search = driver.findElement(By.id("q"));
search.sendKeys("bangal");

// Wait for the suggestion list to actually populate (not just exist)
By suggestions = By.cssSelector("ul.suggestions li");
wait.until(ExpectedConditions.numberOfElementsToBeMoreThan(suggestions, 0));

// Option A: pick by exact text
driver.findElements(suggestions).stream()
    .filter(e -> e.getText().equalsIgnoreCase("Bangalore"))
    .findFirst()
    .orElseThrow(() -> new FrameworkException("Bangalore not suggested for 'bangal'"))
    .click();

// Option B: keyboard navigation (also validates accessibility)
search.sendKeys(Keys.ARROW_DOWN, Keys.ARROW_DOWN, Keys.ENTER);

// Assert every suggestion actually matches the typed text – a real test, not just a click
driver.findElements(suggestions).forEach(e ->
    Assert.assertTrue(e.getText().toLowerCase().contains("bangal"),
        "Irrelevant suggestion returned: " + e.getText()));
```

Debounce gotcha: most type-ahead inputs wait ~300ms after the last keystroke before firing the request. If you `sendKeys` the whole string at once and immediately read the list, you may catch suggestions from a previous keystroke. Waiting for a *content* condition (e.g. the first suggestion contains your text) rather than mere presence solves it.

CORE

Q98

isDisplayed vs isEnabled vs isSelected — and how do you handle checkboxes and radio buttons?

METHOD	RETURNS TRUE WHEN	NOTE
<code>isDisplayed()</code>	Element is rendered and visible (non-zero size, not <code>display:none</code> / <code>visibility:hidden</code>)	Throws <code>NoSuchElementException</code> if not in the DOM at all
<code>isEnabled()</code>	Element is not <code>disabled</code>	Returns true for most non-form elements regardless
<code>isSelected()</code>	A checkbox/radio/option is checked	Only meaningful for those three

```
// Toggle a checkbox to a DESIRED state, not blindly click (idempotent – the right way)
public void setCheckbox(By locator, boolean shouldBeChecked) {
    WebElement cb = clickable(locator);
    if (cb.isSelected() != shouldBeChecked) cb.click();
}

// Radio group
driver.findElements(By.name("gender")).stream()
    .filter(r -> "female".equals(r.getAttribute("value")))
    .findFirst().orElseThrow().click();

// Select all checkboxes on the page
driver.findElements(By.cssSelector("input[type='checkbox']"))
    .forEach(cb -> { if (!cb.isSelected()) cb.click(); });
```

Custom-styled checkboxes: many UI kits hide the real `<input>` (opacity 0) and display a styled `<span>/<label>`. Clicking the input throws `ElementNotInteractableException`. The fix: **click the label** (`//label[@for='terms']`) but read the state from the input's `isSelected()`. That exact scenario comes up in interviews a lot.

CORE

TRAP

Q99**Browser navigation: `get()` vs `navigate().to()`, and the other navigate methods.**

```
driver.get("https://example.com");           // Loads and WAITS for page Load
driver.navigate().to("https://example.com"); // same thing, but part of the history API
driver.navigate().back();
driver.navigate().forward();
driver.navigate().refresh();

driver.getTitle();
driver.getCurrentUrl();
driver.getPageSource();
driver.getWindowHandle();
```

The real difference: functionally they are almost identical today — both load the URL and wait for the page load. `navigate()` exposes the browser **history** (back/forward/refresh), which `get()` does not. Historically `get()` was described as blocking until load completes while `navigate().to()` did not guarantee it; in modern W3C-compliant drivers both honour the page load strategy.

Practical framework note: `navigate().back()` is convenient but risky in tests — it can restore a cached page and cause stale elements. Navigating to a URL directly is usually more deterministic than relying on browser history.

CORE

Q100

driver.close() vs driver.quit().

	CLOSE()	QUIT()
Closes	Only the current window/tab	All windows opened by that driver
WebDriver session	Stays alive (unless it was the last window)	Terminated
Driver process	Keeps running	chromedriver.exe is killed
After calling it	You must <code>switchTo().window(...)</code>	Any command throws — the session is gone

The real-world consequence: using only `close()` in teardown leaks a chromedriver process per test. After a few hundred tests your CI agent runs out of memory and the build starts failing for "no reason". Always `quit()` in `@AfterMethod/@AfterClass`, inside a `finally` or a null-safe helper.

```
@AfterMethod(alwaysRun = true)
public void tearDown() {
    DriverFactory.quit();    // quit() + ThreadLocal.remove()
}
```

CORE

TRAP

```
driver.manage().getCookies(); // Set<Cookie>
Cookie session = driver.manage().getCookieNamed("JSESSIONID");
driver.manage().addCookie(new Cookie("consent", "accepted"));
driver.manage().deleteCookieNamed("promo-banner");
driver.manage().deleteAllCookies();

// Build a cookie with full attributes
Cookie c = new Cookie.Builder("token", jwt)
    .domain("qa.example.com").path("/")
    .expiresOn(Date.from(Instant.now().plus(1, ChronoUnit.DAYS)))
    .isHttpOnly(true).isSecure(true).build();
driver.manage().addCookie(c);
```

The high-value use case — skipping UI login

```
// 1. Get a session token from the API (fast, reliable, no UI flakiness)
String token = RestAssured.given()
    .contentType(ContentType.JSON)
    .body(Map.of("email", user, "password", pass))
    .post(baseUrl + "/api/login").jsonPath().getString("token");

// 2. Seed it into the browser and land straight on the page under test
driver.get(baseUrl); // must be on the domain before adding a cookie
driver.manage().addCookie(new Cookie("auth_token", token));
driver.navigate().to(baseUrl + "/dashboard");
```

This removes 5–8 seconds and a whole class of flakiness from *every* test. Mention that you keep **one** dedicated UI login test so the login page itself is still covered.

You cannot add a cookie for a domain you are not currently on — Selenium throws `InvalidCookieDomainException`. Always `driver.get(baseUrl)` first.

ADVANCED

```

ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new");           // Chrome 109+ new headless mode
options.addArguments("--window-size=1920,1080");  // headless has NO default size – set it
options.addArguments("--disable-gpu");
options.addArguments("--no-sandbox");             // needed in Docker/CI containers
options.addArguments("--disable-dev-shm-usage");  // avoids /dev/shm crashes in Docker
options.addArguments("--incognito");
options.addArguments("--disable-notifications");
options.addArguments("--disable-popup-blocking");
options.addArguments("--remote-allow-origins=*");
options.addArguments("--lang=en-GB");
options.setAcceptInsecureCerts(true);             // self-signed SSL certs in QA envs
options.setPageLoadStrategy(PageLoadStrategy.EAGER);
options.addExtensions(new File("adblock.crx"));

Map<String,Object> prefs = new HashMap<>();
prefs.put("profile.default_content_setting_values.notifications", 2); // 2 = block
prefs.put("credentials_enable_service", false);                     // no save-password popup
prefs.put("profile.password_manager_enabled", false);
options.setExperimentalOption("prefs", prefs);
options.setExperimentalOption("excludeSwitches", List.of("enable-automation"));

WebDriver driver = new ChromeDriver(options);

```

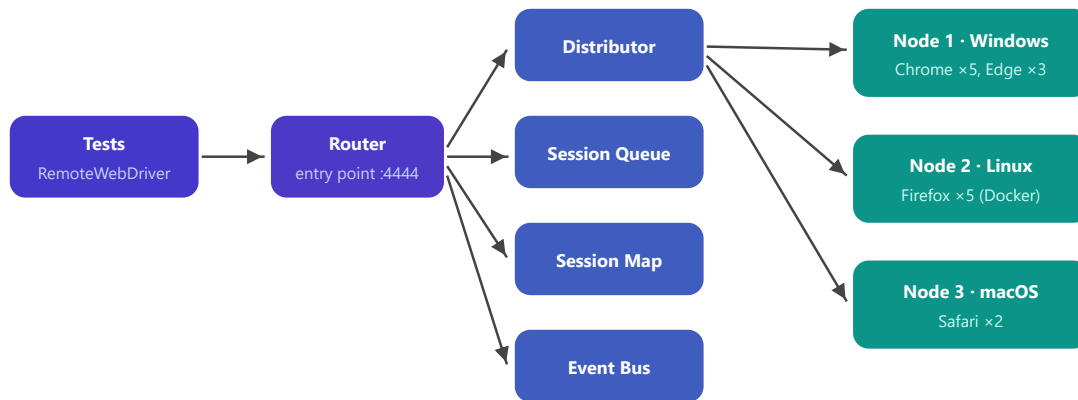
Pros and cons of headless — say both

Pros: ~20–40% faster, less memory, runs on CI agents with no display server, enables higher parallelism. **Cons:** you cannot watch the failure, some rendering/layout and CSS-hover bugs only appear headed, screenshots can differ, and a few sites detect and block headless traffic.

My approach: headless in CI, headed locally when debugging, controlled by a single `Dheadless=true` property.

CORE

Selenium Grid 4 — distributed mode



Standalone mode packs all of these into one process — `java -jar selenium-server.jar standalone`

Figure 7 — Grid 4 components. For an interview, knowing Router → Session Queue → Distributor → Node is enough; the Event Bus is how they communicate.

Grid lets you run tests on **remote machines**, in **parallel**, across different **browser/OS combinations** from a single entry point. Your test code doesn't change — only the driver instantiation.

Grid 4 modes

- **Standalone** — everything in one process on one machine. Simplest, good for local parallel runs.
- **Hub & Node** — classic: one hub, many nodes registering to it.
- **Distributed** — Router, Distributor, Session Queue, Session Map, Event Bus and Nodes as separate scalable services. For large enterprise grids on Kubernetes.

```
# start
java -jar selenium-server-4.x.jar standalone
java -jar selenium-server-4.x.jar hub
java -jar selenium-server-4.x.jar node --hub http://192.168.1.10:4444
```

```
ChromeOptions options = new ChromeOptions();
options.setPlatformName("Windows 11");
options.setBrowserVersion("latest");
WebDriver driver = new RemoteWebDriver(new URL("http://grid-host:4444"), options);
((RemoteWebDriver) driver).setFileDetector(new LocalFileDetector()); // for uploads
```

Grid UI at <http://localhost:4444/ui> shows sessions, queue and node capacity. Docker Compose ([selenium/hub](#) + [selenium/node-chrome](#)) is the usual way to run it in CI; **Selenoid** and cloud vendors (BrowserStack, LambdaTest, Sauce Labs) are the managed alternatives — worth naming because most companies buy rather than build.

Three ingredients: a **driver factory**, **externalised configuration**, and **parallel TestNG XML**.

```
// 1. Factory driven by a parameter, never by an if-else scattered in tests
public static WebDriver create(String browser, boolean headless) {
    switch (browser.toLowerCase()) {
        case "chrome": {
            ChromeOptions o = new ChromeOptions();
            if (headless) o.addArguments("--headless=new", "--window-size=1920,1080");
            return new ChromeDriver(o);
        }
        case "firefox": {
            FirefoxOptions o = new FirefoxOptions();
            if (headless) o.addArguments("-headless");
            return new FirefoxDriver(o);
        }
        case "edge": return new EdgeDriver(new EdgeOptions());
        case "safari": return new SafariDriver();
        default: throw new IllegalArgumentException("Unsupported browser: " + browser);
    }
}
```

```
<!-- 2. testng.xml – one <test> per browser, all running in parallel -->
<suite name="CrossBrowser" parallel="tests" thread-count="3">
    <test name="Chrome"> <parameter name="browser" value="chrome"/>
        <classes><class name="tests.CheckoutTest"/></classes></test>
    <test name="Firefox"> <parameter name="browser" value="firefox"/>
        <classes><class name="tests.CheckoutTest"/></classes></test>
    <test name="Edge"> <parameter name="browser" value="edge"/>
        <classes><class name="tests.CheckoutTest"/></classes></test>
</suite>
```

```
@BeforeMethod
@Parameters({"browser"})
public void setUp(@Optional("chrome") String browser) {
    DriverFactory.init(browser, Boolean.parseBoolean(System.getProperty("headless", "false")));
}
```

Add the strategy point: "I don't run the **whole** suite on every browser — that triples runtime for little value. I run the full suite on Chrome and a curated cross-browser subset (critical journeys + browser-sensitive UI) on Firefox, Edge and Safari nightly."

How do you handle SSL certificate errors, browser notification popups and basic-auth dialogs?

```
// --- SSL / self-signed certificates in QA environments ---
ChromeOptions o = new ChromeOptions();
o.setAcceptInsecureCerts(true);           // W3C standard way
o.addArguments("--ignore-certificate-errors"); // Chrome-specific fallback

// --- Notification / Location / camera permission popups (these are browser UI, not DOM) ---
Map<String, Object> prefs = new HashMap<>();
prefs.put("profile.default_content_setting_values.notifications", 2); // 1=allow 2=block
prefs.put("profile.default_content_setting_values.geolocation", 2);
prefs.put("profile.default_content_setting_values.media_stream_camera", 2);
o.setExperimentalOption("prefs", prefs);
o.addArguments("--disable-notifications");

// --- HTTP Basic authentication ---
// Option 1: credentials in the URL (works, but deprecated in modern Chrome)
driver.get("https://user:pass@staging.example.com");

// Option 2 (recommended, Selenium 4 CDP):
DevTools devTools = ((ChromeDriver) driver).getDevTools();
devTools.createSession();
((HasAuthentication) driver).register(
    () -> new UsernameAndPassword("user", "pass"));
driver.get("https://staging.example.com");
```

Key insight to state: **anything drawn by the browser chrome rather than the page cannot be located with WebDriver** — notification prompts, the print dialog, the OS file picker, the download bar. You must prevent them via options/preferences, or drive them with CDP / Robot / AutoIT.

Web components (Polymer, Lit, Salesforce Lightning, and native `<video>` controls) encapsulate their internals in a **shadow root**. A normal XPath/CSS from the document cannot pierce it — this is a very common "my locator works in DevTools but not in Selenium" cause.

```
// Selenium 4.x native API (open shadow roots only)
WebElement host = driver.findElement(By.cssSelector("my-app"));
SearchContext shadow = host.getShadowRoot();
WebElement inner = shadow.findElement(By.cssSelector("#login-btn"));
inner.click();

// Nested shadow roots – chain them
WebElement lvl2 = host.getShadowRoot()
    .findElement(By.cssSelector("app-header")).getShadowRoot()
    .findElement(By.cssSelector("button.menu"));

// Pre-4.0 / fallback: JavaScript
WebElement el = (WebElement) ((JavascriptExecutor) driver).executeScript(
    "return document.querySelector('my-app').shadowRoot.querySelector('#login-btn');");
```

Closed shadow roots (mode: 'closed') cannot be accessed at all — not by Selenium, not by JavaScript from outside. If a team ships closed shadow roots, the only path is asking developers to expose test hooks. Note that **Playwright pierces open shadow DOM automatically** with its normal CSS/text selectors — a genuine advantage worth mentioning if they ask "why would you pick Playwright?"

ADVANCED

```
// 1. Native HTML tooltip – it's just the title attribute, no hover needed
String tip = driver.findElement(By.id("help")).getAttribute("title");
Assert.assertEquals(tip, "Enter your 10-digit mobile number");

// 2. Custom JS/CSS tooltip – must hover, then read a separate element
new Actions(driver).moveToElement(driver.findElement(By.id("help"))).perform();
WebElement tooltip = wait.until(
    ExpectedConditions.visibilityOfElementLocated(By.cssSelector(".tooltip-body")));
Assert.assertTrue(tooltip.getText().contains("10-digit"));

// 3. Element present in DOM but hidden – getText() returns "" for hidden elements!
String hidden = (String) ((JavascriptExecutor) driver)
    .executeScript("return arguments[0].textContent;", el); // textContent works
String value = el.getAttribute("value"); // for inputs
```

Important detail: `getText()` returns only the **rendered, visible** text. For an element with `display:none` it returns an empty string, which silently makes assertions pass or fail for the wrong reason. Use `getAttribute("textContent")` or `getAttribute("innerText")` when you deliberately need hidden text.

CORE

TRAP

A classic practical exercise. Selenium finds the links; a plain HTTP client checks the status codes (do **not** navigate to each link with the browser — it would take forever).

```
List<WebElement> links = driver.findElements(By.tagName("a"));
List<String> broken = new ArrayList<>();

Set<String> urls = links.stream()
    .map(l -> l.getAttribute("href"))
    .filter(Objects::nonNull)
    .filter(h -> h.startsWith("http"))           // skip mailto:, tel:, javascript:, #
    .collect(Collectors.toCollection(LinkedHashSet::new)); // dedupe

for (String url : urls) {
    HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
    conn.setRequestMethod("HEAD");               // HEAD is enough – no body downloaded
    conn.setConnectTimeout(5000);
    conn.setReadTimeout(5000);
    conn.connect();
    int code = conn.getResponseCode();
    if (code >= 400) broken.add(url + " -> " + code + " " + conn.getResponseMessage());
    conn.disconnect();
}
Assert.assertTrue(broken.isEmpty(), "Broken links found:\n" + String.join("\n", broken));
```

Improvements to mention: run the checks in a thread pool (ExecutorService) because they are I/O bound; treat 401/403 as "protected, not broken"; follow redirects deliberately; and also validate `<img src>`, `<script src>` and `<link href>` the same way.

The correct answer is **you don't automate them — you remove the blocker with the team's help**. Trying to "solve" a CAPTCHA defeats its purpose and is unreliable.

CAPTCHA

- Ask dev/DevOps to **disable CAPTCHA in non-production environments**, or whitelist the test environment/IP. This is the standard industry practice.
- Use a **test key** — reCAPTCHA provides official test site/secret keys that always pass in test environments.
- Bypass via an API-based login for all other tests, keeping one manual test for the CAPTCHA itself.
- Third-party solving services exist but are slow, cost money, and are a poor engineering answer in an interview.

OTP / 2FA

- Use a fixed **test OTP** (e.g. `000000`) enabled for test accounts in lower environments — the most common real solution.
- Fetch the OTP from a **backend API / test endpoint**, or query the database directly with JDBC.
- Read it from a test mail service (Mailinator, MailHog, Mailosaur API) or an SMS sandbox (Twilio test credentials).
- For TOTP-based 2FA, generate the code in the test from the shared secret using a library (e.g. `totp-java`) — this is fully legitimate and impressive to mention.

Frame it as a conversation, not a hack: *"I raise testability as a requirement during refinement. A feature that cannot be tested in a lower environment isn't 'done' — and the team agreed to a test-mode flag for exactly that reason."*

CORE

TRAP

They locate an element by its **visual position** relative to another element, using the browser's rendered box coordinates rather than the DOM structure.

```
import static org.openqa.selenium.support.locators.RelativeLocator.with;

WebElement password = driver.findElement(with(By.tagName("input")).below(By.id("email")));
WebElement submit   = driver.findElement(with(By.tagName("button")).below(By.id("password")));
WebElement label    = driver.findElement(with(By.tagName("label")).toLeftOf(By.id("qty")));
WebElement helpIcon = driver.findElement(with(By.tagName("svg")).toRightOf(By.id("qty")));
WebElement nearby   = driver.findElement(with(By.tagName("input")).near(By.id("label"), 60));

// chainable
WebElement cell = driver.findElement(
    with(By.tagName("td")).below(By.id("header-status")).toRightOf(By.id("cell-id")));
```

Five methods: `above()`, `below()`, `toLeftOf()`, `toRightOf()`, `near()` (default 50px, overloadable).

Be honest about the limitations — that's what a senior candidate does. They depend on the rendered layout, so they break on responsive re-flow, different screen sizes, and headless-vs-headed rendering differences. They are also slower (Selenium injects JS to compute rectangles). I treat them as a fallback for legacy pages with no usable attributes, not as a primary strategy.

ADVANCED

```
DevTools devTools = ((ChromeDriver) driver).getDevTools();
devTools.createSession();

// 1. Capture browser console errors during the test
devTools.send(Log.enable());
devTools.addListener(Log.entryAdded(), entry ->
    logger.warn("CONSOLE {}: {}", entry.getLevel(), entry.getText()));

// 2. Simulate a slow 3G network – test loaders and timeouts realistically
devTools.send(Network.enable(Optional.empty(), Optional.empty(), Optional.empty()));
devTools.send(Network.emulateNetworkConditions(
    false, 200, 250_000, 100_000, Optional.of(ConnectionType.CELLULAR3G)));

// 3. Block requests (e.g. analytics/ads) to speed tests up and remove flakiness
devTools.send(Network.setBlockedURLs(List.of("*google-analytics.com*", "*.gif")));

// 4. Mock geolocation
devTools.send(Emulation.setGeolocationOverride(
    Optional.of(12.9716), Optional.of(77.5946), Optional.of(1)));

// 5. Intercept & stub a network response (mock the backend from the UI test)
devTools.send(Fetch.enable(Optional.empty(), Optional.empty()));
devTools.addListener(Fetch.requestPaused(), req -> { /* fulfil with stub JSON */ });

// 6. Capture all requests/responses for API-level assertions from a UI test
devTools.addListener(Network.responseReceived(),
    r -> logger.info("{} {}", r.getResponse().getStatus(), r.getResponse().getUrl()));

// 7. Basic auth & device emulation
devTools.send(Emulation.setDeviceMetricsOverride(390, 844, 3, true, Optional.empty(), ...));
```

Important caveat to state: CDP is Chromium-only and version-bound — Selenium ships versioned CDP packages, so an upgrade can break your code. That is precisely why Selenium is moving toward the cross-browser **WebDriver BiDi** standard, and why Playwright (built on CDP-style bi-directional communication from day one) does these things more naturally.

ADVANCED

How do you validate something in the backend from a UI test — API and database checks?

This is a grey-box question that separates a "clicker" from an SDET. A UI assertion alone can pass while the data is wrong.

```
// UI action
checkoutPage.placeOrder();
String orderId = confirmationPage.getOrderId(); // e.g. ORD-2026-0042

// 1. API verification – the order really exists with the right total
Response res = RestAssured.given()
    .header("Authorization", "Bearer " + token)
    .get(baseUrl + "/api/orders/" + orderId);
res.then().statusCode(200)
    .body("status", equalTo("CONFIRMED"))
    .body("totalAmount", equalTo(1499.00f));

// 2. Database verification – the row and the audit trail
try (Connection con = DriverManager.getConnection(url, user, pass);
     PreparedStatement ps = con.prepareStatement(
         "SELECT status, total_amount FROM orders WHERE order_ref = ?")) {
    ps.setString(1, orderId);
    try (ResultSet rs = ps.executeQuery()) {
        Assert.assertTrue(rs.next(), "Order row missing in DB for " + orderId);
        Assert.assertEquals(rs.getString("status"), "CONFIRMED");
        Assert.assertEquals(rs.getBigDecimal("total_amount"),
            new BigDecimal("1499.00"));
    }
}
```

Also mention: use a **read-only DB user** for tests, always use PreparedStatement (never string concatenation), close resources with try-with-resources, and keep credentials in environment variables / a secrets manager — never in the repo. Those four points get remembered.

ADVANCED

Answer as an ordered plan — measure first, then optimise.

1. **Measure.** Get per-test durations from the TestNG/Allure report and find the top 20 slowest tests. Optimise those, not everything.
2. **Parallelise.** TestNG `parallel="methods"` with a thread count matched to CPU/grid capacity. This alone often gives 4–8×. Prerequisite: `ThreadLocal` driver and fully independent tests.
3. **Kill all** `Thread.sleep()` and replace with explicit waits. In most legacy suites this is 30–50% of the runtime.
4. **Skip the UI for setup.** Log in via API + cookie injection; create the test data via API instead of clicking through wizards. Typically saves 5–15s per test.
5. **Push tests down the pyramid.** Field validations and error messages become API/component tests; only journeys stay in the UI layer.
6. **Run headless** and use `PageLoadStrategy.EAGER`; block analytics/ads/third-party scripts via CDP.
7. **Split the suite** — a <10 minute smoke set on every commit, the full regression nightly. Tag with TestNG groups.
8. **Scale horizontally** with a Docker Grid or a cloud provider; shard tests across CI agents.
9. **Remove duplication** — audit for tests that assert the same thing through different paths.
10. **Fix the flaky ones** instead of retrying them; retries hide problems and double the runtime of failing tests.

Quantify it if you can: "We took our regression from 3h 40m to 34 minutes: parallelism 8×, API-based login, and moving 120 validation checks to the API layer." Numbers are what interviewers remember.

CORE

ADVANCED

ROOT CAUSE	FIX
Fixed sleeps / no waits	Explicit waits on conditions , never on time
Waiting for presence instead of readiness	Wait for <code>elementToBeClickable</code> , spinner invisibility, or a specific text
Test order dependency / shared data	Every test creates and cleans its own data; unique identifiers per run
Shared static driver in parallel runs	<code>ThreadLocal<WebDriver></code>
Brittle locators (index-based, absolute XPath, generated ids)	<code>data-testid</code> attributes, semantic locators
Animations & transitions	Wait for the animation-end state; or disable CSS animations in test builds
Third-party dependencies (ads, analytics, payment sandbox)	Block or stub them via CDP/proxy
Environment instability, slow CI agents	Right-size timeouts for CI, containerise, monitor agent health
Date/time and timezone assumptions	Relative dates, fixed timezone in browser options, avoid "today" boundary cases
Stale element on re-render	Re-find elements; don't cache WebElements across actions

Process, not just code

- **Quarantine** — move a flaky test out of the blocking pipeline into a separate suite with an owner and a due date. Never let it just get retried forever.
- **Track flakiness as a metric** (pass rate over the last 20 runs) and treat <98% as a bug with a ticket.
- **Rerun-failed is a diagnostic, not a cure** — I use `IRetryAnalyzer` but I report retried tests distinctly so they cannot be silently ignored.

This is one of the highest-value questions for a 5-year candidate. Having a clear, opinionated flakiness policy is exactly what a lead is listening for.

CORE

ADVANCED

Q115

How do you handle authentication popups, browser profiles and running as a specific user?

```
// Reuse an existing Chrome profile (keeps logins, extensions, cookies)
ChromeOptions o = new ChromeOptions();
o.addArguments("user-data-dir=C:/Users/me/AppData/Local/Google/Chrome/User Data");
o.addArguments("profile-directory=Profile 2");

// Or a clean, disposable profile per run - better for test isolation
Path tmp = Files.createTempDirectory("chrome-profile");
o.addArguments("user-data-dir=" + tmp);

// Mobile emulation (responsive testing without a device)
Map<String,String> device = Map.of("deviceName", "iPhone 14 Pro");
o.setExperimentalOption("mobileEmulation", device);

// Selenium 4 auth interception (works for HTTP basic auth dialogs)
((HasAuthentication) driver).register(UsernameAndPassword.of("qa_user", "secret"));
```

Reusing your personal profile is convenient locally but a bad idea in CI: it introduces shared state (cookies from the last run), can lock the profile directory when two runs overlap, and makes failures unreproducible. Use a fresh temp profile per session.

ADVANCED

SECTION 4

TestNG, Framework Design, Maven, Git & CI/CD

Q116 – Q150 · "Explain your framework" is the single most important question in a 5-year automation interview. Everything here feeds into that one answer.

Q116

TestNG vs JUnit — why did you choose TestNG?

FEATURE	TESTNG	JUNIT 5
Suite configuration	<code>testng.xml</code> — grouping, parameters, parallelism declaratively	No XML; uses tags + Maven config / <code>@Suite</code>
Parallel execution	Built in: methods / classes / tests / instances	Available since 5 via <code>junit-platform.properties</code>
Data-driven	<code>@DataProvider</code> (can be parallel, can take Method/ITestContext)	<code>@ParameterizedTest</code> with sources
Dependencies	<code>dependsOnMethods</code> , <code>dependsOnGroups</code>	Not supported by design (tests should be independent)
Grouping	<code>groups = {"smoke","regression"}</code>	<code>@Tag</code>
Retry on failure	<code>IRetryAnalyzer</code> built in	Needs an extension
Soft assertions	<code>SoftAssert</code> included	Needs AssertJ / <code>assertAll</code>
Setup scopes	Suite / Test / Class / Method / Group	All / Each

Honest answer: "TestNG's suite XML, native parallelism and flexible data providers fit UI automation better, which is why it dominates Selenium projects. JUnit 5 has closed most of the gap and is the standard for unit tests, so I'm comfortable with both — the choice depends on the team's existing stack, not dogma."

CORE

```

@BeforeSuite      → once, before everything in the <suite>
@BeforeTest       → once per <test> tag
@BeforeClass      → once per test class
@BeforeMethod     → before EVERY @Test method
@Test
@AfterMethod
@AfterClass
@AfterTest
@AfterSuite

```

Plus `@BeforeGroups` / `@AfterGroups` (run around methods belonging to a named group),
`@DataProvider` , `@Parameters` , `@Factory` , `@Listeners` .

```

public class BaseTest {
    @BeforeSuite(alwaysRun = true)
    public void startReport() { ExtentManager.init(); }

    @BeforeMethod(alwaysRun = true)
    @Parameters({"browser"})
    public void setUp(@Optional("chrome") String browser, Method m) {
        DriverFactory.init(browser);
        DriverFactory.get().get(Config.get("base.url"));
        logger.info("Starting test: {}", m.getName());
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown(ITestResult result) {
        if (result.getStatus() == ITestResult.FAILURE) Screenshot.capture(result);
        DriverFactory.quit();
    }

    @AfterSuite(alwaysRun = true)
    public void flushReport() { ExtentManager.get().flush(); }
}

```

`alwaysRun = true` matters: without it, a `@Before/@After` method is **skipped** if a method it depends on fails — which means your driver is never quit and the process leaks. Always set it on teardown methods.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="RegressionSuite" parallel="methods" thread-count="5"
       verbose="1" preserve-order="true" configfailurepolicy="continue">

  <parameter name="env"      value="qa"/>
  <parameter name="browser" value="chrome"/>

  <listeners>
    <listener class-name="listeners.TestListener"/>
    <listener class-name="listeners.RetryTransformer"/>
  </listeners>

  <test name="SmokeTests">
    <groups>
      <run>
        <include name="smoke"/>
        <exclude name="wip"/>
      </run>
    </groups>
    <packages><package name="tests.smoke.*"/></packages>
  </test>

  <test name="CheckoutTests">
    <classes>
      <class name="tests.CheckoutTest">
        <methods>
          <include name="placeOrderWithCard"/>
          <exclude name="placeOrderWithWallet"/>
        </methods>
      </class>
    </classes>
  </test>
</suite>
```

Hierarchy: **<suite>** → **<test>** → **<classes>** / **<packages>** → **<methods>**. Parameters can be set at suite or test level (test level wins).

Q119

Explain the @Test attributes: priority, dependsOnMethods, invocationCount, timeout, enabled, expectedExceptions.

```
@Test(priority = 1)                // lower number runs first; default 0
@Test(priority = 2, description = "Verify checkout with a saved card")
@Test(enabled = false)              // skip this test entirely
@Test(groups = {"smoke", "checkout"})
@Test(dependsOnMethods = {"login"}) // runs only if login PASSED; else SKIPPED
@Test(dependsOnGroups = {"setup"})
@Test(alwaysRun = true, dependsOnMethods = {"login"}) // runs even if login failed
@Test(invocationCount = 5)          // run 5 times
@Test(invocationCount = 10, threadPoolSize = 3)      // 10 times across 3 threads
@Test(timeout = 5000)               // fail if it exceeds 5000 ms
@Test(expectedExceptions = NoSuchElementException.class)
@Test(retryAnalyzer = RetryAnalyzer.class)
@Test(dataProvider = "loginData", dataProviderClass = TestData.class)
```

Ordering rules — a favourite trap

- Without **priority**, TestNG runs methods in **alphabetical order**, not the order written in the file.
- **priority** is honoured only *within* the same class/test tag; dependencies take precedence over priority.
- Two methods with the same priority run alphabetically.
- If a method with **dependsOnMethods** has its dependency **fail**, the dependent is marked **SKIPPED** (not failed).

Opinion worth stating: "I avoid dependsOnMethods for chaining business steps. It creates order-dependent tests that cannot run in parallel and produce cascading skips that hide the real failure count. I use it only for genuine environment prerequisites."

CORE**TRAP**


```
@Test(groups = {"smoke", "regression", "checkout"})
public void placeOrder() { }

@Test(groups = {"regression"})
public void applyExpiredCoupon() { }

@BeforeGroups("checkout")
public void seedCatalogue() { }
```

```
<!-- run smoke only, excluding known-broken -->
<groups><run>
  <include name="smoke"/>
  <exclude name="flaky"/>
</run></groups>

<!-- group of groups -->
<groups>
  <define name="release-gate">
    <include name="smoke"/><include name="payments"/>
  </define>
  <run><include name="release-gate"/></run>
</groups>
```

```
# from the command line (Maven Surefire)
mvn test -Dgroups=smoke
mvn test -DexcludedGroups=flaky
mvn test -DsuiteXmlFile=testng-regression.xml -Dbrowser=firefox -Denv=staging
```

Typical grouping scheme I use: **smoke** (≤ 10 min, every commit), **regression** (nightly), **e2e** (pre-release), plus feature groups (**payments** , **search**) so a developer can run just the area they changed.

```

@DataProvider(name = "loginData")
public Object[][] loginData() {
    return new Object[][] {
        {"valid@test.com", "Correct@123", true, ""},
        {"valid@test.com", "Wrong@123", false, "Invalid credentials"},
        {"unknown@test.com", "Correct@123", false, "Invalid credentials"},
        {"", "Correct@123", false, "Email is required"},
    };
}

@Test(dataProvider = "loginData")
public void login(String email, String pwd, boolean success, String error) {
    LoginPage page = new LoginPage(driver);
    page.login(email, pwd);
    if (success) Assert.assertTrue(new HomePage(driver).isLoaded());
    else Assert.assertEquals(page.getError(), error);
}

// ---- Reading from Excel/CSV/JSON instead of hardcoding ----
@DataProvider(name = "orders")
public Iterator<Object[]> orders() { // Iterator = Lazy, memory friendly
    return ExcelReader.readSheet("testdata/orders.xlsx", "Checkout")
        .stream().map(row -> new Object[]{row}).iterator();
}

@Test(dataProvider = "orders")
public void checkout(Map<String,String> data) { ... }

// ---- Data provider aware of the test method & suite context ----
@DataProvider(name = "byMethod")
public Object[][] byMethod(Method m, ITestContext ctx) {
    String env = ctx.getCurrentXmlTest().getParameter("env");
    return TestData.forMethod(m.getName(), env);
}

// ---- Parallel data provider ----
@DataProvider(name = "users", parallel = true)
public Object[][] users() { ... }

```

Two things to add: (1) each data row appears as a **separate test result**, so failures are precise; (2) implement `ITestListener.onTestStart` or override `toString()` in your data object so reports show meaningful names instead of "login[0], login[1]".

Q122

@Parameters vs @DataProvider — when do you use each?

	@PARAMETERS	@DATAPROVIDER
Source	<code>testng.xml</code> or Maven <code>-D</code>	A Java method (so: Excel, DB, API, JSON, Faker)
Data type	Strings only	Any object type
Runs the test	Once per configuration	Once per data row
Typical use	Environment config: browser, env, URL	Test data: multiple input combinations

They are complementary: `@Parameters` tells you *where* to run; `@DataProvider` tells you *what* data to run with. A cross-browser data-driven suite uses both.

CORE

```
<suite name="S" parallel="methods" thread-count="5"> <!-- every @Test in its own thread -->
<suite name="S" parallel="classes" thread-count="5"> <!-- each class in one thread -->
<suite name="S" parallel="tests" thread-count="3"> <!-- each <test> tag -->
<suite name="S" parallel="instances" thread-count="4"> <!-- each @Factory instance -->
```

Prerequisites — this is what they're really asking

1. **ThreadLocal WebDriver** — no static/shared driver (Q61).
2. **Independent tests** — no `dependsOnMethods`, no shared mutable state, no assumption about execution order.
3. **Isolated test data** — unique users/records per test, or per-thread data pools. Two threads updating the same record is the classic parallel bug.
4. **Thread-safe reporting** — ExtentTest also needs a ThreadLocal, otherwise logs from different tests interleave.
5. **Capacity** — thread count matched to CPU/RAM and grid nodes. Too many threads makes everything slow and flaky; I usually start at (cores) and tune from measurements.

```
// Thread-safe Extent reporting
public class ExtentManager {
    private static final ThreadLocal<ExtentTest> TEST = new ThreadLocal<>();
    public static void set(ExtentTest t) { TEST.set(t); }
    public static ExtentTest getTest() { return TEST.get(); }
    public static void unload() { TEST.remove(); }
}
```

CORE

ADVANCED

INTERFACE	PURPOSE
<code>ITestListener</code>	Hooks on test start/success/failure/skip — screenshots, report logging
<code>ISuiteListener</code>	onStart/onFinish of the whole suite — init & flush reports
<code>IInvokedMethodListener</code>	Before/after every method including configuration methods
<code>IRetryAnalyzer</code>	Retry failed tests
<code>IAnnotationTransformer</code>	Modify annotations at runtime — the clean way to apply a retry analyzer to all tests
<code>IReporter</code>	Generate a custom report after the suite
<code>IExecutionListener</code>	Before/after the whole TestNG run (across suites)

```

public class TestListener implements ITestListener {
    @Override public void onTestStart(ITestResult r) {
        ExtentManager.set(ExtentManager.get().createTest(
            r.getMethod().getMethodName(), r.getMethod().getDescription()));
    }
    @Override public void onTestSuccess(ITestResult r) {
        ExtentManager.getTest().pass("Passed in " +
            (r.getEndMillis() - r.getStartMillis()) + " ms");
    }
    @Override public void onTestFailure(ITestResult r) {
        String b64 = ((TakesScreenshot) DriverFactory.get()).getScreenshotAs(OutputType.BASE64);
        ExtentManager.getTest().fail(r.getThrowable(),
            MediaEntityBuilder.createScreenCaptureFromBase64String(b64).build());
        ExtentManager.getTest().info("URL at failure: " + DriverFactory.get().getCurrentUrl());
    }
    @Override public void onTestSkipped(ITestResult r) {
        ExtentManager.getTest().skip(r.getThrowable());
    }
}

```

Register via `@Listeners(TestListener.class)` on the base class, in `testng.xml`, or globally through the `ServiceLoader` (`META-INF/services`) — the last option means nobody can forget it.

```

public class RetryAnalyzer implements IRetryAnalyzer {
    private int count = 0;
    private static final int MAX = 2;           // 1 original + 2 retries

    @Override public boolean retry(ITestResult result) {
        if (count < MAX) {
            count++;
            result.setAttribute("retried", count);    // so the report can show it
            LoggerFactory.getLogger(getClass()).warn(
                "Retrying {} - attempt {}/{}", result.getName(), count, MAX);
            return true;
        }
        return false;
    }
}

// Apply it to EVERY test automatically – no annotation needed on each method
public class RetryTransformer implements IAnnotationTransformer {
    @Override public void transform(ITestAnnotation ann, Class<?> c, Constructor<?> m,
                                   Method testMethod) {
        ann.setRetryAnalyzer(RetryAnalyzer.class);
    }
}

```

The judgement question that follows: "Isn't retrying just hiding flakiness?" Yes — if used blindly. My policy: retries exist to absorb genuine **infrastructure** noise (a grid node hiccup, a network blip), not product bugs. So I (a) cap retries at 2, (b) report retried tests as a distinct category rather than a clean pass, and (c) track a flaky-test list that must be fixed, not tolerated. Saying this converts a naive answer into a senior one.

CORE

ADVANCED

```
// HARD – stops the test at the first failure (default)
Assert.assertEquals(actual, expected, "Order total mismatch");
Assert.assertTrue(page.isLoaded(), "Dashboard did not load");
Assert.assertNotNull(orderId);
Assert.assertThrows(NoSuchElementException.class, () -> driver.findElement(gone));

// SOFT – collects failures and reports them all at the end
SoftAssert soft = new SoftAssert();
soft.assertEquals(invoice.getCustomerName(), "Ravi Kumar", "Name");
soft.assertEquals(invoice.getGst(), "18%", "GST");
soft.assertEquals(invoice.getTotal(), "₹1,499.00", "Total");
soft.assertEquals(invoice.getInvoiceDate(), today, "Date");
soft.assertAll(); // MUST be called or failures are silently swallowed
```

Rule I use: hard assert for anything that makes the rest of the test meaningless (the page didn't load, the order wasn't created). Soft assert for independent field-level verifications on the same screen, where I want one run to report all mismatches instead of five debug cycles.

The classic bug: forgetting `soft.assertAll()` — the test passes green while every assertion failed. Some teams put `assertAll()` in an `@AfterMethod` with a per-thread `SoftAssert` to make it impossible to forget.

Always pass the **message** parameter. "expected [true] but found [false]" costs 20 minutes of debugging; "Dashboard did not load after login for user qa_01" costs zero.

```
// 1. Statically
@Test(enabled = false)

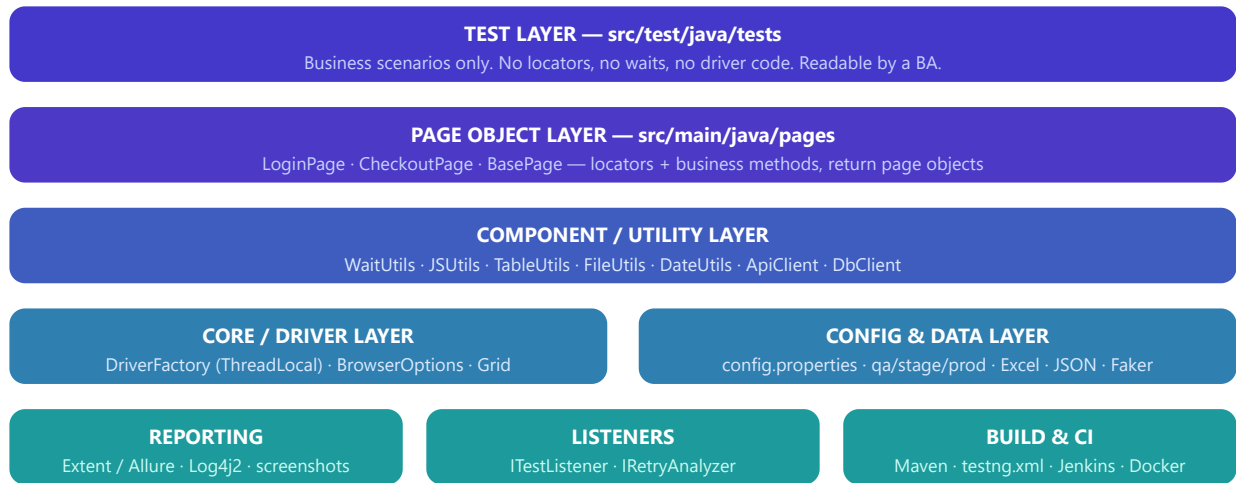
// 2. At runtime – throw SkipException (marks SKIP, not FAIL)
@Test
public void upiPayment() {
    if (!FeatureFlags.isEnabled("upi")) {
        throw new SkipException("UPI feature flag is off in " + Config.get("env"));
    }
    ...
}

// 3. Conditionally in a @BeforeMethod – skips the test it precedes
@BeforeMethod
public void checkEnv() {
    if (!"staging".equals(Config.get("env")))
        throw new SkipException("Runs on staging only");
}

// 4. Via groups / exclusion in testng.xml, or IAnnotationTransformer to disable
//     tests dynamically based on environment, browser or a Jira flag.
```

Prefer `SkipException` over silently returning: a skipped test is **visible** in the report, a return produces a false pass. Same principle as never swallowing exceptions.

Hybrid Framework Architecture (Selenium + TestNG + Maven)



Golden rule: a change to the UI should touch exactly ONE file — the page object.
A change to the test data or environment should touch ZERO Java files.

Figure 8 — Draw this when asked "explain your framework". Then walk top to bottom, naming one class per layer.

POM is a design pattern where every page (or significant component) of the application gets a class that holds its **locators** and exposes its **business actions** as methods. Tests call those methods and never see a locator.

Benefits

- **Single point of maintenance** — the login button's id changes, you edit one line in one file, not 40 tests.
- **Readable tests** — `loginPage.login(user, pass)` reads like the requirement.
- **Reusability** — the same page methods serve dozens of scenarios.
- **Separation of concerns** — test logic (what) vs page interaction (how).

```
public class LoginPage extends BasePage {
    // Locators – private, By-based (not cached WebElements)
    private final By email = By.id("email");
    private final By pwd = By.id("password");
    private final By signIn = By.cssSelector("[data-testid='signin']");
    private final By error = By.cssSelector(".alert-danger");

    public LoginPage(WebDriver driver) { super(driver); }

    public DashboardPage loginAs(String user, String pass) {
        type(email, user);
        type(pwd, pass);
        click(signIn);
        return new DashboardPage(driver);    // return the NEXT page = fluent navigation
    }
    public LoginPage loginExpectingFailure(String user, String pass) {
        type(email, user); type(pwd, pass); click(signIn);
        return this;    // stay on the same page
    }
    public String getError() { return visible(error).getText(); }
}

// The test – clean, business-readable, zero Selenium API
@Test
public void invalidPasswordShowsError() {
    String msg = new LoginPage(driver)
        .loginExpectingFailure("valid@test.com", "Wrong@123")
        .getError();
    Assert.assertEquals(msg, "Invalid credentials");
}
```

```

public class HomePage {
    @FindBy(id = "search")                private WebElement searchBox;
    @FindBy(css = "[data-testid='cart']")   private WebElement cart;
    @FindBy(xpath = "//div[@class='product']") private List<WebElement> products;

    // OR - matches EITHER Locator (union)
    @FindAll({ @FindBy(id = "submit"), @FindBy(name = "submit") })
    private WebElement submit;

    // AND - chained/nested (intersection): find .cart, then inside it find button
    @FindBys({ @FindBy(className = "cart"), @FindBy(tagName = "button") })
    private WebElement cartButton;

    @FindBy(id = "logo") @CacheLookup private WebElement logo;    // cached - see warning

    public HomePage(WebDriver driver) {
        PageFactory.initElements(driver, this);
        // or: PageFactory.initElements(new AjaxElementLocatorFactory(driver, 15), this);
    }
}

```

How it works: `initElements` replaces each annotated field with a **dynamic proxy**. The element is not located at construction time — it is located lazily each time you call a method on it, which is why PageFactory naturally resists stale elements.

@CacheLookup stores the located element after the first use. It is faster, but if the element is re-rendered you get `StaleElementReferenceException`. Only use it on genuinely static elements (a logo, a static header) — and honestly, in modern SPAs, almost never.

Should you use PageFactory at all? A defensible senior answer: "PageFactory is fine, but I often prefer plain By locators. They're explicit, easier to parameterise (`String.format` for dynamic rows), can be passed to explicit-wait conditions directly, and produce clearer failure messages. PageFactory is also no longer actively developed. Both are acceptable — I stay consistent within a project."

Do

- Model **business actions**, not UI mechanics: `searchAndOpenFirstResult("laptop")` , not `clickSearchButton()` .
- Return the next page object so navigation is expressed in the type system.
- Keep locators `private` ; expose only methods.
- Extract reusable widgets into **component objects** (HeaderComponent, DataTableComponent, DatePickerComponent) and compose them — a page object per URL is not enough for a modern SPA.
- Put common helpers (`click` , `type` , `visible` , `selectDropdown`) in `BasePage` .
- Expose **state getters** (`getErrorMessage()` , `isLoading()`) so the test can assert.

Do NOT

- **No assertions inside page objects.** A page object describes the page; the test decides what is correct. (One tolerated exception: an `isLoading()` self-check.) Assertions inside pages make them un reusable for negative tests.
- **No test data** hardcoded in pages.
- **No** `Thread.sleep()` , no raw driver management, no report/log formatting.
- No giant "god page" with 80 methods — split by component.
- Don't expose `WebElement` from public methods; that leaks Selenium into the test layer.

If they ask "how do you keep page objects from becoming unmaintainable?", answer: component objects + a strict rule that tests never import `org.openqa.selenium.*`. That import rule can even be enforced with ArchUnit or a Checkstyle rule — a great detail to mention.

TYPE	IDEA	REALITY CHECK
Linear / record-playback	Sequential scripts, no structure	Fast to create, impossible to maintain. Prototype only.
Modular	Break the app into modules with reusable functions	Better reuse; the ancestor of POM
Data-driven	Same script, data from Excel/CSV/DB/JSON	Essential for input-combination coverage
Keyword-driven	Actions written as keywords in a spreadsheet, interpreted by an engine	Sold as "non-programmers can automate"; in practice the engine becomes a project of its own. Rare in modern teams.
Hybrid	POM + data-driven + modular + reporting	What almost everyone actually builds. Say this one.
BDD	Gherkin feature files bound to step definitions (Cucumber, SpecFlow)	Great when business people genuinely read the scenarios; overhead if they don't

Answer template: "Mine is a hybrid framework — Page Object Model as the structural backbone, data-driven through TestNG DataProviders reading Excel/JSON, a driver factory with ThreadLocal for parallel execution, Extent reports with screenshots on failure, and Maven + Jenkins for execution."

Q132

Explain your automation framework end to end. (The most important question in the interview.)

Structure the answer: **tech stack** → **folder structure** → **execution flow** → **what makes it good** → **a problem you solved**. Aim for 3–4 minutes.

1. Stack

"Java 17, Selenium 4, TestNG, Maven, Page Object Model, Extent Reports, Log4j2, Apache POI + Jackson for data, RestAssured for API setup and verification, Docker Grid, Jenkins for CI, Git/GitHub with PR reviews."

2. Structure

```
src/main/java/  
├─ core/      DriverFactory (ThreadLocal), BrowserOptions, BasePage  
├─ pages/     LoginPage, ProductPage, CartPage, CheckoutPage  
├─ components/ HeaderComponent, DataTable, DatePicker  
├─ utils/     WaitUtils, JSUtils, ExcelReader, JsonReader, ApiClient, DbClient  
├─ config/    ConfigReader, Environment  
└─ constants/ Endpoints, Timeouts, Messages  
  
src/test/java/  
├─ base/      BaseTest (setup/teardown)  
├─ tests/     smoke/, regression/, e2e/  
├─ listeners/ TestListener, RetryAnalyzer, AnnotationTransformer  
└─ dataproviders/  
  
src/test/resources/  
├─ config/    qa.properties, staging.properties  
├─ testdata/  users.xlsx, orders.json  
├─ suites/    testng-smoke.xml, testng-regression.xml  
└─ log4j2.xml  
  
pom.xml · Jenkinsfile · docker-compose.yml · README.md
```

3. Execution flow

"Jenkins triggers `mvn clean test -Dsuite=smoke -Denv=qa -Dbrowser=chrome`. Surefire reads the testng.xml. `@BeforeSuite` initialises the Extent report; `@BeforeMethod` asks DriverFactory for a ThreadLocal driver configured from the properties file and navigates to the base URL. The test calls page-object methods; assertions live in the test. The `ITestListener` logs every step and, on failure, embeds a Base64 screenshot plus the current URL and browser console log. `@AfterMethod` quits the driver and clears the ThreadLocal. `@AfterSuite` flushes the report, which Jenkins publishes and emails to the team with a Slack notification."

4. Design decisions worth calling out

- Config is layered: `-D` system property → environment variable → properties file, so CI can override anything without a code change.
- Login is done once via API + cookie injection; only one dedicated test exercises the login UI.
- Test data is created per test via API and cleaned up in teardown — no shared fixtures.
- Zero implicit waits; a single wait strategy in `BasePage`.
- Tests may not import Selenium classes — enforced in code review.

5. Finish with a story

"The biggest improvement I made was cutting regression from 3h40m to 34 minutes — parallelising with ThreadLocal, replacing every sleep with explicit waits, and moving 120 field validations from UI to API tests."

Rehearse this out loud until it is smooth. Interviewers form their opinion of you during this answer, and every later question is drawn from what you said here — so only mention things you can defend in depth.

CORE

ADVANCED

Which design patterns have you used in your framework?

- **Page Object** — structural backbone (already covered).
- **Factory** — `DriverFactory.create(browser)` returns the right WebDriver without the caller knowing the concrete class.
- **Singleton** — ConfigReader, ExtentManager, DB connection pool.
- **Fluent / method chaining** — page methods return page objects:
`home.search("x").openFirst().addToCart()` .
- **Builder** — constructing complex test data objects readably:

```
User u = User.builder().name("Ravi").role("ADMIN").active(true).build();
```

- **Strategy** — pluggable behaviour: different payment strategies, different locator strategies, different report backends behind one interface.
- **Facade** — `ApiClient` hiding RestAssured details from the tests.
- **Decorator** — Selenium's own `EventFiringDecorator` / `WebDriverListener` to log or screenshot every command:

```
WebDriver decorated = new EventFiringDecorator<>(new  
LoggingListener()).decorate(driver);
```

- **Dependency Injection** — Picocontainer/Spring in Cucumber to share state between step-definition classes.

Don't list all nine. Name three or four you can genuinely explain with your own code — depth beats breadth, and interviewers probe whichever one you mention.

ADVANCED


```
# src/test/resources/config/qa.properties
base.url=https://qa.shop.example.com
api.url=https://qa-api.shop.example.com
db.url=jdbc:postgresql://qa-db:5432/shop
browser=chrome
headless=false
timeout.explicit=20
```

```
public final class Config {
    private static final Properties PROPS = load();

    private static Properties load() {
        String env = System.getProperty("env",
            System.getenv().getOrDefault("TEST_ENV", "qa"));
        Properties p = new Properties();
        try (InputStream in = Config.class.getResourceAsStream("/config/" + env + ".properties")) {
            if (in == null) throw new FrameworkException("No config for env: " + env);
            p.load(in);
        } catch (IOException e) { throw new FrameworkException("Config load failed", e); }
        return p;
    }

    /** Precedence: -D system property → environment variable → properties file */
    public static String get(String key) {
        String v = System.getProperty(key);
        if (v == null) v = System.getenv(key.toUpperCase().replace('.', '_'));
        if (v == null) v = PROPS.getProperty(key);
        if (v == null) throw new FrameworkException("Missing config key: " + key);
        return v;
    }

    public static int getInt(String key) { return Integer.parseInt(get(key)); }
    public static boolean getBool(String key) { return Boolean.parseBoolean(get(key)); }
}
```

```
mvn clean test -Denv=staging -Dbrowser=firefox -Dheadless=true
```

Secrets never go in the repo. Passwords, API keys and DB credentials come from environment variables injected by Jenkins credentials, HashiCorp Vault or AWS Secrets Manager. If they ask "where do you store the test user's password?", the wrong answer is "config.properties in Git". Say: environment variable from the CI credential store, with a local .env that is git-ignored.

Logging — Log4j2

```
private static final Logger log = LogManager.getLogger(CheckoutTest.class);
log.info("Placing order for user {}", user);
log.debug("Cart contents: {}", cart);
log.error("Payment failed", exception);
```

Levels: TRACE < DEBUG < INFO < WARN < ERROR < FATAL. Configure two appenders — console (INFO) and a rolling file (DEBUG) — in `log4j2.xml`. Include the thread name in the pattern (`%t`) so parallel runs stay readable.

Reporting

	EXTENT REPORTS	ALLURE
Output	Self-contained HTML — easy to email/attach	Results JSON + a generated static site
Setup	Code-driven (create tests, log steps manually)	Annotation-driven (<code>@Step</code> , <code>@Severity</code> , <code>@Issue</code> , <code>@Description</code>)
History/trends	Limited (Extent Klov for history)	Excellent — trends, retries, flakiness, categories
CI integration	Publish HTML artifact	Jenkins/GitLab Allure plugin
Best for	Quick, pretty, stakeholder-friendly single report	Long-running suites where trend analysis matters

```
// Allure – minimal and expressive
@Test @Severity(SeverityLevel.CRITICAL)
@Description("User can pay with a saved card")
@Link(name = "JIRA-1234", url = "https://jira/browse/JIRA-1234")
public void payWithSavedCard() { ... }

@Step("Login as {user}")
public void login(String user, String pass) { ... }

Allure.addAttachment("Screenshot", new ByteArrayInputStream(pngBytes));
```

Whichever you use, the report must answer three questions instantly: **what failed, what the screen looked like**, and **which build/environment it ran on**. Say that — it shows you think about the consumer of the report, not just the tooling.

CORE

Maven is a build and dependency management tool driven by `pom.xml` (Project Object Model). It downloads dependencies from a repository into your local `~/.m2` cache and runs a standard lifecycle.

Lifecycle phases (each runs all previous ones)

`validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`, plus `clean` (a separate lifecycle).

- `mvn clean test` — the usual command for a test project.
- `mvn clean install -DskipTests` — build and put the artifact in the local repo without running tests.

```
<build><plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.2.5</version>
    <configuration>
      <suiteXmlFiles>
        <suiteXmlFile>src/test/resources/suites/${suite}.xml</suiteXmlFile>
      </suiteXmlFiles>
      <systemPropertyVariables>
        <env>${env}</env>
        <browser>${browser}</browser>
      </systemPropertyVariables>
      <testFailureIgnore>>false</testFailureIgnore>
    </configuration>
  </plugin>
</plugins></build>

<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <suite>testng-smoke</suite>
  <env>qa</env>
  <browser>chrome</browser>
</properties>
```

	SUREFIRE	FAILSAFE
Runs in phase	<code>test</code>	<code>integration-test</code> / <code>verify</code>

Default pattern	<code>*Test.java</code> , <code>Test*.java</code>	<code>*IT.java</code> , <code>IT*.java</code>
On failure	Fails the build immediately	Fails at <code>verify</code> , so post-integration cleanup still runs
Use for	Unit tests	Integration/UI tests that need a deployed app

Gradle is the common alternative — say you're comfortable with both and that the concepts (dependency coordinates, lifecycle, plugins) map directly.

CORE

Q137

What are Maven dependency scopes, and which dependencies are in your pom?

SCOPE	AVAILABLE AT	PACKAGED?
<code>compile</code> (default)	compile, test, runtime	Yes
<code>provided</code>	compile, test	No — the container supplies it (e.g. servlet-api)
<code>runtime</code>	test, runtime (not compile)	Yes — e.g. a JDBC driver
<code>test</code>	test only	No — TestNG, Selenium in a test-only project
<code>system</code> / <code>import</code>	Legacy / BOM management	—

```
<!-- typical automation pom dependencies -->
org.seleniumhq.selenium : selenium-java      : 4.x
org.testng              : testng             : 7.x
io.rest-assured         : rest-assured       : 5.x
com.aventstack          : extentreports    : 5.x
io.qameta.allure        : allure-testng    : 2.x
org.apache.logging.log4j: log4j-core        : 2.x
org.apache.poi          : poi-ooxml      : 5.x
com.fasterxml.jackson.core : jackson-databind : 2.x
net.datafaker           : datafaker     : 2.x
org.projectlombok       : lombok       : (optional, provided)
org.postgresql          : postgresql    : (runtime)
```

Useful commands: `mvn dependency:tree` (find version conflicts), `mvn versions:display-dependency-updates`, `mvn -o` (offline). Mention **dependencyManagement** for centralising versions in a multi-module project.

CORE

```
git clone <url>                                git status
git checkout -b feature/JIRA-1234-checkout-tests
git add .                                       git commit -m "JIRA-1234: add checkout regression tests"
git pull --rebase origin main                  git push -u origin feature/JIRA-1234-checkout-tests
git log --oneline --graph --decorate -10
git diff / git diff --staged
git stash / git stash pop                      # park work to switch branches
git reset --soft HEAD~1                       # undo the commit, keep changes staged
git reset --hard HEAD~1                       # DANGEROUS – discards changes
git revert <sha>                               # safe undo on a shared branch (new commit)
git cherry-pick <sha>                          # bring one commit to another branch
git branch -d <branch>                        git remote -v
```

Branching strategies

- **GitFlow** — `main` , `develop` , `feature/*` , `release/*` , `hotfix/*` . Heavier; suits scheduled releases.
- **GitHub Flow / trunk-based** — short-lived feature branches off `main` , PR + CI + review, merge and deploy. Most common today for CI/CD.

How test code fits: the automation code lives in the **same repository and the same PR** as the feature wherever possible, so the tests are reviewed and merged with the change. If tests are in a separate repo, the branch name still references the same Jira ticket. PRs must have a green pipeline before merge.

CORE

- **Merge** creates a merge commit joining two histories. Non-destructive, preserves exactly what happened, but the graph gets noisy.
- **Rebase** replays your commits on top of the target branch, producing a linear history. Cleaner log, but it **rewrites commit hashes**.
- **The golden rule:** never rebase a branch that other people have already pulled — you rewrite shared history and force everyone to recover. Rebase your own local feature branch; merge into `main`.

Resolving a conflict

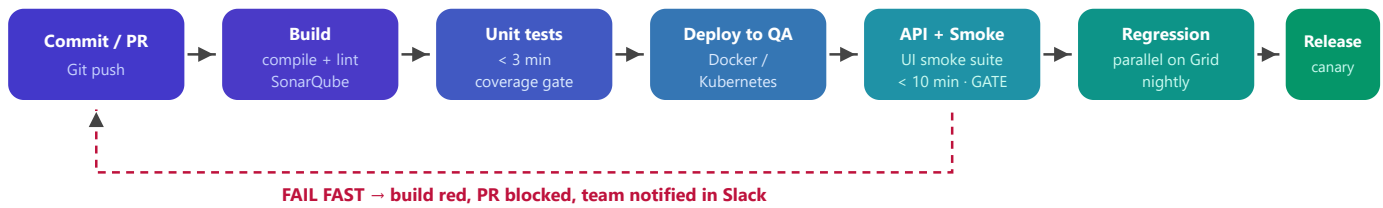
```
git pull --rebase origin main
# CONFLICT (content): Merge conflict in src/test/java/tests/CheckoutTest.java

# 1. Open the file – Git marks the regions:
<<<<<< HEAD
    your version
=====
    incoming version
>>>>>> origin/main

# 2. Edit to the correct final content, remove ALL markers
# 3. git add <file>
# 4. git rebase --continue      (or git commit, if merging)
# 5. Re-run the tests before pushing – a syntactically clean merge can still be
#     semantically wrong.
git rebase --abort             # bail out and start again
```

Add the practical point: "Conflicts in a test framework usually happen in shared files — BasePage, testng.xml, pom.xml. We reduced them by keeping page objects small and single-purpose, and by having each person own a module."

Where automated tests sit in a CI/CD pipeline



Every stage publishes artifacts: Extent/Allure HTML, screenshots, videos, logs.

Feedback target: developer knows within 10 minutes of pushing whether they broke something.

Figure 9 — The key interview point: automation is only valuable if it **gates** the pipeline. Tests that run but never block a release are decoration.

What is CI/CD and how is your automation integrated into it?

- **Continuous Integration** — every commit is merged to the mainline and automatically built and tested, so integration problems surface in minutes rather than at the end of a sprint.
- **Continuous Delivery** — every change that passes the pipeline is *deployable* at the push of a button.
- **Continuous Deployment** — it deploys automatically, with no manual gate.

How I plug automation in

1. **On every PR:** compile + unit tests + a <10-minute UI/API smoke suite. Red build blocks the merge.
2. **On merge to main / after deploy to QA:** the extended smoke + critical journeys.
3. **Nightly:** the full regression in parallel on a Docker Grid, plus cross-browser.
4. **Weekly:** performance and accessibility scans.
5. **Always:** publish the HTML report as a build artifact, notify Slack/Teams/email with a pass rate and a link, and auto-create a Jira bug for a new failure if the team wants it.

The strongest thing you can say here: **"The suite has to be trustworthy before it can gate anything. If the team learns that a red build might be flakiness, they stop looking at it — and then the automation is worthless. So I treat suite reliability as a first-class deliverable."**

CORE

ADVANCED

```

pipeline {
  agent any
  tools { maven 'Maven-3.9'; jdk 'JDK-17' }

  parameters {
    choice(name: 'SUITE', choices: ['testng-smoke','testng-regression'], description: '')
    choice(name: 'BROWSER', choices: ['chrome','firefox','edge'], description: '')
    choice(name: 'ENV', choices: ['qa','staging'], description: '')
    booleanParam(name: 'HEADLESS', defaultValue: true)
  }

  triggers { cron('H 1 * * *') } // nightly regression ~1 a.m.

  options {
    timeout(time: 90, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '30'))
    timestamps()
  }

  environment {
    TEST_USER_PWD = credentials('qa-test-user-password') // never hardcode secrets
  }

  stages {
    stage('Checkout') { steps { checkout scm } }

    stage('Start Grid') {
      steps { sh 'docker compose -f docker-compose-grid.yml up -d --scale chrome=5' }
    }

    stage('Test') {
      steps {
        sh """mvn clean test \
          -Dsuite=${params.SUITE} \
          -Dbrowser=${params.BROWSER} \
          -Denv=${params.ENV} \
          -Dheadless=${params.HEADLESS}"""
      }
    }
  }

  post {
    always {
      sh 'docker compose -f docker-compose-grid.yml down'
      junit 'target/surefire-reports/*.xml'
      archiveArtifacts artifacts: 'test-output/**, screenshots/**', allowEmptyArchive: true
      publishHTML([reportDir: 'test-output', reportFiles: 'ExtentReport.html',
        reportName: 'Automation Report', keepAll: true])
      allure includeProperties: false, results: [[path: 'target/allure-results']]
    }
  }
}

```

```
failure {
  slackSend channel: '#qa-alerts', color: 'danger',
    message: "FAILED ${env.JOB_NAME} #${env.BUILD_NUMBER} (${params.ENV}/${params.BROWSER})
${env.BUILD_URL}"
  emailxnt to: 'qa-team@example.com', subject: "Regression failed: ${env.JOB_NAME}",
    body: "See ${env.BUILD_URL}Automation_Report/", attachLog: true
}
}
```

Also know: **declarative vs scripted** pipelines, `agent { docker { image '...' } }`, parallel stages for sharding, and that the same concepts map to GitHub Actions / GitLab CI / Azure DevOps if that's their stack.

CORE

ADVANCED

```
# docker-compose-grid.yml
services:
  selenium-hub:
    image: selenium/hub:4.20
    ports: ["4442:4442", "4443:4443", "4444:4444"]
  chrome:
    image: selenium/node-chrome:4.20
    shm_size: 2gb # prevents Chrome crashes in containers
    depends_on: [selenium-hub]
    environment:
      SE_EVENT_BUS_HOST: selenium-hub
      SE_EVENT_BUS_PUBLISH_PORT: 4442
      SE_EVENT_BUS_SUBSCRIBE_PORT: 4443
      SE_NODE_MAX_SESSIONS: 5
      SE_NODE_OVERRIDE_MAX_SESSIONS: "true"
  firefox:
    image: selenium/node-firefox:4.20
    shm_size: 2gb
    depends_on: [selenium-hub]
    environment: { SE_EVENT_BUS_HOST: selenium-hub, ... }
```

```
docker compose up -d --scale chrome=5
# selenium/node-chrome:*-debug images expose VNC on 5900 so you can WATCH the tests run
# selenium/video records each session to mp4 – invaluable for debugging CI-only failures
```

Why it matters

- **Identical environments** — no "works on my machine"; the browser and driver versions are pinned in the image.
- **Elastic scale** — spin up 20 nodes for a nightly run, tear them down after.
- **Clean state** per run — no cookie/profile contamination.
- **Testcontainers** deserves a mention: it can start a real PostgreSQL/Kafka/WireMock container per test run from Java code, which makes integration tests reliable and self-contained.

ADVANCED

BDD (Behaviour-Driven Development) is a collaboration practice: business, dev and QA agree on behaviour *before* coding, expressed in plain-language **Gherkin** scenarios that then become executable tests.

```
# src/test/resources/features/checkout.feature
@checkout @regression
Feature: Checkout
  As a registered shopper
  I want to pay for my cart
  So that my order is placed

  Background:
    Given I am logged in as "shopper@test.com"

  @smoke
  Scenario: Successful card payment
    Given I have 2 items in my cart
    When I checkout and pay with a valid card
    Then I should see the order confirmation
    And the order status should be "CONFIRMED"
```

```
public class CheckoutSteps {
    private final World world; // shared state via DI
    public CheckoutSteps(World world) { this.world = world; }

    @Given("I have {int} items in my cart")
    public void addItem(int count) { world.cart().addRandomItems(count); }

    @When("I checkout and pay with a valid card")
    public void payWithCard() { world.checkout().payWith(Card.valid()); }

    @Then("the order status should be {string}")
    public void verifyStatus(String expected) {
        Assert.assertEquals(world.api().getOrder(world.orderId()).status(), expected);
    }
}
```

```
@CucumberOptions(
    features = "src/test/resources/features",
    glue     = {"steps", "hooks"},
    tags     = "@smoke and not @wip",
    plugin   = {"pretty", "html:target/cucumber.html",
               "json:target/cucumber.json",
               "io.qameta.allure.cucumber7jvm.AllureCucumber7Jvm"},
    monochrome = true)

public class TestRunner extends AbstractTestNGCucumberTests {
```

```
@Override @DataProvider(parallel = true)           // parallel scenarios
public Object[][] scenarios() { return super.scenarios(); }
}
```

```
public class Hooks {
    @Before(order = 0) public void setUp() { DriverFactory.init(Config.get("browser")); }
    @Before("@api")    public void apiOnly() { /* tag-specific setup */ }
    @After(order = 1) public void tearDown(Scenario s) {
        if (s.isFailed()) s.attach(((TakesScreenshot) DriverFactory.get())
            .getScreenshotAs(OutputType.BYTES), "image/png", s.getName());
        DriverFactory.quit();
    }
    @BeforeStep / @AfterStep                        // per-step hooks
}
```

CORE

```
# Scenario Outline = data-driven scenario
Scenario Outline: Login validation
  When I login with "<email>" and "<password>"
  Then I should see "<message>"

Examples:
  | email          | password    | message          |
  | valid@test.com | Correct@123 | Welcome back     |
  | valid@test.com | Wrong@123  | Invalid credentials |
  | notouser@test.com | Correct@123 | Invalid credentials |

# Data Table = structured data for ONE step
Scenario: Add multiple products
  When I add the following products to my cart:
    | sku      | qty | size |
    | TSH-001 | 2   | M    |
    | JNS-455 | 1   | 32   |
  Then the cart total should be "₹3,498.00"
```

```
@When("I add the following products to my cart:")
public void addProducts(DataTable table) {
    List<Map<String,String>> rows = table.asMaps(String.class, String.class);
    rows.forEach(r -> cartPage.add(r.get("sku"),
                                   Integer.parseInt(r.get("qty")), r.get("size")));
}
```

- **Background** — steps that run before *every* scenario in the feature. Keep it to 2–3 lines; a long Background makes scenarios unreadable.
- **Tags** — `@smoke` , `@regression` , `@wip` . Filter with `-Dcucumber.filter.tags="@smoke and not @wip"` . Tags can be on the Feature (inherited by all scenarios) or a single Scenario/Examples block.
- **Parameter types** — `{int}` , `{string}` , `{word}` , `{float}` , plus custom `@ParameterType` for domain objects.
- **Sharing state** between step classes — use dependency injection (PicoContainer/Spring), never `static` fields, which break parallel runs.

Best practices

- Write scenarios in **business language**, describing behaviour and outcome — not UI mechanics. ✓ "When I pay with an expired card" ✗ "When I click the button with id 'pay'".
- One scenario = one behaviour. If it has 15 steps and three *Thens*, split it.
- Declarative over imperative: "When I log in as an admin" rather than five steps of typing and clicking.
- Reuse step definitions; keep them thin — a step should call one page-object or API method, with all logic in the layers below.
- Keep the feature files in the same repo, reviewed by the PO.

When NOT to use BDD — an honest, senior answer

"BDD's value is the **conversation** with the business, not the Gherkin syntax. If the product owner never reads the feature files, you're paying a real cost — an extra glue layer, regex-matched steps, harder refactoring and debugging — for zero collaboration benefit. In that case plain TestNG with well-named methods is better. I've used Cucumber where business analysts genuinely reviewed the scenarios, and plain TestNG where they didn't."

This is a differentiator answer. Most candidates recite Cucumber syntax; very few can say *when the tool is the wrong choice*. That is what a lead wants to hear.

ADVANCED

```
// Given / When / Then – the BDD-style DSL
given()
    .baseUrl("https://qa-api.example.com")
    .header("Authorization", "Bearer " + token)
    .contentType(ContentType.JSON)
    .body(Map.of("sku", "TSH-001", "qty", 2))
.when()
    .post("/api/v1/cart/items")
.then()
    .statusCode(201)
    .time(lessThan(1500L)) // response-time assertion
    .body("itemId", notNullValue())
    .body("qty", equalTo(2))
    .body("price.currency", equalTo("INR"))
    .body("items.size()", greaterThan(0))
    .header("Content-Type", containsString("application/json"));

// Extract a value for the next call
String orderId = given().spec(reqSpec).body(payload)
    .post("/api/v1/orders").then().statusCode(201)
    .extract().path("orderId");

// Deserialise into a POJO and assert with normal Java
Order order = given().spec(reqSpec).get("/api/v1/orders/" + orderId)
    .then().statusCode(200).extract().as(Order.class);
Assert.assertEquals(order.getStatus(), "CONFIRMED");

// Reusable specifications – keeps tests clean
RequestSpecification reqSpec = new RequestSpecBuilder()
    .setBaseUrl(Config.get("api.url"))
    .addHeader("Authorization", "Bearer " + token)
    .setContentType(ContentType.JSON)
    .addFilter(new AllureRestAssured()) // Logs request/response in the report
    .build();

// JSON schema validation – catches contract breaks the field assertions miss
.then().body(matchesJsonSchemaInClasspath("schemas/order-schema.json"));
```

What to test at the API layer

Status codes (200/201/400/401/403/404/409/422/500), response body and data types, schema/contract, headers, authentication and **authorisation** (user A must not read user B's order — the most commonly missed test), pagination, rate limiting, idempotency, error message quality, and response time.

Q147

What metrics do you report, and how do you show automation's value to management?

METRIC	WHAT IT TELLS YOU
Automation coverage %	Automated test cases / total test cases — but qualify it: coverage of <i>critical journeys</i> matters more than a raw percentage
Pass rate & trend	Suite health over the last N runs
Flaky-test rate	Tests with inconsistent results — the trust metric
Execution time	Feedback speed; the number developers feel
Defect Removal Efficiency	How much escapes to production
Defects found by automation	The direct value evidence
Mean time to detect / to fix	How quickly a regression is caught
Manual effort saved	Hours per cycle × cycles — the CFO-friendly number
Requirement coverage	From the RTM — are all requirements verified?

Metrics to be careful with: "number of test cases written" and "number of bugs found" are vanity metrics that can be gamed and can create the wrong incentives (splitting one test into five, logging trivial cosmetic bugs). Say that — it shows maturity.

How to present: a one-page dashboard (Allure trends / Grafana) with pass rate, duration, flaky count and top failing areas, plus a short narrative: what broke, what we fixed, what we're doing next. Management wants risk and trend, not a list of test names.

ADVANCED

- **Does the test assert something meaningful?** A test with no assertion, or one that only checks the page title, is theatre.
- **No** `Thread.sleep()`, no implicit+explicit mixing, no swallowed exceptions, no empty catch.
- **Locators** — no absolute XPath, no index-based selectors, no hardcoded generated ids.
- **Layering** — no Selenium API in the test class, no assertions in page objects, no test data hardcoded in pages.
- **Independence** — could this test run alone, in any order, in parallel? Does it clean up after itself?
- **Naming** — the method name should state the behaviour and the expectation: `orderFailsWhenCardIsExpired()`, not `test3()`.
- **Failure quality** — will the failure message tell me what went wrong without a debugger? Are assertion messages present?
- **Duplication** — is this the fourth copy of the same login helper?
- **Secrets** — no credentials, tokens or PII committed.
- **Determinism** — no dependence on today's date, on record ordering, or on data another test created.

Say the underlying principle: "**Test code is production code.**" It gets reviewed, refactored, versioned and held to the same standards — because the whole team depends on it to make release decisions.

ADVANCED

Work through the differences systematically — say them in this order:

1. **Get evidence first.** Screenshot at failure, page source, browser console log, the video (Selenium/video container), and the exact stack trace. Never guess.
2. **Timing** — CI agents are slower and more loaded. Fixed waits that were "just enough" locally now fail. Fix: condition-based waits, and CI-specific timeout values.
3. **Headless vs headed** — different viewport (headless has no default window size!), no hover states, different font rendering. Fix: set an explicit window size; reproduce locally with `-Dheadless=true`.
4. **Screen resolution / responsive layout** — elements may be behind a hamburger menu at the CI viewport.
5. **Environment & data** — different DB state, feature flags off, the record the test expects doesn't exist, another parallel job mutated shared data.
6. **Parallelism** — it passes alone but fails with 5 threads → shared state, static driver, or data collision.
7. **Version drift** — different browser/driver/JDK version on the agent.
8. **Network** — CI may be behind a proxy or firewall; third-party scripts time out.
9. **Reproduce in the same conditions** — run the Docker image locally, or use the Grid's VNC/debug image to watch it live.

Strong closing line: *"I treat 'passes locally, fails in CI' as a defect in the test, not an excuse. CI is the source of truth, because it's the environment the release decision is based on."*

CORE

ADVANCED

You join a project with zero automation. What is your 90-day plan?

A classic senior/lead question. Answer in phases, and lead with **learning and value**, not with tools.

Days 1–15 — understand before building

- Learn the product and the business-critical flows (which ones lose money if broken?).
- Review existing manual test cases, the defect history (where do bugs cluster?), and the release process.
- Assess testability with developers: are there stable attributes, an API, test environments, seedable data, feature flags?
- Understand the team's language/skills — do not impose Java on a JavaScript team.

Days 15–30 — prove value fast

- Pick the **top 5–10 critical journeys** and automate them as a smoke suite.
- Set up the framework skeleton properly from day one: POM, driver factory with ThreadLocal, config layering, reporting, Git repo, README.
- Get it running in CI on every deploy, even if it starts non-blocking. A visible green/red signal changes the conversation.

Days 30–60 — expand and stabilise

- Grow regression coverage by risk priority, not alphabetically.
- Add API-level tests for logic that doesn't need the UI — faster and more stable.
- Introduce parallel execution and get the suite under the feedback-time target.
- Drive the suite's pass rate to a level where a red build is believed. Make it a merge gate.

Days 60–90 — embed it in the team

- Add "automated test written" to the Definition of Done.
- Train developers and manual testers to write and run tests; do pairing sessions.
- Establish the flaky-test policy, code review standards for tests, and the metrics dashboard.
- Present measured results: time saved, defects caught pre-release, feedback time.

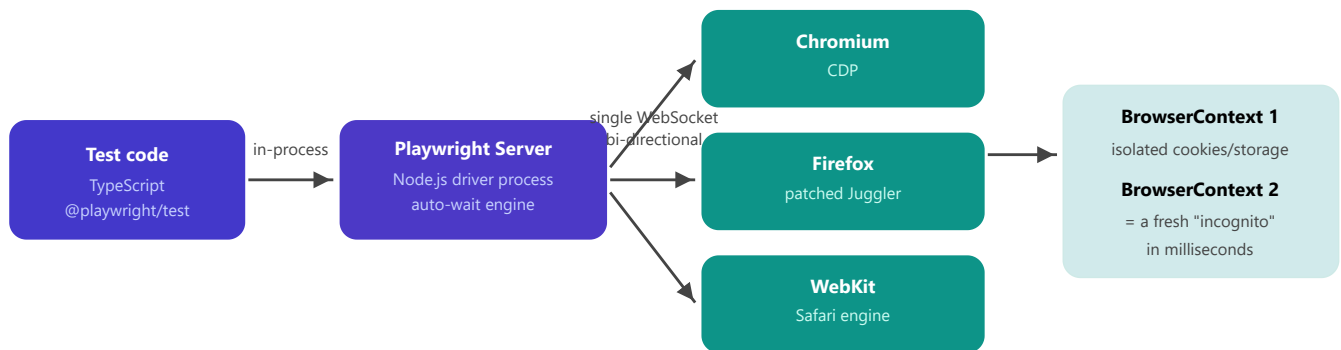
The line that lands: *"I'd rather have 15 rock-solid tests the team trusts and that gate every release, than 400 tests everyone ignores."*

SECTION 5

Playwright with TypeScript / JavaScript

Q151 – Q190 · The fastest-growing skill in QA job descriptions. Interviewers assume you know Selenium; what they're probing is whether you understand **why Playwright is architecturally different** — not just its syntax.

Playwright Architecture — one persistent connection, not per-command HTTP



Why this matters

One persistent duplex connection instead of an HTTP round-trip per command → far faster, and the browser can *push events back*, which is what makes auto-waiting and network interception native.

Figure 10 — The architectural difference to state in an interview: Selenium is request/response over HTTP; Playwright is event-driven over a WebSocket.

Playwright is an open-source end-to-end testing framework from Microsoft that automates Chromium, Firefox and WebKit with a single API. Unlike Selenium it is a **complete test framework** — runner, assertions, fixtures, parallelism, reporting and tracing all come in the box.

Architectural differences

- **Transport** — Selenium sends one HTTP request per command via the W3C protocol. Playwright keeps a **single WebSocket connection** to a Node driver process, which speaks each browser's native debugging protocol. Fewer round trips, and the browser can push events back.
- **Bi-directional** — because events flow both ways, Playwright natively knows when the DOM changed, when a request fired, when navigation completed. That is the foundation of **auto-waiting**.
- **BrowserContext** — a lightweight isolated browser session (own cookies, storage, cache) that starts in milliseconds. Selenium's equivalent isolation requires launching a whole new browser.
- **Bundled browsers** — Playwright downloads and pins patched browser builds, so version mismatch problems largely disappear.

Playwright vs Selenium — give a balanced comparison. Which would you choose?

	SELENIUM	PLAYWRIGHT
Scope	Browser automation library only	Full framework: runner, assertions, fixtures, reporter, trace
Waiting	You write explicit waits	Auto-waiting + auto-retrying assertions built in
Languages	Java, C#, Python, JS, Ruby, Kotlin	TypeScript/JS, Python, Java, .NET
Browsers	Chrome, Firefox, Edge, Safari, IE (legacy)	Chromium, Firefox, WebKit (+ branded Chrome/Edge channels)
Speed	Slower (HTTP per command, browser per test)	Faster (WebSocket, contexts, parallel by default)
Isolation	New browser per test	New context per test — near-instant
Network mocking	Via CDP (Chromium only) or a proxy	First-class <code>page.route()</code> on all browsers
Debugging	Screenshots + logs	Trace Viewer (DOM snapshots, network, console, timeline), video, UI mode, codegen
Shadow DOM / iframes	Manual switching / <code>getShadowRoot</code>	Pierces open shadow DOM automatically; <code>frameLocator</code>
Mobile	Real devices via Appium	Emulation only — no real device support
Ecosystem & jobs	Huge, mature, 15+ years, W3C standard	Newer, growing fast
Grid / cloud	Selenium Grid, all cloud vendors	Own sharding; cloud support improving

How to answer "which would you choose?"

Never say "Playwright is better" flatly. Say: *"For a new greenfield web project I'd pick Playwright — the auto-waiting and Trace Viewer alone cut flakiness and debugging time dramatically, and it's faster out of the box. I'd stay with Selenium where the team's skills and existing suite are*

Java/Selenium and the migration cost isn't justified, where real mobile devices or legacy browsers matter, or where the organisation's grid and cloud tooling is built around WebDriver. The right tool depends on the team, not the hype."

CORE

ADVANCED

How do you set up a Playwright TypeScript project? Walk through playwright.config.ts.

```

npm init playwright@latest      # scaffolds config, tests, GitHub Actions workflow
npx playwright install          # download the browser binaries
npx playwright test              # run everything, headless
npx playwright test --headed --project=chromium
npx playwright test login.spec.ts:24      # a single test by line number
npx playwright test --grep @smoke
npx playwright test --ui          # interactive UI mode (best DX)
npx playwright codegen https://example.com # record a script
npx playwright show-report

```

```

// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,                // run tests inside a file in parallel too
  forbidOnly: !!process.env.CI,      // fail CI if someone left a test.only
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 4 : undefined, // default = half the CPU cores
  timeout: 30_000,                    // per test
  expect: { timeout: 5_000 },         // per web-first assertion
  reporter: [['html', { open: 'never' }], ['list'], ['junit', { outputFile: 'results.xml' }]],

  use: {
    baseURL: process.env.BASE_URL ?? 'https://qa.example.com',
    trace: 'on-first-retry',          // the single most valuable setting
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    actionTimeout: 10_000,
    navigationTimeout: 30_000,
    testIdAttribute: 'data-testid',
    locale: 'en-IN',
    timezoneId: 'Asia/Kolkata',
  },

  projects: [
    { name: 'setup', testMatch: /global\.setup\.ts/ },
    { name: 'chromium', use: { ...devices['Desktop Chrome'], storageState: 'auth/user.json' },
      dependencies: ['setup'] },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] }, dependencies: ['setup'] },
    { name: 'webkit', use: { ...devices['Desktop Safari'] }, dependencies: ['setup'] },
    { name: 'mobile', use: { ...devices['Pixel 7'] } },
  ],

  webServer: {                        // start the app before tests (local dev)
    command: 'npm run start',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },

```

```
},  
});
```

trace: 'on-first-retry' is the answer to give when asked "how do you debug CI failures in Playwright?" — you get a full time-travel recording of the failed attempt at almost no cost for passing runs.

CORE

```
const browser = await chromium.launch({ headless: true }); // 1 OS process – expensive
const context = await browser.newContext({                // isolated session – cheap
  viewport: { width: 1440, height: 900 },
  storageState: 'auth/admin.json',
  locale: 'en-IN',
});
const page = await context.newPage();                    // a tab
await page.goto('/dashboard');
await context.close();
await browser.close();
```

- **Browser** — the launched browser process. Heavy; Playwright reuses one per worker.
- **BrowserContext** — an isolated incognito-like session with its own cookies, localStorage, cache, permissions and geolocation. Creating one takes **milliseconds**.
- **Page** — a single tab within a context. A context can hold several pages that share storage.

Why it matters (the interview point)

Playwright's test runner gives **every test its own context**. That means perfect test isolation — no cookie or storage leakage between tests — *without* the cost of restarting a browser. This is the main reason a Playwright suite runs several times faster than the equivalent Selenium suite while being *more* isolated.

```
// Multi-user scenario in ONE test – trivial in Playwright, painful in Selenium
test('admin approves what the user submitted', async ({ browser }) => {
  const userCtx = await browser.newContext({ storageState: 'auth/user.json' });
  const adminCtx = await browser.newContext({ storageState: 'auth/admin.json' });
  const userPage = await userCtx.newPage();
  const adminPage = await adminCtx.newPage();
  await userPage.goto('/requests/new');
  // ... submit as user, then approve as admin, in the same test
});
```

CORE

ADVANCED

What is auto-waiting? What actionability checks does Playwright perform?

Before performing an action, Playwright automatically waits for the element to pass a series of **actionability checks**, retrying until they pass or the timeout expires. You almost never write a wait.

CHECK	MEANING
Attached	Element is in the DOM
Visible	Non-empty bounding box and not <code>visibility:hidden</code>
Stable	Bounding box hasn't changed for two consecutive animation frames (i.e. animation finished)
Receives events	It is the actual hit target at the click point — not covered by an overlay
Enabled	Not <code>disabled</code>
Editable	For <code>fill()</code> : enabled and not readonly

Which checks apply depends on the action: `click()` does all of visible/stable/receives-events/enabled; `fill()` adds editable; `textContent()` requires none.

Contrast to draw in the interview: "In Selenium, `elementToBeClickable` only checks visible + enabled — it will happily return an element that is covered by a modal overlay, and then `click()` throws `ElementClickInterceptedException`. Playwright's 'receives events' and 'stable' checks eliminate that entire class of flakiness."

Auto-waiting is not magic: it does **not** wait for your application's business state (e.g. "the order has been persisted"). For that you still assert on an observable condition — a visible confirmation, a network response, or an API check.

```
const loginBtn = page.getByRole('button', { name: 'Sign in' }); // a Locator
await loginBtn.click();
```

A **Locator** is a *lazy description of how to find an element* — not the element itself. It is resolved fresh at the moment of every action. That is why Playwright has no equivalent of

StaleElementReferenceException: the reference cannot go stale because there is no reference.

	LOCATOR (USE THIS)	ELEMENTHANDLE (LEGACY)
Resolution	Lazy, re-queried each action	Bound to one DOM node at creation
Stale risk	None	Yes, if the DOM re-renders
Auto-waiting	Yes	No
API	<code>page.getByRole()</code> , <code>page.locator()</code>	<code>page.\$()</code> , <code>page.\$\$()</code> — discouraged

Recommended locators, in priority order (user-facing first)

```
page.getByRole('button', { name: 'Add to cart' }) // ARIA role + accessible name ← best
page.getByLabel('Email address')                // form field by its <label>
page.getByPlaceholder('Search products')
page.getByText('Order confirmed')                // visible text
page.getByAltText('Company logo')
page.getByTitle('Close')
page.getByTestId('checkout-submit')              // data-testid – stable, explicit
page.locator('css=.product-card')                // CSS / XPath – last resort
page.locator('//table//tr[3]/td[2]')
```

Why getByRole is recommended: it queries the accessibility tree, so your test uses the app the way a screen reader (and therefore a user) does. If `getByRole('button', {name: 'Sign in'})` fails, it often means a genuine accessibility defect. That reasoning is exactly what interviewers want to hear.

Strict mode is on by default: if a locator resolves to more than one element, the action throws a **strict mode violation** error instead of silently using the first match. This catches ambiguous locators at the moment you write them, rather than in production when a second match appears.

```
// X throws if there are 12 "Add to cart" buttons
await page.getByRole('button', { name: 'Add to cart' }).click();

// ✓ narrow it properly
await page.getByRole('listitem').filter({ hasText: 'Wireless Mouse' })
    .getByRole('button', { name: 'Add to cart' }).click();

// chaining – scope a search inside another locator
const row = page.getByRole('row', { name: /ORD-991/ });
await row.getByRole('button', { name: 'Cancel' }).click();

// filtering
page.getByRole('listitem').filter({ hasText: 'In stock' });
page.getByRole('listitem').filter({ hasNotText: 'Sold out' });
page.getByRole('listitem').filter({ has: page.getByRole('img') });
page.locator('.card').filter({ hasNot: page.locator('.badge-new') });

// positional (use sparingly – brittle)
page.getByRole('listitem').first();
page.getByRole('listitem').last();
page.getByRole('listitem').nth(2);

// combining with or / and (newer versions)
page.getByRole('button', { name: 'Save' }).or(page.getByRole('button', { name: 'Update' }));

// count and iterate
const items = page.getByTestId('product');
await expect(items).toHaveCount(12);
for (const item of await items.all()) {
    console.log(await item.textContent());
}
```


Assertions on a `Locator` or `Page` **automatically retry** until they pass or the `expect` timeout expires. This removes the classic "assert too early" race that plagues Selenium suites.

```
// Auto-retrying (web-first) – these WAIT
await expect(page.getByText('Order confirmed')).toBeVisible();
await expect(page.getByTestId('total')).toHaveText('₹1,499.00');
await expect(page.getByTestId('total')).toContainText('1,499');
await expect(page.getByRole('button', { name: 'Pay' })).toBeEnabled();
await expect(page.getByLabel('Terms')).toBeChecked();
await expect(page.getByTestId('product')).toHaveCount(12);
await expect(page.getByRole('textbox')).toHaveValue('ravi@test.com');
await expect(page.locator('.error')).toHaveAttribute('role', 'alert');
await expect(page.locator('.badge')).toHaveClass(/active/);
await expect(page).toHaveURL(/\/orders\/\d+/);
await expect(page).toHaveTitle('Checkout – Shop');
await expect(page.getByRole('list')).toHaveScreenshot('cart.png');
await expect(locator).not.toBeVisible();           // negation retries too

// Non-retrying (generic) – these compare a value ONCE
expect(orderId).toBe('ORD-991');
expect(items).toHaveLength(3);
expect(response.status()).toBe(201);
expect(user).toEqual({ id: 1, name: 'Ravi' });
```

The #1 mistake beginners make: forgetting `await` on an assertion.

`expect(locator).toBeVisible()` without `await` returns a floating promise — the test passes regardless. Enable the ESLint rule `@typescript-eslint/no-floating-promises` to catch it. Mentioning this shows real hands-on experience.

```
// Custom timeout for one slow assertion
await expect(page.getByText('Report ready')).toBeVisible({ timeout: 60_000 });

// Poll a non-Locator value with retry (e.g. an API state)
await expect.poll(async () => (await api.getOrder(id)).status,
  { timeout: 30_000, intervals: [1000, 2000, 5000] })
  .toBe('CONFIRMED');

// Retry an arbitrary block of assertions
await expect(async () => {
  const res = await request.get(`/api/orders/${id}`);
  expect(res.status()).toBe(200);
}).toPass({ timeout: 20_000 });
```

Q159

How do soft assertions and custom matchers work in Playwright?

```
// Soft assertions – the test continues and reports ALL failures at the end
await expect(page.getByTestId('name')).toHaveText('Ravi Kumar');
await expect(page.getByTestId('gst')).toHaveText('18%');
await expect(page.getByTestId('total')).toHaveText('₹1,499.00');
// The test is marked failed at the end automatically – no assertAll() needed.

// Stop early if any soft assertion has already failed
expect(test.info().errors).toHaveLength(0);

// Custom matcher – makes domain assertions readable and reusable
import { expect as baseExpect } from '@playwright/test';

export const expect = baseExpect.extend({
  async toBeValidOrderId(received: string) {
    const pass = /^ORD-\d{4}-\d{4}$/.test(received);
    return {
      pass,
      message: () => `expected "${received}" ${pass ? 'not ' : ''}to be a valid order id`,
    };
  },
});

// usage
expect(orderId).toBeValidOrderId();
```

Unlike TestNG's SoftAssert, Playwright's soft assertions cannot be silently forgotten — there is no `assertAll()` to miss. That is a nice comparison point to raise.

ADVANCED

```
import { test, expect } from '@playwright/test';

test.describe('Checkout', () => {
  test.describe.configure({ mode: 'serial' }); // or 'parallel' (default)

  test.beforeAll(async () => { /* once per worker for this describe */ });
  test.beforeEach(async ({ page }) => { await page.goto('/cart'); });
  test.afterEach(async ({ page }, testInfo) => {
    if (testInfo.status !== testInfo.expectedStatus) console.log('failed:', testInfo.title);
  });
  test.afterAll(async () => { /* cleanup */ });

  test('pays with a valid card @smoke', async ({ page }) => {
    await test.step('open checkout', async () => {
      await page.getByRole('button', { name: 'Checkout' }).click();
    });
    await test.step('enter card details', async () => {
      await page.getByLabel('Card number').fill('4111111111111111');
      await page.getByLabel('CVV').fill('123');
    });
    await test.step('verify confirmation', async () => {
      await expect(page.getByText('Order confirmed')).toBeVisible();
    });
  });

  test.skip(({ browserName }) => browserName === 'webkit', 'Not supported on Safari yet');
  test.fixme('known broken', async ({ page }) => { });
  test.fail('expected to fail until BUG-412 is fixed', async ({ page }) => { });
  test.slow(); // triples the timeout
  test.setTimeout(120_000);
  test.only('debug just this one', async ({ page }) => { }); // forbidden in CI
});

test.use({ locale: 'fr-FR', viewport: { width: 375, height: 812 } }); // override config
```

`test.step` is undervalued: steps appear as a collapsible tree in the HTML report and the Trace Viewer, so a failure immediately shows *which business step* broke. Use it in page-object methods too, and your report becomes self-documenting.

What are fixtures in Playwright? Write a custom fixture.

Fixtures are Playwright's dependency-injection mechanism. Instead of `beforeEach` setting up shared variables, you declare what a test **needs** in its arguments and Playwright constructs it, on demand, and tears it down afterwards.

Built-in fixtures: `page`, `context`, `browser`, `browserName`, `request` (API), `baseURL`.

```
// fixtures.ts – page objects + an authenticated API client as fixtures
import { test as base } from '@playwright/test';
import { LoginPage } from './pages/login.page';
import { CheckoutPage } from './pages/checkout.page';

type Fixtures = {
  loginPage: LoginPage;
  checkoutPage: CheckoutPage;
  testOrder: { id: string };
};

export const test = base.extend<Fixtures>({
  loginPage: async ({ page }, use) => { await use(new LoginPage(page)); },
  checkoutPage: async ({ page }, use) => { await use(new CheckoutPage(page)); },

  // A fixture with setup AND teardown – creates data, then cleans it up
  testOrder: async ({ request }, use) => {
    const res = await request.post('/api/orders', { data: { sku: 'TSH-001', qty: 1 } });
    const order = await res.json();
    await use(order); // <-- the test runs here
    await request.delete(`/api/orders/${order.id}`); // always runs, even on failure
  },
});

export { expect } from '@playwright/test';
```

```
// the test – declarative and clean
import { test, expect } from './fixtures';

test('can cancel an existing order', async ({ page, testOrder, checkoutPage }) => {
  await page.goto(`/orders/${testOrder.id}`);
  await checkoutPage.cancelOrder();
  await expect(page.getByText('Order cancelled')).toBeVisible();
});
```

Fixture scopes: `{ scope: 'test' }` (default) or `{ scope: 'worker' }` for expensive setup shared across tests in a worker. Use `auto: true` to run a fixture even if no test requests it (e.g. per-test logging).

The interview point: **fixtures are lazy**. A fixture is only created if a test actually asks for it, so unlike `beforeEach` you never pay for setup a test doesn't need.

ADVANCED

CORE

Q162

How does parallel execution work? Explain workers, sharding and `fullyParallel`.

- **Workers** — independent Node processes, each with its own browser instance. Default is half the CPU cores; set with `workers: 4` or `--workers=4`.
- By default, **files** run in parallel and tests *within* a file run sequentially. `fullyParallel: true` makes individual tests parallel too.
- `test.describe.configure({ mode: 'serial' })` — run in order, and skip the rest if one fails (for genuinely dependent flows). `mode: 'parallel'` forces parallel within a file.
- **Sharding** — split the suite across CI machines:

```
npx playwright test --shard=1/4      # machine 1 of 4
npx playwright test --shard=2/4
npx playwright merge-reports ./all-blob-reports  # combine into one HTML report
```

- **Projects** run in parallel with each other too, so cross-browser costs little extra wall-clock time.

```
// Worker-scoped index for isolated test data per worker
test('uses a dedicated account', async ({ page }, testInfo) => {
  const account = ACCOUNTS[testInfo.workerIndex % ACCOUNTS.length];
  // ...
});
```

Same prerequisite as TestNG: parallel only works if tests are independent. Playwright helps by isolating browser state automatically, but it cannot isolate **your backend data** — two workers cancelling the same order will still collide. Per-test data creation via API is the fix.

CORE

How do you debug a Playwright test? Explain the Trace Viewer.

```
npx playwright test --ui           # UI mode: watch, time-travel, pick locators live
npx playwright test --debug        # step through with the Playwright Inspector
PWDEBUG=1 npx playwright test      # same, via env var
npx playwright test --headed --slowmo=500
npx playwright codegen example.com # record actions into code; also a locator picker
npx playwright show-trace trace.zip
npx playwright show-report
```

```
await page.pause();                // opens the Inspector at this line
console.log(await page.content());
await page.screenshot({ path: 'debug.png', fullPage: true });
```

Trace Viewer — the killer feature

With `trace: 'on-first-retry'`, a failed test produces a `trace.zip` containing:

- A **timeline filmstrip** of screenshots for every action.
- A **full DOM snapshot before and after each action** — you can open DevTools *on the past state* and inspect elements as they were at the moment of failure.
- All network requests and responses, console logs, and the source line for each action.
- Action duration, so you can see what was slow.

When asked "how do you debug a failure that only happens in CI?", this is the answer: *"The trace from the CI run is downloaded as an artifact and opened locally — I get the exact DOM, network and console state at failure, without reproducing anything. In Selenium the same investigation means adding logging and re-running."* That single point often wins the Playwright portion of an interview.

CORE

ADVANCED

`storageState` saves cookies + localStorage to a JSON file, which any context can then load. Log in once, reuse everywhere.

```
// tests/global.setup.ts – a "setup project" that other projects depend on
import { test as setup, expect } from '@playwright/test';

const USER_FILE = 'playwright/.auth/user.json';
const ADMIN_FILE = 'playwright/.auth/admin.json';

setup('authenticate as user', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill(process.env.USER_EMAIL!);
  await page.getByLabel('Password').fill(process.env.USER_PASSWORD!);
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
  await page.context().storageState({ path: USER_FILE });
});
```

```
// playwright.config.ts
projects: [
  { name: 'setup', testMatch: /global\.setup\.ts/ },
  { name: 'chromium', dependencies: ['setup'],
    use: { ...devices['Desktop Chrome'], storageState: 'playwright/.auth/user.json' } },
]
```

```
// Faster still – get the token from the API, skip the UI entirely
setup('authenticate via API', async ({ request }) => {
  const res = await request.post('/api/login',
    { data: { email: process.env.USER_EMAIL, password: process.env.USER_PASSWORD } });
  await request.storageState({ path: USER_FILE });
});

// Per-test override for a different role
test.use({ storageState: 'playwright/.auth/admin.json' });

// Or a fresh, unauthenticated context for the login tests themselves
test.use({ storageState: { cookies: [], origins: [] } });
```

Add `playwright/.auth` to `.gitignore` — the file contains live session tokens. And keep **one** real UI login test so the login page itself is still covered.

```

// Stub an API response – test the UI against any backend state you like
await page.route('**/api/orders', async route => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ orders: [], total: 0 }),      // test the EMPTY state
  });
});

// Serve from a fixture file
await page.route('**/api/products', route => route.fulfill({ path: 'mocks/products.json' }));

// Simulate a server error – how does the UI behave on 500?
await page.route('**/api/checkout', route => route.fulfill({ status: 500,
  body: JSON.stringify({ message: 'Payment service unavailable' }) }));

// Simulate a network failure / timeout
await page.route('**/api/slow', route => route.abort('failed'));
await page.route('**/api/slow', async route => {
  await new Promise(r => setTimeout(r, 5000));
  await route.continue();
});

// Block noisy third parties – makes tests faster AND less flaky
await page.route(/(analytics|doubleclick|hotjar|\.gif$)/, r => r.abort());

// Modify a real response (spy + patch)
await page.route('**/api/user', async route => {
  const response = await route.fetch();
  const json = await response.json();
  json.role = 'ADMIN';                                // escalate for a UI-only test
  await route.fulfill({ response, json });
});

// Modify the outgoing request
await page.route('**/api/**', route =>
  route.continue({ headers: { ...route.request().headers(), 'x-test-run': runId } }));

await page.unroute('**/api/orders');                  // remove the handler

```

Waiting for and asserting on network traffic

```

const responsePromise = page.waitForResponse(r =>
  r.url().includes('/api/orders') && r.status() === 201);
await page.getByRole('button', { name: 'Place order' }).click();
const response = await responsePromise;                // set up BEFORE the action
const body = await response.json();
expect(body.status).toBe('CONFIRMED');

```


Ordering trap: you must create the `waitForResponse` promise *before* the click that triggers it. If you `await` the click first, the response may already have arrived and you'll hang until timeout. Interviewers ask this specifically.

Balance to mention: mocking makes tests fast and deterministic but stops testing the real contract. My rule: mock for edge cases and error states that are hard to produce for real; keep a smaller set of true end-to-end tests against the real backend.

ADVANCED

CORE

```
// Pure API test – no browser launched at all
import { test, expect } from '@playwright/test';

test('creates an order via API', async ({ request }) => {
  const res = await request.post('/api/v1/orders', {
    headers: { Authorization: `Bearer ${token}` },
    data: { sku: 'TSH-001', qty: 2 },
  });
  expect(res.status()).toBe(201);
  expect(res.ok()).toBeTruthy();
  const body = await res.json();
  expect(body).toMatchObject({ status: 'CONFIRMED', qty: 2 });
  expect(res.headers()['content-type']).toContain('application/json');
});

// A reusable, pre-authenticated API context
import { request as pwRequest } from '@playwright/test';
const api = await pwRequest.newContext({
  baseURL: process.env.API_URL,
  extraHTTPHeaders: { Authorization: `Bearer ${token}` },
});

// The high-value pattern: API for setup, UI for the thing under test
test('order appears in the UI order history', async ({ page, request }) => {
  const created = await (await request.post('/api/orders',
    { data: { sku: 'TSH-001', qty: 1 } })).json(); // fast setup
  await page.goto('/orders');
  await expect(page.getByRole('row', { name: created.id })).toBeVisible();
});
```

Emphasise the architectural advantage: "In Playwright, API and UI testing share the same runner, fixtures, reporter and trace. In the Java world I'd wire RestAssured into TestNG separately. That single-toolchain point is a real productivity argument."

```
// frameLocator – no switching, no switching back
const payment = page.frameLocator('#payment-iframe');
await payment.getByLabel('Card number').fill('4111111111111111');
await payment.getByRole('button', { name: 'Pay' }).click();

// Nested frames – just chain
await page.frameLocator('#outer').frameLocator('#inner')
    .getByRole('button', { name: 'Confirm' }).click();

// Frame object API (when you need frame-level operations)
const frame = page.frame({ name: 'payment' });
const frame2 = page.frame({ url: /stripe\.com/ });
await frame?.goto('...');
console.log(page.frames().length);
```

The contrast to draw: in Selenium you must `switchTo().frame()` and remember to `switchTo().defaultContent()` — forgetting the second is a classic source of "NoSuchElement" bugs. Playwright's `frameLocator` is scoped to the expression, so there is no global state to forget to reset. Same for shadow DOM: Playwright pierces open shadow roots automatically with normal locators.

```
// A link that opens a new tab – capture it with waitForEvent BEFORE clicking
const popupPromise = page.waitForEvent('popup');
await page.getByRole('link', { name: 'Terms & Conditions' }).click();
const popup = await popupPromise;
await popup.waitForLoadState();
await expect(popup).toHaveTitle(/Terms/);
await popup.close();

// Opening a page yourself in the same context (shares cookies/session)
const second = await page.context().newPage();
await second.goto('/help');

// All open pages
const pages = page.context().pages();

// Wait for a new page opened by any means
const newPagePromise = page.context().waitForEvent('page');
await page.getByRole('button', { name: 'Open report' }).click();
const reportPage = await newPagePromise;

// Bring a tab to the front
await popup.bringToFront();
```

Compare with Selenium: no window handles, no `Set<String>` iteration, no "switch back to parent". The popup is just another Page object you hold a reference to, and you can interact with both simultaneously.

```
// ---- Dialogs (alert / confirm / prompt) ----
// Playwright AUTO-DISMISSSES dialogs by default. Register a handler BEFORE the trigger.
page.on('dialog', async dialog => {
  expect(dialog.type()).toBe('confirm');
  expect(dialog.message()).toBe('Delete this order?');
  await dialog.accept(); // or dialog.dismiss() / dialog.accept('text')
});
await page.getByRole('button', { name: 'Delete' }).click();

// Auto-dismiss recurring overlays (Playwright 1.42+)
await page.addLocatorHandler(page.getByRole('dialog'), async () => {
  await page.getByRole('button', { name: 'Close' }).click(); // auto-dismiss cookie banners
});

// ---- File upload ----
await page.getByLabel('Invoice').setInputFiles('fixtures/invoice.pdf');
await page.getByLabel('Photos').setInputFiles(['a.png', 'b.png']);
await page.getByLabel('Invoice').setInputFiles([]); // clear the selection

// In-memory file – no fixture file needed
await page.getByLabel('Upload').setInputFiles({
  name: 'data.csv', mimeType: 'text/csv', buffer: Buffer.from('id,name\n1,Ravi'),
});

// Non-input upload widget (drag-drop zone) – use the filechooser event
const chooserPromise = page.waitForEvent('filechooser');
await page.getByText('Drop files here').click();
const chooser = await chooserPromise;
await chooser.setFiles('fixtures/invoice.pdf');

// ---- Download ----
const downloadPromise = page.waitForEvent('download');
await page.getByRole('button', { name: 'Export CSV' }).click();
const download = await downloadPromise;
expect(download.suggestedFilename()).toBe('orders.csv');
const path = await download.path(); // temp path; already fully written
await download.saveAs('./downloads/orders.csv');
const content = fs.readFileSync(path!, 'utf-8');
expect(content).toContain('ORD-991');
```

Point out the contrast: downloads in Selenium mean configuring browser prefs and polling a folder for .crdownload files. In Playwright the download event resolves only when the file is complete, so there is no polling and no race.

Show the common interaction APIs — typing, clicking, hovering, drag & drop, keyboard.

```
// Typing
await page.getByLabel('Email').fill('ravi@test.com'); // clears + sets instantly (preferred)
await page.getByLabel('Search').pressSequentially('lap', { delay: 100 }); // real keystrokes
await page.getByLabel('Email').clear();

// Clicking
await page.getByRole('button', { name: 'Save' }).click();
await locator.click({ button: 'right' });
await locator.click({ clickCount: 2 }); // or .dblclick()
await locator.click({ modifiers: ['Control'] });
await locator.click({ position: { x: 10, y: 5 } });
await locator.click({ force: true }); // SKIPS actionability – see warning
await locator.click({ trial: true }); // check actionability, don't click

// Hover, focus, scroll
await page.getByRole('menuitem', { name: 'Products' }).hover();
await locator.focus();
await locator.scrollIntoViewIfNeeded();

// Keyboard
await page.keyboard.press('Enter');
await page.keyboard.press('Control+A');
await page.keyboard.type('hello');
await locator.press('Escape');

// Mouse (Low Level)
await page.mouse.move(100, 200);
await page.mouse.down(); await page.mouse.move(300, 400, { steps: 10 }); await page.mouse.up();

// Drag and drop
await page.getByTestId('card-1').dragTo(page.getByTestId('column-done'));

// Reading state
await locator.textContent(); await locator.innerText(); await locator.inputValue();
await locator.getAttribute('href'); await locator.isVisible(); await locator.isEnabled();
await locator.count(); await locator.boundingBox();
```

force: true is the Playwright equivalent of a JS click — it skips every actionability check. If a test needs it, something is usually wrong with the app or the locator. Treat it as a documented exception, never a habit. Interviewers ask "when would you use force?" precisely to see whether you understand this.

fill() vs pressSequentially(): fill() sets the value and fires one input event — fast and right for most forms. Use pressSequentially() when the app reacts to *each* keystroke

(autocomplete, input masks, character counters).

CORE

Q171

How do you handle dropdowns, checkboxes and radio buttons?

```
// ---- Native <select> ----
await page.getByLabel('Country').selectOption('IN');           // by value
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByLabel('Country').selectOption({ index: 2 });
await page.getByLabel('Skills').selectOption(['java', 'ts']);   // multi-select
await expect(page.getByLabel('Country')).toHaveValue('IN');

// ---- Custom dropdown (div/ul based) ----
await page.getByTestId('country-trigger').click();
await page.getByRole('option', { name: 'India' }).click();
await expect(page.getByTestId('country-trigger')).toHaveText('India');

// ---- Checkbox / radio - declarative, idempotent ----
await page.getByLabel('I accept the terms').check();           // no-op if already checked
await page.getByLabel('Newsletter').uncheck();
await page.getByLabel('Newsletter').setChecked(true);
await expect(page.getByLabel('I accept the terms')).toBeChecked();

await page.getByRole('radio', { name: 'Express delivery' }).check();
```

`check()` is better than `click()`: it verifies the element is a checkbox, asserts the resulting state, and does nothing if it is already in the desired state. That removes the classic "the test toggled it off" bug you get with a blind `click()`.

CORE

Rarely — auto-waiting plus retrying assertions cover most cases. But these exist for real situations:

```
await page.waitForURL('**/orders/*');
await page.waitForURL(/\//dashboard/, { timeout: 15_000 });

await page.waitForLoadState('domcontentloaded'); // 'load' | 'domcontentloaded' | 'networkidle'
await page.waitForSelector('.spinner', { state: 'hidden' }); //
'attached' | 'detached' | 'visible' | 'hidden'

await page.waitForResponse(r => r.url().includes('/api/orders') && r.ok());
await page.waitForRequest('**/api/track');
await page.waitForEvent('popup');
await page.waitForEvent('download');

// Wait for arbitrary JS state in the page
await page.waitForFunction(() => window.dataLayer?.length > 0);
await page.waitForFunction(sel => document.querySelectorAll(sel).length > 10, '.row');

// Retry a whole block of assertions
await expect(async () => { expect(await getStatus()).toBe('READY'); }).toPass();

await page.waitForTimeout(1000); // ← the only "sleep". Debugging only, never commit it.
```

`waitForLoadState('networkidle')` is discouraged by the Playwright team. Modern apps hold open WebSockets, polling and analytics beacons, so "network idle" may never happen — or may happen before your data arrives. Wait for what you actually care about: a visible element, a specific response, or a URL.

CORE

TRAP

Q173

Explain the Playwright timeout hierarchy.

TIMEOUT	DEFAULT	SET WITH
Test timeout	30 s	<code>timeout</code> in config, <code>test.setTimeout()</code> , <code>test.slow()</code>
Expect (assertion)	5 s	<code>expect: { timeout }</code> , or per assertion
Action (click/fill)	no limit (bounded by the test timeout)	<code>use: { actionTimeout }</code> , or per call
Navigation	no limit	<code>use: { navigationTimeout }</code>
Global (whole run)	none	<code>globalTimeout</code>
beforeEach / afterEach	shares the test timeout	—
beforeAll / afterAll	own 30 s budget	<code>test.setTimeout()</code> inside the hook

```
test.setTimeout(120_000);
test.slow(); // x3 the test timeout
await expect(locator).toBeVisible({ timeout: 60_000 });
await page.goto('/report', { timeout: 90_000 });
await locator.click({ timeout: 10_000 });
```

Guidance to offer: keep the default timeouts **low**. Long timeouts don't fix flakiness — they just make failures slow. A test that needs 60 seconds usually has a missing wait condition or a genuine performance problem worth reporting.

ADVANCED

```
// First run creates the baseline; Later runs compare against it
await expect(page).toHaveScreenshot('dashboard.png');
await expect(page.getByTestId('price-card')).toHaveScreenshot('price-card.png');

await expect(page).toHaveScreenshot('dashboard.png', {
  fullPage: true,
  maxDiffPixelRatio: 0.01, // tolerance
  threshold: 0.2, // per-pixel colour tolerance
  mask: [page.getByTestId('timestamp'), page.getByTestId('avatar')], // hide dynamic bits
  animations: 'disabled', // freeze CSS animations
  caret: 'hide', // hide the text cursor
});
```

```
npx playwright test --update-snapshots # regenerate baselines after an intended change
```

Practical warnings to mention

- Baselines are **OS- and browser-specific** — a screenshot taken on Windows will not match one taken on the Linux CI agent. Generate baselines **inside the same Docker image CI uses**, or Playwright will fail every run.
- Mask everything non-deterministic: dates, avatars, ad slots, animated charts, random ids.
- Visual tests are high-maintenance. I apply them to a handful of stable, high-value components (design system, print/PDF templates, marketing landing page), not to every screen.
- Alternatives for larger programmes: Applitools / Percy with AI-based comparison and cross-browser baselines.

ADVANCED

```
// In the config, per project
{ name: 'Mobile Safari', use: { ...devices['iPhone 14 Pro'] } }

// Per test / per context
test.use({
  viewport: { width: 375, height: 812 },
  isMobile: true,
  hasTouch: true,
  deviceScaleFactor: 3,
  locale: 'fr-FR',
  timezoneId: 'Europe/Paris',
  colorScheme: 'dark',
  geolocation: { latitude: 12.9716, longitude: 77.5946 },
  permissions: ['geolocation', 'notifications', 'clipboard-read'],
  offline: false,
  httpCredentials: { username: 'qa', password: 'secret' }, // HTTP basic auth
  extraHTTPHeaders: { 'x-test-run': 'nightly' },
});

// At runtime
await context.setGeolocation({ latitude: 19.0760, longitude: 72.8777 });
await context.grantPermissions(['geolocation'], { origin: 'https://example.com' });
await context.setOffline(true); // test the offline/error UI
await page.emulateMedia({ media: 'print', colorScheme: 'dark',
  reducedMotion: 'reduce' });
```

Two things that impress: (1) setting `timezoneId` and `locale` makes date/currency assertions deterministic across CI agents — a real flakiness fix; (2) `setOffline(true)` lets you test the offline experience, which almost nobody tests and which is a genuinely good scenario to volunteer.

Device emulation is **not** real device testing. It changes the viewport, user agent, touch support and pixel ratio, but it is still desktop Chromium/WebKit rendering. Real device bugs (memory, GPU, actual iOS Safari quirks, native keyboard behaviour) need BrowserStack/real hardware. Say this — claiming Playwright replaces mobile testing is a red flag.

```
// pages/base.page.ts
import { Page, Locator, expect } from '@playwright/test';

export abstract class BasePage {
  constructor(protected readonly page: Page) {}
  abstract readonly path: string;

  async open(): Promise<void> {
    await this.page.goto(this.path);
    await this.waitForLoad();
  }
  protected abstract waitForLoad(): Promise<void>;
}

// pages/login.page.ts
export class LoginPage extends BasePage {
  readonly path = '/login';
  // Locators are fields – created once, resolved lazily on each use
  private readonly email = this.page.getByLabel('Email address');
  private readonly password = this.page.getByLabel('Password');
  private readonly signIn = this.page.getByRole('button', { name: 'Sign in' });
  private readonly error = this.page.getByRole('alert');

  protected async waitForLoad() { await expect(this.signIn).toBeVisible(); }

  async loginAs(user: string, pass: string): Promise<DashboardPage> {
    await test.step(`login as ${user}`, async () => {
      await this.email.fill(user);
      await this.password.fill(pass);
      await this.signIn.click();
    });
    return new DashboardPage(this.page);
  }

  async expectError(message: string): Promise<void> {
    await expect(this.error).toHaveText(message);
  }
}
```

```
// tests/login.spec.ts – with the page object injected as a fixture
import { test, expect } from '../fixtures';

test('shows an error for a wrong password', async ({ loginPage }) => {
  await loginPage.open();
  await loginPage.loginAs('valid@test.com', 'Wrong@123');
  await loginPage.expectError('Invalid credentials');
});
```

Playwright-specific POM notes worth saying: (1) locator fields are safe to define as class properties because Locators are lazy — unlike Selenium WebElements; (2) exposing assertion helpers (`expectError`) from a page object is more accepted in Playwright than in Selenium, because web-first assertions are part of the interaction model — but I still keep business assertions in the test; (3) wrap methods in `test.step` so the trace and report read like the scenario.

CORE

Q177 Why TypeScript over plain JavaScript for test automation?

- **Compile-time safety** — a typo in a method name or a wrong argument type fails at build time, not 40 minutes into a nightly run.
- **IDE autocomplete and inline docs** for the whole Playwright API — a huge productivity difference for a team.
- **Safer refactoring** — rename a page-object method and the compiler finds every call site.
- **Self-documenting** — typed test data models (`interface User`) make the intent obvious.
- **Typed fixtures** — `base.extend<MyFixtures>()` only works properly with types.
- Playwright itself is written in TypeScript, so the typings are first-class; it transpiles TS out of the box with no extra build step.

Cost, to be fair: a slightly steeper learning curve and occasional type gymnastics. For anything beyond a throwaway script the trade is clearly worth it.

CORE

```

// ---- Basic types ----
let name: string; let age: number; let active: boolean;
let tags: string[]; let pair: [string, number];           // tuple
let anything: any;    // avoid - disables type checking
let safe: unknown;    // must be narrowed before use - the safe alternative to any
function log(msg: string): void {}
function fail(): never {}                                // never returns

// ---- interface vs type ----
interface User { id: number; name: string; email?: string; readonly role: string; }
interface User { phone?: string; }                      // interfaces MERGE (declaration merging)
interface Admin extends User { permissions: string[]; }

type Status = 'NEW' | 'PAID' | 'SHIPPED';               // union - types can do this
type Id = string | number;
type WithTimestamps = User & { createdAt: Date };       // intersection
// Rule of thumb: interface for object shapes you may extend; type for unions,
// intersections, primitives and function signatures.

// ---- enums ----
enum Role { ADMIN = 'ADMIN', USER = 'USER' }
const enum Env { QA = 'qa', PROD = 'prod' }             // inlined at compile time

// ---- generics ----
function first<T>(items: T[]): T | undefined { return items[0]; }
class ApiClient<T> { async get(id: string): Promise<T> { /* ... */ } }
const client = new ApiClient<Order>();

// ---- utility types (asked surprisingly often) ----
Partial<User>      // all properties optional - great for test-data overrides
Required<User>
ReadOnly<User>
Pick<User, 'id'|'name'>
Omit<User, 'email'>
Record<string, number>
Awaited<ReturnType<typeof fetchUser>>

// A real test-data builder using Partial
function makeUser(overrides: Partial<User> = {}): User {
  return { id: Date.now(), name: 'Test User', role: 'USER', ...overrides };
}
const admin = makeUser({ role: 'ADMIN' });

// ---- modern syntax you should know ----
const city = user?.address?.city ?? 'Unknown';          // optional chaining + nullish coalescing
const { name, ...rest } = user;                         // destructuring + rest
const merged = { ...defaults, ...overrides };           // spread
const el = document.querySelector('#x') as HTMLInputElement; // type assertion
const value = process.env.TOKEN!;                       // non-null assertion

```


Q179

Explain async/await and Promises. What mistakes do people make in Playwright tests?

A **Promise** represents a value that will be available later (pending → fulfilled/rejected). `async` marks a function that returns a Promise; `await` pauses inside it until the Promise settles. Almost every Playwright API returns a Promise.

```
// X MISTAKE 1 – missing await on an assertion: the test always passes
expect(page.getByText('Order confirmed')).toBeVisible();
// ✓
await expect(page.getByText('Order confirmed')).toBeVisible();

// X MISTAKE 2 – missing await on an action: race condition / unhandled rejection
page.getByRole('button', { name: 'Save' }).click();
// ✓
await page.getByRole('button', { name: 'Save' }).click();

// X MISTAKE 3 – await inside forEach: forEach ignores the returned promise
items.forEach(async item => { await item.click(); }); // does NOT wait
// ✓
for (const item of items) { await item.click(); }
// ✓ or, for independent operations, run them concurrently:
await Promise.all(items.map(i => i.click()));

// X MISTAKE 4 – awaiting the trigger before setting up the waiter
await page.getByRole('button', { name: 'Export' }).click();
const download = await page.waitForEvent('download'); // may hang – event already fired
// ✓
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export' }).click(),
]);

// Error handling
try { await page.goto('/slow'); }
catch (e) { console.error('navigation failed', e); throw e; }

// Promise.all (fail fast) vs Promise.allSettled (collect all results)
const [a, b] = await Promise.all([apiCall1(), apiCall2()]);
const results = await Promise.allSettled([apiCall1(), apiCall2()]);
```

Enable `@typescript-eslint/no-floating-promises` and `eslint-plugin-playwright` in the repo. They catch missing awaits automatically — mentioning this shows you think about preventing whole classes of bugs, not just fixing them.

CORE

TRAP


```
// 1. Simple loop – each iteration is a separate, independently reported test
const cases = [
  { email: 'valid@test.com', password: 'Correct@123', error: null },
  { email: 'valid@test.com', password: 'Wrong@123', error: 'Invalid credentials' },
  { email: '', password: 'Correct@123', error: 'Email is required' },
];

for (const { email, password, error } of cases) {
  test(`login: "${email}" / "${password}" → ${error ?? 'success'}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(email);
    await page.getByLabel('Password').fill(password);
    await page.getByRole('button', { name: 'Sign in' }).click();
    if (error) await expect(page.getByRole('alert')).toHaveText(error);
    else      await expect(page).toHaveURL(/\/dashboard/);
  });
}

// 2. From a JSON file
import users from '../data/users.json';
users.forEach(u => test(`checkout as ${u.role}`, async ({ page }) => { /* ... */ })));

// 3. From a CSV (csv-parse)
import { parse } from 'csv-parse/sync';
const rows = parse(fs.readFileSync('data/orders.csv'), { columns: true });

// 4. Generated data – no collisions across parallel workers
import { faker } from '@faker-js/faker';
const email = faker.internet.email();
const unique = `qa_${Date.now()}_${process.env.TEST_WORKER_INDEX}@test.com`;
```

The trap: the loop must generate **separate test() calls**, not a single test with a loop inside. With one test, the first failure stops the rest and the report shows one result instead of ten. Interviewers ask exactly this.

```
// Tags in the title (classic)
test('checkout works @smoke @payments', async ({ page }) => { });

// Tags as metadata (Playwright 1.42+)
test('checkout works', { tag: ['@smoke', '@payments'] }, async ({ page }) => { });

// Annotations – appear in the HTML report
test('refund flow', {
  annotation: [
    { type: 'issue', description: 'https://jira/browse/QA-1234' },
    { type: 'severity', description: 'critical' },
  ],
}, async ({ page }) => { });

test.info().annotations.push({ type: 'note', description: 'Ran against build 4521' });
```

```
npx playwright test --grep @smoke
npx playwright test --grep-invert @slow
npx playwright test --grep "@smoke|@payments"
npx playwright test tests/checkout/           # by path
npx playwright test --project=chromium --repeat-each=3  # flakiness hunting
npx playwright test --last-failed                # rerun only what failed
```

`--repeat-each=10` is the standard way to prove a test is flaky (or prove you've fixed it). `grep` can also be set in the config per project so your smoke project always filters itself.

```
reporter: [  
  ['list'], // Live console output  
  ['html', { outputFolder: 'playwright-report', open: 'never' }],  
  ['junit', { outputFile: 'results/junit.xml' }], // for Jenkins/Azure test tabs  
  ['json', { outputFile: 'results/results.json' }],  
  ['blob'], // for merging sharded runs  
  ['github'], // inline PR annotations in GH Actions  
  ['allure-playwright'], // third-party  
]
```

The built-in HTML report shows: pass/fail/flaky/skipped counts, duration, per-test step tree, error with the source line, attached screenshot, video and trace — and the trace opens directly from the report. That is usually enough; teams add Allure mainly for historical trends.

```
// A minimal custom reporter  
import type { Reporter, TestCase, TestResult } from '@playwright/test/reporter';  
class SlackReporter implements Reporter {  
  private failures: string[] = [];  
  onTestEnd(test: TestCase, result: TestResult) {  
    if (result.status === 'failed') this.failures.push(test.title);  
  }  
  async onEnd() { if (this.failures.length) await postToSlack(this.failures); }  
}  
export default SlackReporter;
```

```

name: E2E Tests
on:
  pull_request:
  push: { branches: [main] }
  schedule: [{ cron: '0 1 * * *' }]      # nightly regression

jobs:
  test:
    timeout-minutes: 60
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      shard: [1, 2, 3, 4]                # shard across 4 runners
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: 'npm' }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test --shard=${{ matrix.shard }}/4
      env:
        BASE_URL: ${vars.BASE_URL}
        USER_PASSWORD: ${secrets.QA_USER_PASSWORD}
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: blob-report-${{ matrix.shard }}
          path: blob-report
          retention-days: 7

  merge-reports:
    if: always()
    needs: [test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - uses: actions/download-artifact@v4
        with: { path: all-blob-reports, pattern: blob-report-* , merge-multiple: true }
      - run: npx playwright merge-reports --reporter html ./all-blob-reports
      - uses: actions/upload-artifact@v4
        with: { name: html-report, path: playwright-report }

```

Alternative: run inside the official image mcr.microsoft.com/playwright:v1.x-jammy, which has all browser dependencies pre-installed — the standard fix for "browsers won't launch on the CI

agent".

CORE

ADVANCED

Q184

How do you manage environments and secrets in a Playwright project?

```
// playwright.config.ts
import dotenv from 'dotenv';
dotenv.config({ path: `.env.${process.env.TEST_ENV ?? 'qa'}` });

export default defineConfig({
  use: { baseURL: process.env.BASE_URL },
});
```

<i># .env.qa (committed, no secrets)</i>	<i># .env.local (git-ignored, secrets)</i>
BASE_URL=https://qa.example.com	USER_PASSWORD=...
API_URL=https://qa-api.example.com	API_TOKEN=...
TEST_ENV=qa	

```
// global setup / teardown – run once for the whole run
// playwright.config.ts: globalSetup: './global-setup.ts'
async function globalSetup(config: FullConfig) {
  await seedReferenceData();
  process.env.RUN_ID = crypto.randomUUID();
}
export default globalSetup;

// A typed, fail-fast config accessor beats process.env sprinkled everywhere
export const env = {
  baseUrl: required('BASE_URL'),
  apiUrl: required('API_URL'),
} as const;
function required(key: string): string {
  const v = process.env[key];
  if (!v) throw new Error(`Missing required env var: ${key}`);
  return v;
}
```

.env* files with real credentials must be git-ignored, and CI must inject them from its secret store (GitHub Secrets, Jenkins credentials, Vault). If a candidate says they keep the QA password in the repo, that's an immediate negative signal.

CORE

```
npm i -D @axe-core/playwright
```

```
import AxeBuilder from '@axe-core/playwright';

test('dashboard has no critical accessibility violations', async ({ page }) => {
  await page.goto('/dashboard');

  const results = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa', 'wcag21aa'])
    .exclude('#third-party-widget') // out of our control
    .analyze();

  // attach the full report so failures are actionable
  await test.info().attach('axe-results', {
    body: JSON.stringify(results.violations, null, 2),
    contentType: 'application/json',
  });

  const critical = results.violations.filter(v => v.impact === 'critical');
  expect(critical, critical.map(v => `${v.id}: ${v.help}`).join('\n')).toEqual([]);
});
```

Common findings: missing form labels, insufficient colour contrast, missing alt text, wrong heading order, non-focusable interactive elements, missing ARIA landmarks.

Be accurate about the limits: automated scanning catches roughly a third of WCAG issues. It cannot judge whether alt text is *meaningful*, whether the tab order is *logical*, or whether a screen-reader flow makes sense. Automated scan in CI + periodic manual keyboard/screen-reader testing is the honest answer.

ADVANCED

	PLAYWRIGHT	CYPRESS
Execution model	Out-of-process, drives the browser via a debug protocol	Runs inside the browser, in the same event loop as the app
Multiple tabs / origins	Native support	Historically restricted; improved but still awkward
Browsers	Chromium, Firefox, WebKit	Chrome family, Firefox, Electron; WebKit experimental
Language	TS/JS, Python, Java, .NET	JS/TS only
Syntax	Standard async/await	Chained commands with an internal queue (not real promises)
Parallelism	Free, built-in workers & sharding	Parallelisation historically tied to Cypress Cloud
Debugging	Trace Viewer, UI mode	Excellent interactive runner with time-travel
Strength	Cross-browser E2E, multi-user, network control, speed	Superb developer experience for front-end devs on a single app

Balanced conclusion: "Cypress has a lovely developer experience and front-end teams love it. Playwright wins on cross-browser coverage, true parallelism at no cost, multi-tab/multi-user scenarios and language choice — which is why it's become the default for QA-owned E2E suites."

ADVANCED

```
// Find a row by content and act on it – readable, no XPath axes needed
const row = page.getByRole('row', { name: /ORD-991/ });
await expect(row.getByRole('cell', { name: 'Delivered' })).toBeVisible();
await row.getByRole('button', { name: 'Cancel' }).click();

// Or with filter()
const row2 = page.locator('tbody tr').filter({ hasText: 'ORD-991' });

// Read a whole column
const statuses = await page.locator('tbody tr td:nth-child(4)').allTextContents();
expect(statuses).toContain('Delivered');

// Assert sorting
const prices = (await page.getByTestId('price').allTextContents())
  .map(t => Number(t.replace(/[^\d-9.]/g, '')));
expect(prices).toEqual([...prices].sort((a, b) => a - b));

// Row count with auto-retry (waits for the table to load)
await expect(page.locator('tbody tr')).toHaveCount(25);

// Iterate safely
for (const row of await page.locator('tbody tr').all()) {
  const id = await row.getByTestId('order-id').textContent();
  console.log(id);
}

// Pagination
while (await page.getByRole('button', { name: 'Next' }).isEnabled()) {
  await collectCurrentPage();
  await page.getByRole('button', { name: 'Next' }).click();
  await expect(page.getByTestId('page-indicator')).not.toHaveText(previousPage);
}
```

Compare with the Selenium equivalent (Q95): no stale-element loops, no `ancestor::` XPath, and `toHaveCount` waits for the table to populate. This is the concrete example to give when asked "what does Playwright actually make easier?"

What is component testing in Playwright, and when would you use it?

Playwright can mount a single React/Vue/Svelte component in a real browser and test it in isolation — no full app, no backend, no navigation.

```
import { test, expect } from '@playwright/experimental-ct-react';
import { PriceTag } from './PriceTag';

test('formats Indian currency and shows a discount badge', async ({ mount }) => {
  const component = await mount(<PriceTag amount={1499} discount={20} currency="INR" />);
  await expect(component).toContainText('₹1,499.00');
  await expect(component.getByTestId('discount-badge')).toHaveText('20% off');
});
```

When to use it: for a design system or a component with many states (loading, empty, error, long text, RTL, disabled). Testing 12 states of a component takes seconds here versus minutes of navigating the full app in E2E. It sits between unit tests (jsdom, not a real browser) and E2E (real browser, full app).

Note that it is still marked experimental — say so rather than presenting it as a mainstream production choice.

ADVANCED

Never answer "rewrite everything". Answer with a risk-managed plan.

1. **Justify it with data, not preference.** Current runtime, flakiness rate, maintenance hours per sprint, debugging time per CI failure. If those numbers aren't painful, don't migrate.
2. **Pilot.** Port 10–15 representative tests (including the trickiest: iframes, file upload, multi-tab, dynamic tables) in a spike. Measure runtime and stability against the Selenium equivalents. Present the comparison.
3. **Run both in parallel.** Keep Selenium as the gate while Playwright coverage grows; nothing loses coverage during the transition.
4. **Migrate by value, not by file order.** Start with the flakiest and slowest suites — the biggest wins first — plus any new tests, which are written only in Playwright from day one.
5. **Rebuild rather than transliterate.** A line-by-line port carries over every explicit wait and every brittle XPath. Re-express tests with `getByRole` and web-first assertions, or you inherit the old flakiness in new syntax.
6. **Address the team.** Java/Selenium people need TypeScript and async/await training; budget for it. (Playwright does have a Java binding, but the TS ecosystem is where the tooling and examples are.)
7. **Retire deliberately.** Delete each Selenium test only when its Playwright counterpart has been green for a couple of weeks.

The mature closing line: *"Migration is a cost with no new features for the business, so it has to be justified by measurable pain and delivered incrementally without ever leaving the team without a working regression suite."*

ADVANCED

Q190

What are the most common Playwright mistakes and best practices?

Mistakes

- Missing `await` — the assertion never runs and the test always passes.
- Using `page.waitForTimeout()` as a wait strategy.
- Using CSS/XPath everywhere instead of role/label/testid locators.
- Using `force: true` to "make it work" instead of understanding the failure.
- Using `page.$()` / `ElementHandles` instead of `Locators`.
- Logging in through the UI in every test instead of `storageState`.
- Sharing state between tests (module-level variables, a fixed test record) — breaks parallelism.
- Committing `test.only` (set `forbidOnly: true` in CI).
- Turning up timeouts to hide a real problem.
- Not enabling traces, then having no idea why CI failed.

Best practices

- Test user-visible behaviour, not implementation details.
- Prefer `getByRole` → `getByLabel` → `getByTestId` ; CSS/XPath last.
- Web-first assertions everywhere; let them do the waiting.
- Each test independent, self-seeding its data via API, running from a clean context.
- `trace: 'on-first-retry'` , screenshots and video on failure.
- Page objects + fixtures + `test.step` for readable reports.
- Lint with `eslint-plugin-playwright` ; type everything.
- Keep the smoke suite under 10 minutes and make it a merge gate.

CORE

ADVANCED

SECTION 6

API, Database, Scenario & Behavioural

Q191 – Q200 · The last 15 minutes of the interview. These decide whether they can imagine working with you — and they are the easiest marks to win, because most candidates prepare only the technical part.

METHOD	PURPOSE	SAFE?	IDEMPOTENT?
GET	Read a resource	Yes	Yes
POST	Create a resource / trigger an action	No	No
PUT	Replace a resource entirely	No	Yes
PATCH	Partially update	No	Usually not guaranteed
DELETE	Remove	No	Yes
HEAD / OPTIONS	Headers only / supported methods (CORS preflight)	Yes	Yes

Idempotent = calling it 5 times has the same effect as calling it once. That is exactly why a double-clicked "Place order" button (POST) can create two orders — a great real bug to mention.

CODE	MEANING
200 / 201 / 202 / 204	OK / Created / Accepted (async) / No Content
301 / 302 / 304	Moved permanently / Found / Not Modified (cache)
400	Bad request — malformed payload
401 vs 403	401 = not authenticated (who are you?) · 403 = authenticated but not authorised (I know you, you can't do this). This distinction is asked constantly.
404 / 405 / 409	Not found / Method not allowed / Conflict (duplicate)
415 / 422 / 429	Unsupported media type / Unprocessable entity (validation) / Too many requests (rate limit)

500 / 502 /
503 / 504

Server error / Bad gateway / Service unavailable / Gateway timeout

What I actually test on an API

Happy path + status code; response schema and data types; field-level validation (missing, null, wrong type, too long, boundary); authentication and — critically — **authorisation** (user A must get 403 when reading user B's order); pagination and sorting; idempotency and duplicate submission; rate limiting; error message quality and that stack traces are not leaked; response time; and backward compatibility of the contract.

CORE

Explain API authentication types and how you handle tokens in tests.

- **Basic auth** — `Authorization: Basic base64(user:pass)`. Simple, must be over HTTPS.
- **API key** — a static key in a header or query param. Identifies the app, not the user.
- **Bearer token / JWT** — `Authorization: Bearer eyJhbGci...`. A JWT has three dot-separated parts: **header.payload.signature**. The payload is only Base64-encoded, *not encrypted* — so you can decode it in a test to assert claims (`sub` , `exp` , `role`), and you should verify the app never puts sensitive data in it.
- **OAuth 2.0** — token issued by an authorisation server. Grant types: authorization code (+PKCE) for user-facing apps, client credentials for machine-to-machine (the one you usually automate).
- **Session cookie** — traditional server-side session.

Handling tokens in a test framework

```
// Fetch once, cache, and refresh when it expires – not per test
public static synchronized String token() {
    if (cached == null || Instant.now().isAfter(expiry.minusSeconds(60))) {
        Response r = given().contentType(JSON)
            .body(Map.of("client_id", id, "client_secret", secret,
                "grant_type", "client_credentials"))
            .post(authUrl + "/oauth/token");
        cached = r.jsonPath().getString("access_token");
        expiry = Instant.now().plusSeconds(r.jsonPath().getInt("expires_in"));
    }
    return cached;
}
```

Security tests to mention: expired token → 401; tampered signature → 401; token from user A used on user B's resource → 403; token still valid after logout (it shouldn't be); token visible in URL or browser logs (it shouldn't be).

ADVANCED

```
-- JOINS
SELECT o.order_ref, c.name, o.total
FROM orders o
INNER JOIN customers c ON c.id = o.customer_id      -- only matching rows
LEFT JOIN payments p ON p.order_id = o.id          -- all orders, payment if it exists
WHERE p.id IS NULL;                                -- orders with NO payment (a real bug hunt)

-- Aggregation: WHERE filters rows BEFORE grouping, HAVING filters groups AFTER
SELECT status, COUNT(*) AS cnt, SUM(total) AS revenue
FROM orders
WHERE created_at >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY status
HAVING COUNT(*) > 10
ORDER BY revenue DESC;

-- Find duplicates (a data-integrity test)
SELECT email, COUNT(*) FROM customers GROUP BY email HAVING COUNT(*) > 1;

-- 2nd highest salary – the classic interview question, three ways
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;

SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = 2;

-- Nth highest per department (window function)
SELECT * FROM (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) rnk
  FROM employees e
) x WHERE rnk <= 3;

-- Useful for test data
SELECT * FROM orders WHERE status IN ('NEW','PAID') AND total BETWEEN 100 AND 500;
SELECT * FROM customers WHERE email LIKE '%@test.com';
SELECT COALESCE(nickname, name) FROM customers;      -- NULL handling
DELETE FROM orders WHERE order_ref LIKE 'AUTOTEST-%'; -- cleanup
```

Two things interviewers check: (1) WHERE vs HAVING — filtering before vs after GROUP BY; (2) that you understand NULL never equals anything, so WHERE col = NULL returns nothing — you need IS NULL. Also: use a read-only account, and never run an UPDATE/DELETE without a WHERE on a shared environment.

Q194

"Tell me about yourself." How do you structure the answer and your project introduction?

This is the highest-leverage 3 minutes of the interview and the one candidates prepare least. Use **Present** → **Past** → **Why here**, and end on your strongest technical asset.

Template (adapt with your own numbers)

"I'm a test automation engineer with about five years' experience, currently working on [domain — e.g. an e-commerce platform] at [company]. I own the UI and API automation for the [checkout and payments] modules.

Day to day I work in a Scrum team of [8]: I join refinement to shape acceptance criteria, write and automate the regression tests, and maintain our framework — Java, Selenium 4, TestNG, Maven, Page Object Model, running in parallel on a Docker Grid through Jenkins. Over the last year I've also built out our Playwright/TypeScript suite for the new front end.

The thing I'm proudest of is [cutting regression from 3h40 to 34 minutes by introducing parallel execution and replacing UI logins with API-based authentication] — that let the team move from weekly to twice-weekly releases.

Before this I was at [X] doing [manual + automation], which is where I learned [test design, domain]. I'm looking to move because [I want to work on a product with a stronger engineering culture / deeper CI-CD maturity / a bigger scale challenge], and this role stood out because [specific thing about their job description]."

Rules

- Two to three minutes. Practise it out loud until it needs no thinking.
- Include at least one **number** — runtime, coverage, defects caught, release frequency.
- Only mention technologies you can defend for ten minutes. Everything you name is a licence for them to dig.
- Do not narrate your CV chronologically from college. Start from now.
- End by connecting to *their* role — it shows you read the job description.

CORE

BEHAVIOURAL

Q195

"Tell me about the biggest challenge you faced." How do you structure behavioural answers?

Use **STAR**: **S**ituation (context, 2 sentences) → **T**ask (your specific responsibility) → **A**ction (what *you* did — most of the answer) → **R**esult (measurable outcome + what you learned).

Worked example

Situation: "Our regression suite had a 60% pass rate on any given night. The team had stopped looking at the results, so failures were being ignored and two real regressions reached production in one quarter."

Task: "I took ownership of suite reliability for that quarter, alongside my sprint work."

Action: "I started by measuring instead of guessing — I ran the suite 20 times and classified every failure: 40% were fixed sleeps and presence-only waits, 30% were test-data collisions between parallel threads, 20% were a shared static WebDriver, and 10% were genuine product bugs. I fixed them in that order: replaced every sleep with condition-based explicit waits, moved data creation into each test through the API with unique identifiers, and moved the driver to a ThreadLocal factory. I also introduced a quarantine policy — a test that fails twice for non-product reasons leaves the blocking pipeline with a named owner and a deadline."

Result: "Pass rate went from 60% to 98%, and because the team trusted it again we were able to make the smoke suite a merge gate. Escaped regressions dropped to zero over the following two quarters. What I took away is that **trust is the real product of a test suite** — coverage means nothing if nobody believes the result."

Prepare **three** STAR stories in advance: a technical challenge, a conflict/collaboration story, and a failure/mistake story. Almost every behavioural question is a variation of one of those three.

CORE

BEHAVIOURAL

They are testing your **collaboration and evidence discipline**, not your debugging skill. Never answer with "I escalate to my manager" as the first step.

1. **Assume good faith and check yourself first.** Re-read the requirement — is my expected result actually right? Re-run on a clean environment with the exact build.
2. **Strengthen the evidence.** Exact build number, environment, browser and version, user role, test data, timestamp, screenshot/video, browser console error, network HAR, server log with the correlation ID. Most "works on my machine" disputes die the moment you attach a request/response pair.
3. **Reproduce together.** A 10-minute screen share resolves more than a day of ticket comments. Often it turns out to be an environment, data, cache or feature-flag difference — which is itself a finding worth fixing.
4. **Separate the fact from the judgement.** "The behaviour is X, the requirement says Y" is a fact. Whether it's a defect or a requirement gap is a decision — take that to the BA/PO, not to a debate.
5. **Escalate only with data**, and framed as risk to the release, never as a personal complaint.

Close with: *"I try to make the bug report so complete that there's nothing left to argue about. If I'm proven wrong, I say so quickly — that's how I keep the credibility I need for the times I'm right."*

CORE

BEHAVIOURAL

Q197

"The release is tomorrow, testing isn't complete, and there are open bugs. What do you do?"

The wrong answers: "I'd work overnight and finish everything" (unrealistic, and signals poor planning) or "I'd block the release" (QA doesn't own that decision).

1. **Re-prioritise by risk immediately.** Cover the critical revenue paths and the areas that actually changed. Explicitly deprioritise low-risk, unchanged areas.
2. **Present the facts, not feelings.** A short written summary: what was tested, what wasn't, the open defect list with severity and workaround status, and the specific risk of shipping each one. "If we ship, users on Safari cannot complete UPI payments — that's roughly 12% of transactions."
3. **Offer options rather than a veto:** ship with the feature behind a flag / disabled; ship to a canary segment first; ship without the risky module; delay 24 hours; ship with a documented known-issue and a hotfix plan.
4. **Let the business decide, and record the decision.** The PO owns go/no-go. My job is to make sure they decide with full information, and that the decision is written down.
5. **Follow up after release** — monitor error rates, be ready with the rollback, and raise the underlying planning problem in the retro so it doesn't repeat.

Say the principle out loud: **"QA's job is to make risk visible, not to make the release decision alone."** That single sentence is what a hiring manager is listening for.

CORE

BEHAVIOURAL

Keeping current — be specific, not generic

Name actual sources and actual outputs: the official Selenium and Playwright release notes and docs, the Ministry of Testing community, specific newsletters/blogs, conference talks, and — most importantly — **something you built**. "I maintain a personal GitHub repo where I rebuild our framework patterns from scratch; last month I added CDP network mocking and a Playwright visual-regression pilot." A public repo is worth more than a list of blogs you read.

Weakness — the honest formula

Pick a **real** weakness that is not core to the job, then show the concrete action you're taking and the progress made. Avoid the fake ones ("I'm a perfectionist", "I work too hard") — experienced interviewers discount them immediately.

Good examples: *"My performance-testing depth is shallower than my functional automation. I've been working through JMeter and k6 on a side project and ran our first baseline load test last quarter, but I'd still ask for support on a serious performance programme."* Or: *"Early on I over-automated — I built UI tests for things that belonged at the API layer, and I paid for it in maintenance. Now I consciously ask 'what is the cheapest layer that can catch this bug?' before writing a test."*

BEHAVIOURAL

Q199

"Why are you leaving?", "Where do you see yourself in 5 years?", and salary questions.

Why are you leaving?

Never criticise your current employer, manager or colleagues — it is the fastest way to lose an offer. Frame it as moving *towards* something: "I've automated our regression to a good level and the work has become steady maintenance. I'm looking for a role with more scale and more CI/CD depth, where I can also mentor — which is what attracted me to this position."

Five years

Show ambition in the direction of the role, not away from it. "I'd like to grow into an SDET/lead position — owning quality strategy across teams, mentoring juniors, and being more involved in architecture and shift-left practices. I want to stay technical rather than move into pure people management." Avoid "I want to become a manager in 2 years" if the role has no such path, and avoid "I want to move into development" — it tells them you'll leave.

Salary

- Research the market range for your years, stack and city before the interview.
- If asked early, deflect politely once: "I'd like to understand the role and responsibilities a bit better first — but I'm confident we can align if the fit is right. Could you share the band for this position?"
- If pressed, give a **range** anchored to research, with the bottom of the range being a number you'd genuinely accept.
- Speak in **total compensation** (fixed + variable + benefits), and never inflate your current salary — offer letters get verified.

BEHAVIOURAL

SCENARIO	KEY POINTS IN A STRONG ANSWER
"How would you test a pen / a lift / an ATM?"	Ask clarifying questions first, then answer in categories: functional, usability, boundary/limits, safety/security, performance, environment, accessibility, negative. They're testing your structure , not the pen.
"You have 100 test cases and 1 hour."	Risk-rank; run critical business paths and the areas that changed; communicate what wasn't covered in writing.
"Requirements are missing/unclear. How do you test?"	Talk to the PO/BA and users; use the existing behaviour, comparable products and domain knowledge as an oracle; do exploratory sessions; document assumptions and get them confirmed; log requirement gaps as defects.
"A test that passed for months suddenly fails."	Check first whether it's a genuine product regression (most likely), then environment/data change, dependency upgrade, or a hidden timing assumption that finally lost the race. Bisect against the last good build.
"How would you test an application with no UI?"	API/contract testing, DB validation, message queue/event verification, log assertions, scheduled-job outputs, CLI-level testing.
"Would you automate 100% of tests?"	No — explain ROI, exploratory/usability value, churn cost, and the "what can't be automated" list (Q34).
"How do you test a payment flow safely?"	Sandbox gateway with test card numbers, never real cards; test decline/timeout/3DS/partial refund; verify idempotency on double submit; check that no PAN/CVV is logged; reconcile against the DB and the gateway's report.
"Your manager asks you to mark a failing test as skipped to make the build green."	Push back with data and offer the alternative: quarantine with an owner and a ticket, or a documented known-issue. Never silently disable. Escalate transparently, not confrontationally.
"How do you onboard a junior tester?"	Pair on real tickets; give them a small, safe area to own; code review with explanations rather than corrections; a written framework README and a "first week" checklist.

"What would you improve about your current framework?"

Have a real, specific answer ready — it shows self-awareness. E.g. "test data is still partly shared; I'd move fully to per-test API seeding" or "our reporting has no historical trend; I'd add Allure or a dashboard."

For every scenario question: **ask a clarifying question first, then answer in a structure, then state a trade-off.** That three-part shape works for anything they invent.

CORE

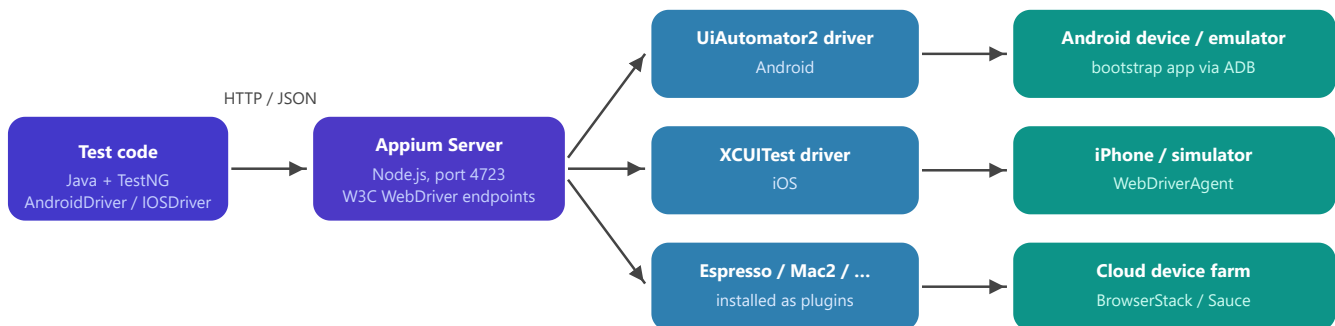
BEHAVIOURAL

SECTION 7

Appium & Mobile Automation

Q201 – Q215 · Asked in almost every product-company automation interview, even for web-first roles. You are not expected to be an expert — you are expected to know the architecture, the locator strategies, and why mobile is harder than web.

Appium 2 Architecture — one API, many platform drivers



The key idea: Appium speaks the **same W3C WebDriver protocol as Selenium**, so your framework, page objects, TestNG setup and reporting are reused almost unchanged. Only the driver and locators differ.

Figure 11 — Appium architecture. If you can say "it's WebDriver for apps, with a platform driver behind it", you have answered 80% of the mobile question.

Appium is an open-source tool for automating **native, hybrid and mobile-web** apps on Android and iOS, using the same **W3C WebDriver protocol** as Selenium.

1. Your test creates an **AndroidDriver** / **IOSDriver** with a set of capabilities.
2. The client sends HTTP/JSON commands to the **Appium server** (Node.js, default port 4723).
3. The server routes them to the right **platform driver**: **UiAutomator2** for Android, **XCUITest** for iOS.
4. That driver talks to the vendor's own automation framework on the device — via **ADB** and a bootstrap agent on Android, via **WebDriverAgent** on iOS.
5. The action executes on the real device or emulator and the result travels back up the same chain.

Why teams choose it

- **Cross-platform** — one API and often one test codebase for Android and iOS.
- **No app modification** — you test the *same binary* you ship; no SDK to embed.
- **Any language** that has a WebDriver client (Java, Python, JS, C#, Ruby).
- **Reuses your Selenium knowledge** — waits, page objects, TestNG, reporting all carry over.

The sentence that lands: *"Appium is WebDriver for mobile. My framework layers — driver factory, page objects, waits, listeners, reporting — were reused as-is; I only added a mobile driver factory and mobile locators."*

AREA	APPIUM 1.X	APPIUM 2.X
Drivers	Bundled inside the server	Installed separately as plugins — <code>appium driver install uiautomator2</code>
Versioning	Server upgrade forced a driver upgrade	Drivers version independently of the server
Protocol	JSON Wire + W3C	W3C only — JSONWP removed
Capabilities	<code>DesiredCapabilities</code> , flat keys	<code>UiAutomator2Options</code> ; non-standard keys need the <code>appium:</code> prefix
Plugins	—	Plugin ecosystem: images, relaxed-caps, execute-driver, device-farm
Base path	<code>/wd/hub</code>	Root <code>/</code> by default

```

UiAutomator2Options options = new UiAutomator2Options()
    .setPlatformName("Android")
    .setPlatformVersion("14")
    .setDeviceName("Pixel_7_API_34")
    .setAutomationName("UiAutomator2")
    .setApp(System.getProperty("user.dir") + "/apps/shop-qa.apk")
    .setAppPackage("com.example.shop")
    .setAppActivity(".ui.MainActivity")
    .setNoReset(false) // fresh app state each run
    .setAutoGrantPermissions(true) // skip the OS permission dialogs
    .setNewCommandTimeout(Duration.ofSeconds(120));

AndroidDriver driver = new AndroidDriver(new URL("http://127.0.0.1:4723"), options);

```

The `appium:` prefix trips people up constantly. Standard W3C capabilities (`platformName`, `browserName`) need no prefix; everything vendor-specific (`appium:app`, `appium:deviceName`, `appium:automationName`) does. The typed Options classes add it for you, which is why they are the recommended approach.

STRATEGY	EXAMPLE	NOTES
Accessibility ID	<code>AppiumBy.accessibilityId("cart_button")</code>	Best choice. May be used on Android with <code>content-desc</code> and <code>accessibility</code> on iOS — so one works on both platforms and it doubles as accessibility checker
ID	<code>AppiumBy.id("com.example.shop:id/btn_pay")</code>	Android resource ID is unique and stable
Class name	<code>AppiumBy.className("android.widget.Button")</code>	Rarely unique on Android
XPath	<code>//android.widget.TextView[@text='Checkout']</code>	Works everywhere but is slowest on mobile. Appium driver serialises the whole view hierarchy for query
Android UiAutomator	<code>AppiumBy.androidUIAutomator("new UiSelector().text(\"Pay\")")</code>	Native and fast; scrolls to an element
iOS Predicate	<code>AppiumBy.iOSNsPredicateString("label == 'Pay' AND visible == 1")</code>	Fast, expressive
iOS Class Chain	<code>AppiumBy.iOSClassChain("**/XCUIElementTypeButton[`label == 'Pay'`]")</code>	Between XPath and predicate in power, faster than XPath

```
// Scroll to an element that is off-screen – the idiomatic Android way
driver.findElement(AppiumBy.androidUIAutomator(
    "new UiScrollable(new UiSelector().scrollable(true))"
    + ".scrollIntoView(new UiSelector().text(\"Order history\"))").click();
```

The senior answer to "which do you prefer": "Accessibility ID first, because a single locator works on both platforms and it forces the app to be accessible. XPath last, because on mobile it is genuinely slow — the driver dumps the entire view hierarchy on every lookup, which on a long list can take seconds."

CORE

Q204 Native vs Hybrid vs Mobile web apps — and how do you switch context?

- **Native** — built with the platform SDK (Kotlin/Java, Swift/Objective-C). Best performance and full device access. Automated entirely through the native context.
- **Hybrid** — a native shell wrapping a `WebView` that renders HTML/JS (Cordova, Ionic, Capacitor). Automating it needs **context switching**.
- **Mobile web** — a normal website in the phone's browser. Automated with Appium driving Chrome/Safari, using ordinary web locators.
- (Worth naming: **React Native / Flutter** — they render native components, so they're treated as native, though Flutter needs a dedicated driver.)

```
Set<String> contexts = driver.getContextHandles();
// [ "NATIVE_APP", "WEBVIEW_com.example.shop" ]

for (String context : contexts) {
    if (context.contains("WEBVIEW")) {
        driver.context(context);           // switch into the web view
        break;
    }
}
driver.findElement(By.cssSelector("#promo-code")).sendKeys("SAVE20"); // web locators now

driver.context("NATIVE_APP");           // switch back – always switch back
```

Two classic hybrid problems: (1) the WebView context appears only **after** the web content has loaded, so you must wait for it rather than reading the handles immediately; (2) Android requires the app to be built with `setWebContentsDebuggingEnabled(true)`, otherwise the WebView is invisible to Appium — a request you have to make to the developers.

CORE

ADVANCED

Appium 2 uses the **W3C Actions API** (the old `TouchAction` class is deprecated). You build a pointer sequence.

```
PointerInput finger = new PointerInput(PointerInput.Kind.TOUCH, "finger");

// --- Tap ---
Sequence tap = new Sequence(finger, 0)
    .addAction(finger.createPointerMove(Duration.ZERO, viewport(), x, y))
    .addAction(finger.createPointerDown(PointerInput.MouseButton.LEFT.asArg()))
    .addAction(finger.createPointerUp(PointerInput.MouseButton.LEFT.asArg()));
driver.perform(List.of(tap));

// --- Swipe up (scroll down the screen) ---
Dimension size = driver.manage().window().getSize();
int startX = size.width / 2, startY = (int) (size.height * 0.8), endY = (int) (size.height * 0.2);
Sequence swipe = new Sequence(finger, 1)
    .addAction(finger.createPointerMove(Duration.ZERO, viewport(), startX, startY))
    .addAction(finger.createPointerDown(0))
    .addAction(finger.createPointerMove(Duration.ofMillis(600), viewport(), startX, endY))
    .addAction(finger.createPointerUp(0));
driver.perform(List.of(swipe));

// --- Long press: same as tap, with a pause between down and up ---
    .addAction(new Pause(finger, Duration.ofSeconds(2)))

// --- The pragmatic shortcut: mobile: commands (fast and readable) ---
driver.executeScript("mobile: swipe", Map.of("direction", "up"));
driver.executeScript("mobile: scroll", Map.of("strategy", "accessibility id", "selector", "Pay"));
driver.executeScript("mobile: longClickGesture", Map.of("elementId",
    ((RemoteWebElement) el).getId(), "duration", 2000));
driver.executeScript("mobile: pinchCloseGesture", Map.of("percent", 0.75));
```

Mention `mobile: execute-script` commands — they are the modern, driver-native way and far less code than hand-building pointer sequences. Candidates who only know the deprecated `TouchAction` immediately date themselves.

Q206

Emulator/simulator vs real device vs cloud device farm — what do you use and when?

	EMULATOR / SIMULATOR	REAL DEVICE	CLOUD FARM
Cost	Free	Hardware + maintenance	Subscription
Speed	Fast to spin up, scriptable in CI	Fastest execution, slow to provision	Fast, elastic
Coverage	Any OS version, any screen size	Only what you own	Hundreds of real devices
Fidelity	Cannot test camera, GPS accuracy, biometrics, real network, battery, push, performance, or OEM skins	Full	Full

My strategy: emulators for the bulk of functional regression in CI (cheap and parallelisable), a small set of **real devices** for the release candidate covering the top handsets in our analytics, and a **cloud farm** for the long tail of OS/OEM combinations plus anything needing real network conditions.

Anchor it in data: *"We pulled the device/OS distribution from our analytics and covered the models representing 80% of our users, rather than guessing."* That is exactly the risk-based thinking they want.

CORE

This is the question that separates people who have really tested mobile from people who have only read about Appium.

- **Interrupts** — incoming call, SMS, alarm, push notification, or another app taking focus mid-transaction. Does state survive?
- **App lifecycle** — background the app, kill it from recents, relaunch. Is the session preserved? Is the cart intact? Is the payment double-charged?
- **Install / upgrade / uninstall** — upgrading from the previous version must migrate local data, not crash or wipe it. Very commonly missed.
- **Permissions** — deny camera/location and confirm graceful degradation, then grant it later from OS settings.
- **Network transitions** — WiFi → 4G → airplane mode → back. Offline mode, retry, and queued requests.
- **Device conditions** — low battery, low storage, low memory (does the OS kill the app?), device rotation, split screen, foldables.
- **Hardware back button** (Android) and swipe-back gesture (iOS) at every screen.
- **Deep links / push notification taps** landing on the right screen, including when the app is cold-started.
- **Accessibility & display** — large system font, dark mode, TalkBack/VoiceOver, small screens.
- **Localisation** — RTL layouts, long translated strings overflowing buttons.

CORE

ADVANCED

Which ADB commands should a mobile tester know?

```
adb devices                # list connected devices/emulators
adb -s emulator-5554 shell # target a specific device

adb install -r app-qa.apk   # -r = reinstall, keep data
adb uninstall com.example.shop
adb shell pm clear com.example.shop # wipe app data – a clean-state reset

adb logcat                 # live device log
adb logcat *:E             # errors only
adb logcat | grep -i "com.example.shop" # filter to your app – crash stack traces live here
adb bugreport              # full diagnostic bundle

adb shell input text "hello" # type
adb shell input keyevent 4   # 4 = BACK, 3 = HOME, 26 = POWER, 82 = MENU
adb shell input swipe 500 1500 500 300 # swipe

adb push file.pdf /sdcard/Download/ # put a test file on the device
adb pull /sdcard/report.csv ./      # fetch a file the app produced
adb shell screencap /sdcard/s.png   # screenshot
adb shell screenrecord /sdcard/v.mp4 # video

adb shell dumpsys battery set level 5 # simulate low battery
adb shell svc wifi disable            # network conditions
adb shell am start -a android.intent.action.VIEW -d "shopapp://orders/991" # deep link
adb shell settings put global window_animation_scale 0 # disable animations = less flaky
```

That last line is a genuine framework tip: turning off window, transition and animator scales on the device removes a whole class of mobile flakiness. Saying it shows you have actually stabilised a mobile suite.

ADVANCED

```
// Explicit waits work exactly as in Selenium
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));
wait.until(ExpectedConditions.visibilityOfElementLocated(AppiumBy.accessibilityId("cart_button")));

// Appium-specific: keep the session alive during long operations
options.setNewCommandTimeout(Duration.ofSeconds(120));
```

Mobile-specific flakiness causes

- **Animations and transitions** — the element exists but is still sliding in. Fix: disable animation scales via ADB, and wait for a stable state.
- **Slow emulator cold start** — the app is not ready when the driver session begins. Fix: wait for a known first-screen element, not a fixed sleep.
- **Soft keyboard covering elements** — `driver.hideKeyboard()` after typing, or use `setDisableWindowAnimation` /keyboard capabilities.
- **XPath timeouts on long lists** — the hierarchy dump is expensive; switch to accessibility id or UiSelector.
- **Elements outside the viewport** — mobile only exposes what is rendered, so you must scroll before the element even exists in the tree.
- **Stale sessions** — the device locks or the app is killed by the OS. Fix: `newCommandTimeout`, keep the screen awake, and reset cleanly between tests.
- **Shared device contention** in CI — two jobs grabbing the same device. Fix: a device pool/lock, or a cloud farm.

Same principle as web — `ThreadLocal` driver — plus one extra dimension: each thread needs its **own device and its own Appium server port**.

```
<suite name="MobileSuite" parallel="tests" thread-count="3">
  <test name="Pixel7">
    <parameter name="udid"      value="emulator-5554"/>
    <parameter name="systemPort" value="8200"/>
    <parameter name="serverPort" value="4723"/>
    <classes><class name="tests.CheckoutTest"/></classes>
  </test>
  <test name="Pixel5">
    <parameter name="udid"      value="emulator-5556"/>
    <parameter name="systemPort" value="8201"/>
    <parameter name="serverPort" value="4725"/>
    <classes><class name="tests.CheckoutTest"/></classes>
  </test>
</suite>
```

```
@BeforeMethod
@Parameters({"udid", "systemPort", "serverPort"})
public void setUp(String udid, int systemPort, int serverPort) throws MalformedURLException {
    UiAutomator2Options options = new UiAutomator2Options()
        .setUdid(udid)
        .setSystemPort(systemPort) // MUST be unique per device, or sessions collide
        .setApp(appPath);
    MobileDriverFactory.set(new AndroidDriver(
        new URL("http://127.0.0.1:" + serverPort), options));
}
```

The mistake almost everyone makes: forgetting `systemPort` (Android) or `wdaLocalPort` (iOS). Without a unique port per device, the second session hijacks the first one's bridge and you get bizarre cross-device failures that look like application bugs.

Q211

How do you structure a mobile automation framework? Can you share code with the web suite?

Yes — and saying how is a strong senior answer.

```
src/main/java/  
  core/  
    DriverFactory.java      // returns WebDriver: Chrome, Firefox, AndroidDriver, IOSDriver  
    BasePage.java          // click/type/waitFor – written against WebDriver, not ChromeDriver  
  pages/  
    web/    LoginPage.java  CheckoutPage.java  
    mobile/ LoginScreen.java CheckoutScreen.java  
  utils/                                // SHARED: ApiClient, DbClient, DataFactory, ExcelReader, Logger
```

What is shared: the driver factory pattern, the wait helpers, config layering, test data creation via API, reporting/listeners, and the whole build/CI setup. What is platform-specific: locators, gestures, and app lifecycle handling.

Cross-platform page objects (Android + iOS from one class)

```
public class LoginScreen extends BaseScreen {  
    private final By email = isAndroid()  
        ? AppiumBy.id("com.example.shop:id/email")  
        : AppiumBy.accessibilityId("emailField");  
  
    // ...or better: ask developers for the SAME accessibility id on both platforms,  
    // and then a single locator serves both:  
    private final By password = AppiumBy.accessibilityId("passwordField");  
}
```

Push the **shared accessibility id** convention as a team practice. It removes platform branching from your locators *and* improves the app's accessibility. That "one change, two benefits" framing is exactly how a senior engineer argues for testability.

ADVANCED

Q212

How do you find locators in a mobile app?

- **Appium Inspector** — the standard tool. Connect to a running session, tap an element in the mirrored screen and it shows the id, accessibility id, class, XPath and a suggested locator.
- **uiautomatorviewer** (legacy Android SDK tool) and **Android Studio's Layout Inspector**.
- **Xcode Accessibility Inspector** for iOS.
- `driver.getPageSource()` — dumps the XML view hierarchy; useful for scripting and for CI debugging where no GUI exists.
- Ask the developers to add `contentDescription` (Android) / `accessibilityIdentifier` (iOS) — the mobile equivalent of `data-testid`.

A trap: elements that look identical on screen can have different hierarchies on the two platforms, so a locator verified in the Inspector on Android may not exist on iOS. Always verify both, and prefer the shared accessibility id so you only maintain one.

CORE

```
// Device Logs through Appium (no ADB needed)
LogEntries logs = driver.manage().logs().get("logcat"); // Android
logs.forEach(e -> logger.info("{} {}", e.getLevel(), e.getMessage()));
driver.manage().logs().getAvailableLogTypes(); // logcat, bugreport, server, syslog...

// Screenshot – identical API to Selenium
File shot = driver.getScreenshotAs(OutputType.FILE);

// Screen recording – invaluable for CI-only mobile failures
driver.startRecordingScreen(new AndroidStartScreenRecordingOptions()
    .withTimeLimit(Duration.ofMinutes(3)));
// ... run the test ...
String base64Video = driver.stopRecordingScreen();
Files.write(Paths.get("videos/test.mp4"), Base64.getDecoder().decode(base64Video));

// Device state helpers
driver.runAppInBackground(Duration.ofSeconds(5)); // simulate an interruption
driver.activateApp("com.example.shop");
driver.terminateApp("com.example.shop");
driver.installApp(path); driver.removeApp("com.example.shop");
driver.rotate(ScreenOrientation.LANDSCAPE);
driver.toggleAirplaneMode();
driver.setLocation(new Location(12.9716, 77.5946, 0));
```

Attaching the **logcat output around the failure timestamp** to your report is the single most useful thing for mobile debugging — a screenshot shows the symptom, logcat shows the stack trace of the crash.

ADVANCED

What are the biggest challenges in mobile automation?

- **Device fragmentation** — thousands of Android device/OS/OEM-skin combinations. Mitigate with analytics-driven device selection and a cloud farm.
- **Slow, unstable infrastructure** — emulator boot times, WebDriverAgent rebuilds on iOS, devices disconnecting, ports colliding. Mobile CI needs far more babysitting than web CI.
- **iOS friction** — signing certificates, provisioning profiles, needing a Mac build agent, Xcode upgrades breaking WebDriverAgent.
- **Slow feedback** — a mobile suite is inherently slower than a web suite; you have to be ruthless about what belongs at the UI level.
- **Locator instability** in cross-platform frameworks (React Native/Flutter) where the rendered tree changes between versions.
- **App state management** — deciding between `noReset` (fast, dirty) and full reset (slow, clean) per suite.
- **Things you simply cannot automate** — biometrics, real payments (NFC), camera scanning, OS-level dialogs on some versions, push notification delivery.

End with the mitigation, not the complaint: *"Because mobile UI tests are expensive, we kept the mobile UI layer to the top 15 user journeys and pushed everything else to API tests and unit tests on the app side. That kept the mobile pipeline under 25 minutes."*

CORE

ADVANCED

Q215

How would you test a mobile app end to end, if asked in an open question?

Answer in layers, exactly as with the login page question — categories, not a random list.

1. **Functional** — install, onboarding, signup/login, core journeys (search → cart → pay), logout, edge cases and validations.
2. **App lifecycle** — background/foreground, kill and relaunch, upgrade from the previous version, uninstall/reinstall, cold vs warm start.
3. **Interrupts** — call, SMS, notification, alarm, another app, screen lock during a transaction.
4. **Connectivity** — offline behaviour, slow 2G/3G, network switching, request retry, sync on reconnect.
5. **Device & OS matrix** — screen sizes, OS versions, OEM skins, tablets, foldables, notches, dark mode, large fonts.
6. **Performance** — cold start time, screen render time, memory and battery usage, APK/IPA size, ANRs and crash-free rate.
7. **Security** — data at rest not stored in plain text, no sensitive data in logs, certificate pinning, root/jailbreak handling, session expiry, screenshot blocking on sensitive screens.
8. **Accessibility** — TalkBack/VoiceOver, touch target sizes, contrast, dynamic type.
9. **Store readiness** — permissions declared, privacy policy, store metadata, and compliance rules.

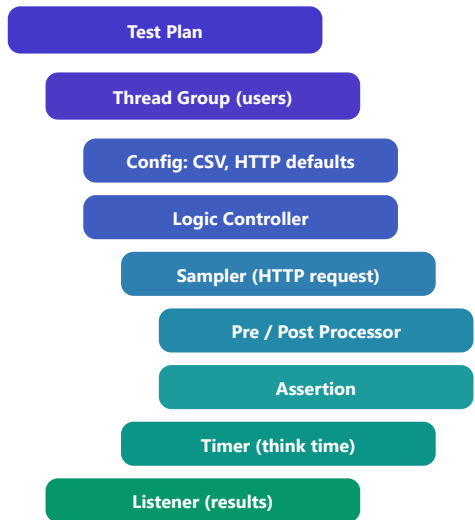
CORE

SECTION 8

Performance, Security & Non-Functional Testing

Q216 – Q230 · You are not applying for a performance-engineer role, so nobody expects mastery. What they check is whether you can **design** a load test, **read** a result, and name the OWASP risks — enough to be dangerous and to know when to call a specialist.

A load test: structure on the left, what you read on the right



Read these, in this order

1. Error rate

If it is not ~0%, response times are meaningless — fix errors first.

2. Throughput (req/sec)

Did it keep rising with load, or flatten? Flat = saturation point found.

3. Response time percentiles

p50 / p90 / p95 / p99 — never the average. The average hides the 5% of users who are actually suffering.

4. Resource utilisation

CPU, memory, GC pauses, DB connection pool, disk I/O, thread count.

5. Correlate the three graphs

Response time up + throughput flat + CPU flat = the bottleneck is waiting on something else (usually the database or a downstream API).

"Average response time was 1.2s" is a weak answer.

"p95 was 3.4s against a 2s SLA" is a strong one.

Figure 12 — Load test anatomy and the order in which you interpret the results. The reading order matters more than the tool.

Q216

Explain JMeter's main components.

COMPONENT	PURPOSE	EXAMPLE
Test Plan	The container for everything	—
Thread Group	Defines the virtual users: count, ramp-up period, loop count / duration	200 users, ramp up over 120s, run 30 min
Sampler	The actual request sent	HTTP Request, JDBC Request, JMS, FTP
Logic Controller	Controls flow	Loop, If, Transaction, Throughput, Once Only
Config Element	Shared configuration/data	HTTP Request Defaults, CSV Data Set Config, Cookie/Header Manager
Pre/Post Processor	Runs before/after a sampler	JSON Extractor, Regular Expression Extractor (correlation)
Assertion	Validates the response — without these you are only measuring, not testing	Response Assertion, JSON Assertion, Duration Assertion
Timer	Adds think time between requests to model real users	Constant, Gaussian Random, Uniform Random
Listener	Collects and displays results	Summary Report, Aggregate Report, View Results Tree

Two things interviewers listen for: (1) you run real tests in **non-GUI mode** — `jmeter -n -t plan.jmx -l results.jtl -e -o report/` — because the GUI itself consumes resources and skews results; (2) you **disable View Results Tree** during the run for the same reason. Candidates who only know the GUI have never run a real load test.

CORE

What is correlation and parameterisation in performance testing?

- **Parameterisation** — replacing hardcoded input with varied data so every virtual user is different. Without it, 200 users all log in as the same account, all hit the same cached row, and your numbers are fiction.
- **Correlation** — capturing a **dynamic value the server generated** (session id, CSRF token, order id, JSESSIONID) from one response and feeding it into the next request. Without it, every request after login fails with 401/403 and you "prove" the system is blazing fast at returning errors.

```
# Parameterisation – CSV Data Set Config
users.csv: email,password
          qa_user_01@test.com,Pass@123
          qa_user_02@test.com,Pass@123
Then reference them as ${email} / ${password} in the sampler.

# Correlation – JSON Extractor on the Login response
Variable name : authToken
JSON path    : $.access_token
Then use ${authToken} in the Authorization header of every later request.

# Correlation – Regular Expression Extractor (HTML/CSRF)
Reference name      : csrf
Regular Expression  : name="_csrf" value="(.*?)"
Template            : $1$
Match No.           : 1
```

If you have only ever done functional automation, say exactly that and then explain the concepts anyway: *"I've supported performance runs rather than owned them — I prepared the data and validated the functional correctness of the scripts. Correlation was the thing that broke most often."* Honest scope plus real understanding beats bluffing every time.

ADVANCED

1. **Get the requirement in numbers.** "Fast" is not a requirement. Ask for: expected concurrent users, peak hour transactions, growth headroom, and the SLA — e.g. "*p95 < 2s at 500 concurrent users, error rate < 0.1%*". If nobody knows, derive it from production analytics.
2. **Build a workload model** from real traffic, not from imagination: 70% browse, 20% search, 8% add-to-cart, 2% checkout. Model the *mix*, not just the one flow you care about.
3. **Add realistic think time** — humans pause 3–10 seconds between clicks. Zero think time turns a 500-user test into an unintentional stress test.
4. **Prepare data** — unique users, unique carts, enough product inventory that you don't all collide on one SKU, and a DB seeded to production-like volume (query plans change dramatically between 1k and 10M rows).
5. **Environment** — production-like scale, or a known ratio you can extrapolate from. Document the difference; a test on a half-size environment does not prove full-size behaviour.
6. **Ramp up gradually** and include a steady state; don't slam all users in at second zero unless you are deliberately spike testing.
7. **Monitor the server side**, not just the client: CPU, memory, GC, DB connections, slow query log, downstream API latency. The client only tells you *that* it was slow, never *why*.
8. **Baseline first**, then compare every later run to it. A single run without a baseline tells you almost nothing.
9. **Exclude third parties** you don't own, or stub them — you can't fix someone else's payment sandbox, and it will dominate your numbers.

ADVANCED

Q219

The load test shows response times rising. How do you find the bottleneck?

Work from the outside in, and use the pattern of the three curves.

PATTERN	LIKELY CAUSE
Response time ↑, throughput flat, CPU low	Waiting on something: DB, downstream API, lock contention, thread/connection pool exhaustion
Response time ↑, CPU pegged at 100%	Genuine compute limit or an inefficient algorithm; scale out or optimise the hot path
Response time rises <i>gradually</i> over hours	Memory leak, connection leak, unbounded cache, log disk filling — classic soak-test finding
Periodic latency spikes	Garbage collection pauses, cron/batch jobs, cache expiry stampede
Errors start at a specific user count	A hard limit: connection pool size, thread pool, rate limiter, file descriptors
Only one endpoint is slow	Missing DB index or an N+1 query — check the slow query log first

Tools to name: APM (New Relic, Dynatrace, AppDynamics), distributed tracing (Jaeger, Zipkin, OpenTelemetry), **EXPLAIN** plans and the slow query log, JVM tools (JFR, VisualVM, GC logs), and Grafana dashboards.

Be clear about your role: *"I identify and evidence the bottleneck; fixing it is usually a joint call with the backend team. What I own is making sure the finding is reproducible and quantified."*

ADVANCED

	JMETER	K6	GATLING
Script	XML via GUI (or code)	JavaScript	Scala / Java / Kotlin DSL
Learning curve	Easy to start, GUI-driven	Easy for JS developers	Steeper
Resource use per VU	Thread-per-user — heavy	Goroutine-based — very light	Async — light
Version control / diffs	Painful (large XML)	Clean (plain code)	Clean (plain code)
Protocols	Very broad (HTTP, JDBC, JMS, LDAP, FTP...)	Mostly HTTP, gRPC, WebSocket	HTTP, WebSocket, JMS
CI/CD fit	Good in non-GUI mode	Excellent — designed for it	Excellent
Reporting	HTML dashboard	Built-in + Grafana	Very good built-in

Balanced answer: "JMeter where the team needs breadth of protocols or already has a library of plans; k6 for a modern CI-first setup where performance tests live in the repo next to the code and run on every release. The scripting model is what really decides it — plain code is reviewable and diffable, XML is not."

ADVANCED

OWASP Top 10 — and the one check a QA can run for each

A01 Broken Access Control

Change an id in the URL/API and see if you get someone else's data (IDOR)

A02 Cryptographic Failures

HTTPS everywhere? Passwords hashed? Card data or tokens in logs/localStorage?

A03 Injection (SQL / NoSQL / command / XSS)

Submit ' OR '1'='1 and <script>alert(1)</script> into every input

A04 Insecure Design

Threat-model in refinement: what could an attacker do with this feature?

A05 Security Misconfiguration

Default credentials, directory listing, stack traces shown to users, debug on

A06 Vulnerable & Outdated Components

npm audit / OWASP Dependency-Check in the pipeline

A07 Identification & Authentication Failures

Session valid after logout? Brute force allowed? Weak password accepted?

A08 Software & Data Integrity Failures

Unsigned updates, untrusted CI plugins, insecure deserialisation

A09 Logging & Monitoring Failures

Is a failed login attempt logged and alertable? Are logs leaking PII?

A10 Server-Side Request Forgery (SSRF)

Any field taking a URL — can it be pointed at an internal address?

Figure 13 — You are not expected to be a penetration tester. You are expected to know these ten names and one practical check for each.

Q221 Walk me through the OWASP Top 10.

See Figure 13 for the full list with a practical check for each. Structure your spoken answer as: **the risk** → **what it looks like** → **how I'd test it**. You do not need to recite all ten in order; naming five with a concrete test each scores better than listing ten you cannot explain.

The three to know cold

- **A01 Broken Access Control** — the #1 risk. The test is trivial and finds real bugs: log in as user A, capture an API call like `GET /api/orders/1001`, change it to `1002` and see whether you get user B's order. Same for changing a role field in a request body, or hitting an admin URL directly as a normal user.
- **A03 Injection** — untrusted input reaching an interpreter. SQL injection, and XSS (stored and reflected).
- **A07 Authentication failures** — session not invalidated on logout, no rate limiting on login, weak password policy, session fixation, tokens that never expire.

Add the shift-left framing: *"I raise these as acceptance criteria during refinement — 'a user must not be able to read another user's order' is a testable requirement, not a security afterthought."*

ADVANCED

SQL injection

```
' OR '1'='1          -- classic auth bypass in a login field
admin'--             -- comment out the password check
' UNION SELECT null,null-- -- column probing
'; WAITFOR DELAY '0:0:5'-- -- blind/time-based: did the response take 5s longer?
1 OR 1=1             -- numeric parameters, no quotes needed
```

Where to try it: login fields, search boxes, URL query parameters, sort/filter parameters, and **API request bodies** — which are frequently unprotected because the UI validates but the API does not.

What "safe" looks like: the input is treated as a literal string, you get a normal "no results" or a validation error — never a database error message, never a different result set, and never a timing difference.

XSS

```
<script>alert(1)</script>
<img src=x onerror=alert(1)>
"><svg onload=alert(1)>
javascript:alert(1)          -- in any field that becomes a link
```

- **Reflected** — the payload comes back in the same response (e.g. in a search results heading).
- **Stored** — it is saved and executes for *other* users later (profile name, review, comment). Far more dangerous.
- **DOM-based** — the payload never reaches the server; client-side JS writes it into the DOM.
- **Pass criteria:** the payload is displayed as literal text, HTML-encoded in the response, and no dialog appears.

Say this explicitly, because it matters: **only ever run these against systems you are authorised to test** — your own QA environment, with the team's knowledge. Never against production, and never against a third party. Interviewers notice when a candidate shows that instinct unprompted.

What security tools would you put in the pipeline?

TYPE	WHAT IT DOES	TOOLS	WHEN IT RUNS
SAST (static)	Scans source code for insecure patterns	SonarQube, Semgrep, CodeQL	Every PR
SCA (dependencies)	Flags known CVEs in libraries	<code>npm audit</code> , OWASP Dependency-Check, Snyk, Dependabot	Every build + scheduled
DAST (dynamic)	Attacks the running app	OWASP ZAP (has a CI-friendly baseline scan), Burp Suite	Nightly against QA
Secret scanning	Finds committed keys/passwords	gitleaks, truffleHog, GitHub secret scanning	Pre-commit + CI
Container scanning	Vulnerabilities in base images	Trivy, Gype	On image build
Pen test	Human expert, creative attacks	External specialists	Per major release / annually

The realistic QA role: own the **automated** layers (SCA, secret scanning, a ZAP baseline scan), include security cases in functional testing (authorisation, session, input validation), and know when to escalate to a specialist. Do not claim to be a penetration tester.

ADVANCED

Authentication asks "who are you?"; **authorisation** asks "are you allowed to do this?" — and it is where the expensive bugs live, because the UI hides the button but the API often still accepts the call.

The matrix I build

Rows = roles (guest, user, premium user, support agent, admin). Columns = protected operations. Every cell is a test: allowed → 200, not allowed → **403** (not 200, not 500, and not a silent success).

```
// The test that catches it – call the API directly, bypassing the UI
@Test(dataProvider = "roles")
public void userCannotReadAnotherUsersOrder(String role, String token) {
    Response res = given().header("Authorization", "Bearer " + token)
        .get("/api/v1/orders/" + OTHER_USERS_ORDER_ID);
    assertEquals(res.statusCode(), 403,
        role + " was able to read another user's order – broken access control");
}
```

Variants to cover

- **IDOR / BOLA** — increment or guess an object id and see whose data comes back.
- **Vertical escalation** — a normal user calling an admin endpoint directly.
- **Horizontal escalation** — user A acting on user B's resource at the same privilege level.
- **Mass assignment** — adding `"role": "ADMIN"` to a profile-update payload and seeing if it sticks.
- **Hidden UI ≠ protected** — the button is hidden for this role, but is the endpoint itself protected?
- **Token misuse** — an expired token, a token from another tenant, a token after logout, a modified JWT payload.

This single answer can carry an entire interview. It shows API thinking, security awareness, data-driven test design and an understanding of why UI-only testing is insufficient.

WCAG (Web Content Accessibility Guidelines) has three conformance levels: **A** (minimum), **AA** (the usual legal and contractual target), **AAA** (rarely required in full). Its four principles are **POUR**:

- **Perceivable** — text alternatives for images, captions for video, sufficient colour contrast (4.5:1 normal text, 3:1 large text), information not conveyed by colour alone.
- **Operable** — everything reachable and usable by keyboard, no keyboard traps, visible focus, enough time to complete tasks, no flashing content.
- **Understandable** — predictable navigation, clear labels, helpful error messages that say how to fix the problem, consistent components.
- **Robust** — valid semantic HTML and correct ARIA so assistive technologies can parse it.

A realistic three-layer programme

1. **Automated in CI** — axe-core in every Playwright/Selenium run. Catches roughly 30–40%: missing labels, contrast, missing alt, heading order.
2. **Manual keyboard pass** — tab through the whole flow with the mouse unplugged. Fast, needs no tooling, and finds what scanners can't: illogical tab order, invisible focus, unreachable modals.
3. **Screen reader spot checks** — NVDA (Windows), VoiceOver (Mac/iOS), TalkBack (Android) on the critical journeys, plus zoom to 200% and a large-font test.

Never claim "we're WCAG compliant because axe passes". Automated tools cannot judge whether alt text is *meaningful*, whether the reading order makes sense, or whether an error is *understandable*. Stating that limitation is what makes the answer credible.

What is contract testing and when do you need it?

In a microservices system, full end-to-end tests across every service are slow, flaky and hard to own. **Contract testing** verifies the agreement between a *consumer* and a *provider* independently, without running both together.

1. The **consumer** (e.g. the web app) writes a test declaring "when I call `GET /orders/1`, I expect a 200 with fields `id`, `status`, `total`". Running it produces a **pact file** and runs the consumer against a local mock built from it.
2. The pact is published to a broker.
3. The **provider** (the order service) replays every pact against its real implementation in its own pipeline. If a developer renames `total` to `totalAmount`, the provider build goes red **before** the change is deployed.

When you need it: multiple independently deployed services, teams releasing on different schedules, and a full E2E suite that is too slow or too flaky to gate every deploy. **When you don't:** a monolith, or a single team owning both sides where a shared integration test is simpler.

The framing that lands: "Contract tests let me delete most of my cross-service end-to-end tests. I keep a handful of true E2E journeys for confidence, and catch integration breakages at the contract level, where the feedback is in seconds instead of an hour." Tool to name: **Pact**.

ADVANCED

Functional tests pass while the layout is broken — the button works, it's just invisible behind the header. Visual regression catches that by comparing rendered screenshots against approved baselines.

APPROACH	HOW IT COMPARES	TRADE-OFF
Built-in (<code>toHaveScreenshot</code>)	Pixel diff with a tolerance	Free, in-repo; baselines are OS/browser-specific and noisy
Applitools / Percy	AI/perceptual comparison	Ignores anti-aliasing noise, handles cross-browser baselines; paid
Storybook + snapshot tests	Per-component	Very stable — the best place to start

Making it survivable

- Generate baselines **in the same container CI uses**, or every run fails on font rendering alone.
- **Mask** everything dynamic: dates, avatars, ads, charts, random ids, session-specific text.
- Freeze animations and hide the caret.
- Apply it to **components and a few key pages**, not every screen — otherwise baseline maintenance eats the team.
- Treat a diff as a *question*, not a failure: someone must approve whether the change was intended.

The honest risk to state: visual tests have the worst signal-to-noise ratio of any test type. A suite that cries wolf gets ignored within two sprints. Start with 10 high-value screens and grow only if the team is actually acting on the diffs.

ADVANCED

Copying a production database into QA is the single most common compliance violation in testing teams — and increasingly illegal under GDPR, India's DPDP Act, HIPAA and PCI-DSS.

- **Anonymisation / masking** — irreversibly replace names, emails, phone numbers, card and ID numbers while preserving format and distribution, so the data still behaves realistically.
- **Pseudonymisation** — replace identifiers with consistent tokens so relationships across tables still work.
- **Synthetic data** — generate it (Faker, Mockaroo) rather than copying. Safest option; the challenge is realistic edge cases and volume.
- **Subsetting** — take a referentially-consistent 1–5% slice instead of the whole database.
- **Minimisation** — don't copy the columns you don't need to test.
- **Never log or screenshot PII** — mask it in test reports too; a failure screenshot of a real customer's account is still a data breach.
- **Access control & retention** — restrict who can reach lower environments and delete test data on a schedule.
- **Payment data** — never real card numbers. Use the gateway's official test cards.

Mentioning GDPR/DPDP unprompted is a genuine differentiator — it signals you think about risk beyond the feature, which is exactly what a QA lead needs.

ADVANCED

Deliberately injecting failure into a system to verify it degrades gracefully instead of collapsing — testing the *recovery* path, which is the path that is almost never exercised before a real incident.

Experiments

- Kill a service instance / pod — does traffic reroute? Does the user notice?
- Add latency or drop packets between services — do timeouts, retries and circuit breakers behave, or do they cascade?
- Take the database read replica down — does it fail over cleanly?
- Exhaust CPU/memory/disk on a node.
- Make a third-party dependency (payment gateway, email provider) fail or hang — does the user get a sensible message and is the order left in a recoverable state?
- Clock skew and expired certificates — an underrated source of real outages.

The discipline: form a hypothesis ("if the payment gateway times out, orders stay in PENDING and the customer sees a retry option"), define the blast radius, run the smallest possible experiment, measure, and always have an abort switch. Tools: Chaos Monkey, Gremlin, LitmusChaos, Toxiproxy.

Even without chaos tooling, you can say something true and valuable: *"I test the failure paths deliberately — I stub the payment API to return 500 and to hang, and assert the UI message, the retry, and the final data state. Most teams only test the happy path of their error handling."*

ADVANCED

What is observability, and how does it change a tester's job?

Monitoring tells you *that* something is wrong against known thresholds. **Observability** is the ability to ask new questions about system behaviour without shipping new code — built on three pillars:

- **Logs** — discrete events with context and a correlation id.
- **Metrics** — numeric time series (error rate, latency percentiles, throughput, saturation).
- **Traces** — one request followed across every service, showing where the time went.

What changes for QA

- **Testability becomes a requirement you advocate for** — correlation ids, structured logs, meaningful error codes. "How will we debug this in production?" is a refinement question.
- **Better bug reports** — attaching a trace id turns a two-day investigation into a ten-minute one.
- **Shift-right testing** — synthetic monitoring that reuses your E2E tests against production on a schedule, so you detect breakage before customers report it.
- **Data-driven regression scope** — production usage data tells you which flows actually matter, so risk-based selection stops being guesswork.
- **Release verification** — after a deploy, watch error rate and latency rather than only running a smoke suite.

ADVANCED

SECTION 9

Practical Coding Exercises

Q231 – Q250 · The live-coding round. Type every one of these by hand at least once — reading them is not the same as producing them under pressure on a shared screen while someone watches.

How to behave in a live-coding round. Say the plan out loud before typing ("I'll find the row by its order id, then scope the button lookup inside that row"). Name your variables properly even under pressure. Handle the empty/not-found case. Add the assertion message. If you blank, say what you'd search for — *"I'd check the ExpectedConditions class for the right condition name"* — because knowing where to look is a real skill and interviewers accept it.

```

public abstract class BasePage {
    protected final WebDriver driver;
    protected final WebDriverWait wait;

    protected BasePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(
            Config.getInt("timeout.explicit")));
    }

    protected WebElement visible(By locator) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }
    protected WebElement clickable(By locator) {
        return wait.until(ExpectedConditions.elementToBeClickable(locator));
    }
    protected void click(By locator) { clickable(locator).click(); }

    protected void type(By locator, String text) {
        WebElement el = visible(locator);
        el.clear();
        el.sendKeys(text);
    }
    protected String textOf(By locator) { return visible(locator).getText().trim(); }
    protected boolean isPresent(By locator) { return !driver.findElements(locator).isEmpty(); }

    protected boolean isVisible(By locator, Duration timeout) {
        try {
            new WebDriverWait(driver, timeout)
                .until(ExpectedConditions.visibilityOfElementLocated(locator));
            return true;
        } catch (TimeoutException e) { return false; }
    }

    protected void waitForSpinnerToClear() {
        wait.until(ExpectedConditions.invisibilityOfElementLocated(
            By.cssSelector("[data-testid='loading']")));
    }
}

```

Point out *why* this class exists: one wait strategy, one place to change a timeout, and tests that never touch the Selenium API. That reasoning is what's being marked, not the syntax.

Q232

Write a method that returns a table cell value given a row identifier and a column name.

The single most-asked Selenium coding exercise.

```
/**
 * Returns the value of {@code columnName} in the row whose any cell equals {@code rowKey}.
 * Works regardless of column order, so it survives the UI adding a column.
 */
public String getCellValue(String rowKey, String columnName) {
    List<WebElement> headers = driver.findElements(By.cssSelector("#orders thead th"));
    int columnIndex = -1;
    for (int i = 0; i < headers.size(); i++) {
        if (headers.get(i).getText().trim().equalsIgnoreCase(columnName)) {
            columnIndex = i;
            break;
        }
    }
    if (columnIndex == -1)
        throw new IllegalArgumentException("Column not found: " + columnName
            + ". Available: " + headers.stream().map(WebElement::getText).toList());

    WebElement row = driver.findElements(By.cssSelector("#orders tbody tr")).stream()
        .filter(r -> r.findElements(By.tagName("td")).stream()
            .anyMatch(c -> c.getText().trim().equals(rowKey)))
        .findFirst()
        .orElseThrow(() -> new NoSuchElementException("No row containing: " + rowKey));

    return row.findElements(By.tagName("td")).get(columnIndex).getText().trim();
}

// usage
assertEquals(getCellValue("ORD-991", "Status"), "Delivered");
```

The detail that impresses: looking the column up **by name** instead of hardcoding `td[4]`. Say it out loud — "if the team inserts a column, index-based code silently asserts the wrong cell; this doesn't."

CORE**HANDS-ON**


```

public void selectDropdownValue(By dropdown, String visibleText) {
    WebElement el = clickable(dropdown);

    if ("select".equalsIgnoreCase(el.getTagName())) {           // native <select>
        new Select(el).selectByVisibleText(visibleText);
        return;
    }

    el.click();                                                  // custom widget
    By option = By.xpath("//*[ @role='option' or contains(@class,'option')] "
        + "[normalize-space()=' " + quote(visibleText) + "']");
    clickable(option).click();
    wait.until(ExpectedConditions.invisibilityOfElementLocated(
        By.cssSelector("[role='listbox']")));                    // wait for it to close
}

/** Safely quotes a value for XPath, including values containing an apostrophe. */
private static String quote(String value) {
    if (!value.contains("'")) return "'" + value + "'";
    if (!value.contains("\"")) return "\"" + value + "\"";
    return "concat(' " + value.replace("'", "\'", "'") + "')";
}

```

The `quote()` helper is a genuine senior touch — a customer named *O'Brien* breaks naive XPath string concatenation, and almost nobody handles it. Mention it even if you don't write the full implementation.

ADVANCED

HANDS-ON

Q234**Write a stale-safe click with retry.**

```
public void safeClick(By locator) {
    int attempts = 0;
    while (attempts < 3) {
        try {
            clickable(locator).click();
            return;
        } catch (StaleElementReferenceException e) {
            attempts++;
            // DOM re-rendered – Look it up again
        } catch (ElementClickInterceptedException e) {
            // something is covering it: scroll it to the middle and retry once
            ((JavascriptExecutor) driver).executeScript(
                "arguments[0].scrollIntoView({block:'center'})";
            driver.findElement(locator));
            attempts++;
        }
    }
    throw new FrameworkException("Could not click " + locator + " after 3 attempts. URL="
        + driver.getCurrentUrl());
}
```

Interviewers often follow up with "isn't this hiding a problem?" The answer: *"It absorbs a genuine race in a re-rendering SPA. It does not silently pass — it fails loudly after three attempts with the URL in the message. And I'd still prefer to fix the root cause by waiting for the re-render to finish."*

CORE**HANDS-ON**

Q235

Write code to scroll an infinite-scroll page until a given item appears (or the page stops growing).

```
public WebElement scrollUntilFound(By locator, int maxScrolls) {
    JavascriptExecutor js = (JavascriptExecutor) driver;
    long previousHeight = -1;

    for (int i = 0; i < maxScrolls; i++) {
        List<WebElement> found = driver.findElements(locator);
        if (!found.isEmpty()) {
            js.executeScript("arguments[0].scrollIntoView({block:'center'})", found.get(0));
            return found.get(0);
        }

        long height = (Long) js.executeScript("return document.body.scrollHeight");
        if (height == previousHeight) break;           // nothing new loaded - stop
        previousHeight = height;

        js.executeScript("window.scrollTo(0, document.body.scrollHeight);");
        wait.until(d -> (Long) ((JavascriptExecutor) d)
            .executeScript("return document.body.scrollHeight") != height
            || !d.findElements(locator).isEmpty());
    }
    throw new NoSuchElementException("Not found after scrolling: " + locator);
}
```

Two things to say aloud: the **maxScrolls guard** (never write an unbounded loop in a test) and the **height-unchanged break** (detects the true end of the list instead of scrolling forever).

ADVANCED**HANDS-ON**

Q236**Write code to switch to a browser window by its title, and back.**

```
public String switchToWindowByTitle(String expectedTitle) {
    String original = driver.getWindowHandle();

    wait.until(d -> d.getWindowHandles().size() > 1);           // wait for the new window

    for (String handle : driver.getWindowHandles()) {
        driver.switchTo().window(handle);
        if (driver.getTitle().contains(expectedTitle)) {
            return original;                                     // caller keeps it to switch back
        }
    }
    driver.switchTo().window(original);                         // nothing matched – restore state
    throw new NoSuchElementException("No window with title containing: " + expectedTitle);
}

// usage
String parent = switchToWindowByTitle("Terms");
assertTrue(driver.getPageSource().contains("Privacy Policy"));
driver.close();
driver.switchTo().window(parent);
```

CORE**HANDS-ON**

```

public enum SortOrder { ASC, DESC }

/**
 * @param parser converts the cell text into something Comparable
 *           (e.g. Double::parseDouble for prices, LocalDate::parse for dates)
 */
public <T extends Comparable<T>> void assertColumnSorted(
    By cellLocator, Function<String, T> parser, SortOrder order) {

    List<T> actual = driver.findElements(cellLocator).stream()
        .map(WebElement::getText)
        .map(String::trim)
        .filter(s -> !s.isEmpty())
        .map(parser)
        .toList();

    if (actual.size() < 2) throw new SkipException("Not enough rows to verify sorting");

    List<T> expected = new ArrayList<>(actual);
    expected.sort(order == SortOrder.ASC
        ? Comparator.naturalOrder() : Comparator.reverseOrder());

    Assert.assertEquals(actual, expected,
        "Column is not sorted " + order + ". Actual order was: " + actual);
}

// usage
assertColumnSorted(By.cssSelector("td.price"),
    s -> Double.parseDouble(s.replaceAll("[^0-9.]", "")), SortOrder.ASC);
assertColumnSorted(By.cssSelector("td.date"),
    s -> LocalDate.parse(s, DateTimeFormatter.ofPattern("dd/MM/yyyy")), SortOrder.DESC);

```

Generics + a parser function is exactly the kind of reusable design a 5-year candidate is expected to reach for, instead of writing `verifyPriceSorted()`, `verifyDateSorted()`, `verifyNameSorted()`.

ADVANCED

HANDS-ON

```

public final class ExcelReader {
    public static List<Map<String,String>> read(String path, String sheetName) {
        List<Map<String,String>> rows = new ArrayList<>();
        try (FileInputStream fis = new FileInputStream(path);
            Workbook wb = new XSSFWorkbook(fis)) {

            Sheet sheet = Optional.ofNullable(wb.getSheet(sheetName))
                .orElseThrow(() -> new FrameworkException("No sheet: " + sheetName));
            Row header = sheet.getRow(0);
            DataFormatter fmt = new DataFormatter();           // reads any cell type as text

            for (int r = 1; r <= sheet.getLastRowNum(); r++) {
                Row row = sheet.getRow(r);
                if (row == null) continue;
                Map<String,String> map = new LinkedHashMap<>();
                for (int c = 0; c < header.getLastCellNum(); c++) {
                    map.put(fmt.formatCellValue(header.getCell(c)),
                        fmt.formatCellValue(row.getCell(c)));
                }
                if (map.values().stream().anyMatch(v -> !v.isBlank())) rows.add(map);
            }
        } catch (IOException e) {
            throw new FrameworkException("Failed reading " + path, e);
        }
        return rows;
    }
    private ExcelReader() { }
}

// ---- DataProvider ----
@DataProvider(name = "checkoutData")
public Iterator<Object[]> checkoutData() {
    return ExcelReader.read("src/test/resources/testdata/orders.xlsx", "Checkout")
        .stream().map(row -> new Object[]{ row }).iterator();
}

@Test(dataProvider = "checkoutData")
public void checkout(Map<String,String> data) {
    checkoutPage.payWith(data.get("paymentMethod"), data.get("cardNumber"));
    assertEquals(confirmationPage.getTotal(), data.get("expectedTotal"));
}

```

```

public class TestListener implements ITestListener {

    @Override public void onTestStart(ITestResult result) {
        ExtentManager.set(ExtentManager.report()
            .createTest(result.getMethod().getMethodName(),
                result.getMethod().getDescription()));
        ExtentManager.test().assignCategory(result.getMethod().getGroups());
    }

    @Override public void onTestSuccess(ITestResult result) {
        ExtentManager.test().pass("Passed in "
            + (result.getEndMillis() - result.getStartMillis()) + " ms");
        ExtentManager.unload();
    }

    @Override public void onTestFailure(ITestResult result) {
        WebDriver driver = DriverFactory.get();
        if (driver != null) {
            String base64 = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BASE64);
            ExtentManager.test().fail(result.getThrowable(),
                MediaEntityBuilder.createScreenCaptureFromBase64String(base64).build());
            ExtentManager.test().info("URL at failure: " + driver.getCurrentUrl());
            driver.manage().logs().get(LogType.BROWSER).getAll().stream()
                .filter(e -> e.getLevel() == Level.SEVERE)
                .forEach(e -> ExtentManager.test().warning("console: " + e.getMessage()));
        } else {
            ExtentManager.test().fail(result.getThrowable());
        }
        ExtentManager.unload();
    }

    @Override public void onTestSkipped(ITestResult result) {
        ExtentManager.test().skip(result.getThrowable());
        ExtentManager.unload();
    }

    @Override public void onFinish(ITestContext context) {
        ExtentManager.report().flush();
    }
}

```

The null check on the driver matters: if setup itself failed, there is no driver, and a listener that throws an NPE hides the original failure completely. That is a real bug I'd expect a senior candidate to anticipate.

Q240

Write code that verifies every image on a page has meaningful alt text and actually loads.

```
@Test
public void allImagesHaveAltTextAndLoad() {
    JavascriptExecutor js = (JavascriptExecutor) driver;
    List<WebElement> images = driver.findElements(By.tagName("img"));
    Assert.assertFalse(images.isEmpty(), "No images found – is the page loaded?");

    SoftAssert soft = new SoftAssert();
    for (WebElement img : images) {
        String src = img.getAttribute("src");
        String alt = img.getAttribute("alt");

        // alt="" is VALID for decorative images; a missing attribute is not
        soft.assertNotNull(alt, "Missing alt attribute on image: " + src);

        // did the image actually render? naturalWidth is 0 for a broken image
        Long naturalWidth = (Long) js.executeScript(
            "return arguments[0].naturalWidth;", img);
        soft.assertTrue(naturalWidth != null && naturalWidth > 0,
            "Broken image (failed to load): " + src);
    }
    soft.assertAll();
}
```

`naturalWidth === 0` is the standard way to detect a broken image — the `<img>` element exists and is "visible" to Selenium even when the file 404s, so `isDisplayed()` tells you nothing. Knowing that distinction is a strong signal.

ADVANCED**HANDS-ON**


```

public Map<String, Long> capturePageTimings() {
    JavascriptExecutor js = (JavascriptExecutor) driver;

    Long navigationStart = (Long) js.executeScript(
        "return window.performance.timing.navigationStart;");
    Long domContentLoaded = (Long) js.executeScript(
        "return window.performance.timing.domContentLoadedEventEnd;");
    Long loadComplete = (Long) js.executeScript(
        "return window.performance.timing.loadEventEnd;");
    Long responseEnd = (Long) js.executeScript(
        "return window.performance.timing.responseEnd;");

    return Map.of(
        "serverResponseMs", responseEnd - navigationStart,
        "domReadyMs",      domContentLoaded - navigationStart,
        "fullLoadMs",      loadComplete - navigationStart);
}

@Test
public void homePageLoadsWithinBudget() {
    driver.get(Config.get("base.url"));
    Map<String, Long> timings = capturePageTimings();
    logger.info("Page timings: {}", timings);
    Assert.assertTrue(timings.get("fullLoadMs") < 3000,
        "Home page took " + timings.get("fullLoadMs") + "ms, budget is 3000ms");
}

```

Playwright equivalent, plus modern Core Web Vitals:

```

const nav = JSON.parse(await page.evaluate(
    () => JSON.stringify(performance.getEntriesByType('navigation')[0])));
expect(nav.loadEventEnd).toBeLessThan(3000);

const lcp = await page.evaluate(() => new Promise(resolve => {
    new PerformanceObserver(list => {
        const entries = list.getEntries();
        resolve(entries[entries.length - 1].startTime);
    }).observe({ type: 'largest-contentful-paint', buffered: true });
}));
expect(lcp).toBeLessThan(2500);           // Google's "good" LCP threshold

```

Be honest about the limit: this is **single-user front-end timing**, not load testing. It catches a regression where someone ships a 4MB unoptimised hero image; it says nothing about behaviour at 500 concurrent users.

ADVANCED

HANDS-ON

```
// pages/checkout.page.ts
import { Page, Locator, expect } from '@playwright/test';

export class CheckoutPage {
  private readonly cardNumber: Locator;
  private readonly expiry: Locator;
  private readonly cvv: Locator;
  private readonly payButton: Locator;
  private readonly confirmation: Locator;
  private readonly errorBanner: Locator;

  constructor(private readonly page: Page) {
    this.cardNumber = page.getByLabel('Card number');
    this.expiry = page.getByLabel('Expiry');
    this.cvv = page.getByLabel('CVV');
    this.payButton = page.getByRole('button', { name: 'Pay now' });
    this.confirmation = page.getByRole('heading', { name: 'Order confirmed' });
    this.errorBanner = page.getByRole('alert');
  }

  async goto(orderId: string): Promise<void> {
    await this.page.goto(`/checkout/${orderId}`);
    await expect(this.payButton).toBeVisible();
  }

  async payWithCard(card: { number: string; expiry: string; cvv: string }): Promise<void> {
    await test.step(`pay with card ending ${card.number.slice(-4)}`, async () => {
      await this.cardNumber.fill(card.number);
      await this.expiry.fill(card.expiry);
      await this.cvv.fill(card.cvv);
      await this.payButton.click();
    });
  }

  async expectConfirmed(): Promise<void> { await expect(this.confirmation).toBeVisible(); }
  async expectError(message: string): Promise<void> {
    await expect(this.errorBanner).toHaveText(message);
  }
}
```

```
// tests/checkout.spec.ts
import { test } from '../fixtures';
import { CheckoutPage } from '../pages/checkout.page';

test('declines an expired card with a clear message', async ({ page, testOrder }) => {
  const checkout = new CheckoutPage(page);
  await checkout.goto(testOrder.id);
  await checkout.payWithCard({ number: '4000000000000069', expiry: '01/20', cvv: '123' });
});
```

```
await checkout.expectError('Your card has expired. Please use a different card.');
```

CORE

HANDS-ON

```
// fixtures.ts
import { test as base, APIRequestContext } from '@playwright/test';

type Order = { id: string; sku: string; total: number };

type Fixtures = {
  testOrder: Order;
  api: APIRequestContext;
};

export const test = base.extend<Fixtures>({
  api: async ({ playwright }, use) => {
    const ctx = await playwright.request.newContext({
      baseURL: process.env.API_URL,
      extraHTTPHeaders: { Authorization: `Bearer ${process.env.API_TOKEN}` },
    });
    await use(ctx);
    await ctx.dispose();
  },

  testOrder: async ({ api }, use, testInfo) => {
    // unique per test AND per worker, so parallel runs never collide
    const ref = `AUTOTEST-${testInfo.workerIndex}-${Date.now()}`;

    const res = await api.post('/api/v1/orders', {
      data: { reference: ref, sku: 'TSH-001', qty: 1 },
    });
    if (!res.ok()) throw new Error(`Order setup failed: ${res.status()} ${await res.text()}`);
    const order: Order = await res.json();

    await use(order); // ← the test body runs here

    // teardown runs even if the test failed
    await api.delete(`/api/v1/orders/${order.id}`).catch(() => { /* already gone */ });
  },
});

export { expect } from '@playwright/test';
```

Three things to call out: setup through the **API** not the UI (fast and stable), a **unique reference including the worker index** (parallel-safe), and teardown that **tolerates already-deleted data** so cleanup never masks the real failure.

Write a Playwright test that mocks an API to test an error state.

```
test('shows a friendly message when the orders API is down', async ({ page }) => {
  await page.route('**/api/v1/orders**', route =>
    route.fulfill({
      status: 500,
      contentType: 'application/json',
      body: JSON.stringify({ message: 'Internal Server Error' }),
    }));

  await page.goto('/orders');

  await expect(page.getByRole('alert'))
    .toHaveText("We couldn't load your orders. Please try again.");
  await expect(page.getByRole('button', { name: 'Retry' })).toBeVisible();
  // and it must NOT show a raw stack trace or the word "Error 500"
  await expect(page.locator('body')).not.toContainText('Internal Server Error');
});

test('shows the empty state when the user has no orders', async ({ page }) => {
  await page.route('**/api/v1/orders**', route =>
    route.fulfill({ status: 200, json: { orders: [], total: 0 } }));

  await page.goto('/orders');
  await expect(page.getByText('You have no orders yet')).toBeVisible();
  await expect(page.getByRole('link', { name: 'Start shopping' })).toBeVisible();
});
```

Empty states and error states are the two things almost nobody tests, because producing them for real is hard. Mocking makes them trivial — say that, and you've explained *why* mocking earns its place rather than just how to write it.

CORE

HANDS-ON

```

@Test
public void createOrderReturnsAValidConfirmedOrder() {
    String payload = ""
        { "sku": "TSH-001", "qty": 2, "currency": "INR" }
        "";

    Response response = given()
        .spec(requestSpec)
        .body(payload)
        .when()
        .post("/api/v1/orders")
        .then()
        .statusCode(201)
        .time(lessThan(2000L))
        .header("Location", matchesPattern(".*api/v1/orders/\\d+"))
        .body(matchesJsonSchemaInClasspath("schemas/order-schema.json"))
        .body("status", equalTo("CONFIRMED"))
        .body("qty", equalTo(2))
        .body("total", greaterThan(0f))
        .body("createdAt", notNullValue())
        .body("items.size()", equalTo(1))
        .body("customer.email", not(emptyString()))
        .extract().response();

    // chain: the created resource must be readable and consistent
    String orderId = response.path("id").toString();
    given().spec(requestSpec).get("/api/v1/orders/" + orderId)
        .then().statusCode(200)
        .body("id", equalTo(Integer.parseInt(orderId)))
        .body("status", equalTo("CONFIRMED"));
}

@Test
public void createOrderRejectsANegativeQuantity() {
    given().spec(requestSpec).body("{\"sku\":\"TSH-001\",\"qty\":-1}")
        .when().post("/api/v1/orders")
        .then().statusCode(422)
        .body("errors[0].field", equalTo("qty"))
        .body("errors[0].message", containsStringIgnoringCase("must be greater than 0"));
}

```

Schema validation is the line to point at: field-by-field assertions pass happily when the API adds, removes or retypes a field. The schema check catches contract drift that individual assertions miss.

```

@Test
public void placedOrderIsConsistentAcrossUiApiAndDatabase() {
    // 1. UI - perform the business action
    String orderRef = checkoutPage.placeOrder();
    BigDecimal uiTotal = confirmationPage.getTotalAmount();

    // 2. API - the service agrees
    Response api = given().spec(requestSpec).get("/api/v1/orders/" + orderRef);
    api.then().statusCode(200);
    BigDecimal apiTotal = new BigDecimal(api.path("total").toString());

    // 3. DB - the data was actually persisted, with the audit trail
    OrderRow db = queryOrder(orderRef);

    SoftAssert soft = new SoftAssert();
    soft.assertEquals(apiTotal.compareTo(uiTotal), 0,
        "UI total " + uiTotal + " != API total " + apiTotal);
    soft.assertEquals(db.total.compareTo(uiTotal), 0,
        "UI total " + uiTotal + " != DB total " + db.total);
    soft.assertEquals(db.status, "CONFIRMED");
    soft.assertNotNull(db.createdAt, "Audit timestamp missing");
    soft.assertAll();
}

private OrderRow queryOrder(String ref) {
    String sql = "SELECT status, total_amount, created_at FROM orders WHERE order_ref = ?";
    try (Connection con = DriverManager.getConnection(
        Config.get("db.url"), Config.get("db.user"), Config.get("db.password"));
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, ref);
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) throw new AssertionError("No DB row for order " + ref);
            return new OrderRow(rs.getString("status"),
                rs.getBigDecimal("total_amount"),
                rs.getTimestamp("created_at"));
        }
    } catch (SQLException e) {
        throw new FrameworkException("DB verification failed for " + ref, e);
    }
}

```

Note `BigDecimal.compareTo(...) == 0` rather than `equals`: new `BigDecimal("1499.0").equals(new BigDecimal("1499.00"))` is **false** because `equals` compares scale. That is a real money bug and a superb detail to mention unprompted.


```

public final class TestDataFactory {
    private static final Faker FAKER = new Faker(new Locale("en-IND"));
    private static final AtomicInteger COUNTER = new AtomicInteger();

    /** Unique across parallel threads AND across re-runs on the same machine. */
    public static String uniqueEmail() {
        return String.format("qa_%d_%d_%d@test.example.com",
            System.currentTimeMillis(),
            Thread.currentThread().getId(),
            COUNTER.incrementAndGet());
    }

    public static User newUser() { return newUser(u -> {}); }

    /** Builder-with-overrides: sensible defaults, override only what the test cares about. */
    public static User newUser(Consumer<User.Builder> overrides) {
        User.Builder b = User.builder()
            .email(uniqueEmail())
            .name(FAKER.name().fullName())
            .phone("9" + FAKER.number().digits(9))
            .role("USER")
            .active(true);
        overrides.accept(b);
        return b.build();
    }

    /** Dates must be RELATIVE – a hardcoded date expires and the test fails next year. */
    public static LocalDate futureBookingDate() { return LocalDate.now().plusDays(7); }
    public static String futureCardExpiry() {
        return LocalDate.now().plusYears(2).format(DateTimeFormatter.ofPattern("MM/yy"));
    }

    private TestDataFactory() { }
}

// usage – the test states only what matters to it
User admin = TestDataFactory.newUser(u -> u.role("ADMIN"));
User locked = TestDataFactory.newUser(u -> u.active(false));

```

The "defaults plus overrides" pattern is worth naming explicitly: the test declares only the attribute under test, so when a new mandatory field is added you change one factory instead of 200 tests.

```

public final class Mask {
    private static final Pattern CARD = Pattern.compile("\\b(?:\\d[ -]*?){13,19}\\b");
    private static final Pattern EMAIL = Pattern.compile("([\\w.+]{1,3})[\\w.+]*@([\\w.-]+)");
    private static final Pattern PHONE = Pattern.compile("\\b(\\d{2})\\d{6}(\\d{2})\\b");
    private static final List<String> SECRET_KEYS =
        List.of("password", "token", "authorization", "secret", "cvv", "otp", "apiKey");

    public static String text(String input) {
        if (input == null) return null;
        String out = CARD.matcher(input).replaceAll(m -> keepLast4(m.group()));
        out = EMAIL.matcher(out).replaceAll("$1***@2");
        out = PHONE.matcher(out).replaceAll("$1*****2");
        return out;
    }

    /** Masks values of known-sensitive keys in a JSON body before logging it. */
    public static String json(String body) {
        String out = body;
        for (String key : SECRET_KEYS) {
            out = out.replaceAll("(?i)(\"" + key + "\"\\s*:\\s*\\\"[^\"]*(\\\"))", "$1***$2");
        }
        return out;
    }

    private static String keepLast4(String digits) {
        String clean = digits.replaceAll("\\D", "");
        return clean.length() < 4 ? "****"
            : "****".repeat(clean.length() - 4) + clean.substring(clean.length() - 4);
    }

    private Mask() { }
}

// usage
logger.info("Request: {}", Mask.json(payload));
ExtentManager.test().info("Paying with " + Mask.text(cardNumber)); // **** **** **** 1111

```

Volunteering this is a strong move: *"Test reports get attached to Jira tickets and shared in Slack. A failure screenshot or a logged request body containing a real customer's card number is a data breach, even though it came from a test."* Very few candidates think this far.

ADVANCED

HANDS-ON

```

public final class DriverFactory {
    private static final ThreadLocal<WebDriver> TL = new ThreadLocal<>();

    public static void init() {
        String target = Config.get("run.target");    // local | grid | mobile
        String browser = Config.get("browser");
        boolean headless = Config.getBool("headless");

        WebDriver driver = switch (target.toLowerCase()) {
            case "grid" -> remote(browser, headless);
            case "mobile" -> mobile();
            default -> local(browser, headless);
        };
        driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(45));
        if (!"mobile".equalsIgnoreCase(target)) driver.manage().window().maximize();
        TL.set(driver);
    }

    private static WebDriver local(String browser, boolean headless) {
        return switch (browser.toLowerCase()) {
            case "firefox" -> new FirefoxDriver(firefoxOptions(headless));
            case "edge" -> new EdgeDriver(new EdgeOptions());
            case "chrome" -> new ChromeDriver(chromeOptions(headless));
            default -> throw new IllegalArgumentException("Unsupported browser: " + browser);
        };
    }

    private static WebDriver remote(String browser, boolean headless) {
        try {
            AbstractDriverOptions<?> options = "firefox".equalsIgnoreCase(browser)
                ? firefoxOptions(headless) : chromeOptions(headless);
            RemoteWebDriver driver = new RemoteWebDriver(
                new URL(Config.get("grid.url")), options);
            driver.setFileDetector(new LocalFileDetector());    // enables file upload on nodes
            return driver;
        } catch (MalformedURLException e) {
            throw new FrameworkException("Bad grid URL: " + Config.get("grid.url"), e);
        }
    }

    private static ChromeOptions chromeOptions(boolean headless) {
        ChromeOptions o = new ChromeOptions();
        if (headless) o.addArguments("--headless=new", "--window-size=1920,1080");
        o.addArguments("--no-sandbox", "--disable-dev-shm-usage", "--disable-notifications");
        o.setExperimentalOption("prefs",
            Map.of("profile.default_content_setting_values.notifications", 2));
        return o;
    }

    public static WebDriver get() {

```

```

        WebDriver d = TL.get();
        if (d == null) throw new IllegalStateException(
            "Driver not initialised on thread " + Thread.currentThread().getName());
        return d;
    }

    public static void quit() {
        WebDriver d = TL.get();
        if (d != null) {
            try { d.quit(); } finally { TL.remove(); }    // remove() even if quit() throws
        }
    }
    private DriverFactory() { }
}

```

The try { quit(); } finally { TL.remove(); } detail matters: if quit() throws because the browser already crashed, a naive implementation leaks the ThreadLocal and the next test on that pooled thread inherits a dead driver.

CORE

ADVANCED

Live exercise: "Automate this flow on a site you've never seen." How do you approach it?

Many companies now do this instead of theory questions — they open a demo site (saucedemo, the-internet, automationexercise) and ask you to automate a flow while sharing your screen. They are watching your **process**, not your typing speed.

1. **Clarify first.** "Should I write this as a full framework or a single script? Do you want me to prioritise structure or coverage in the time we have?" This alone puts you ahead of most candidates.
2. **Explore the app for two minutes** before writing anything. Walk the flow manually, open DevTools, and look at what locators exist. Say what you see: "there are no test ids, but the ids look stable, so I'll use those."
3. **Write the test first, top-down**, as if the page objects already existed:

```
@Test
public void userCanBuyAProduct() {
    loginPage.loginAs(user, pass);
    productsPage.addToCart("Sauce Labs Backpack");
    cartPage.checkout();
    checkoutPage.fillDetails("Ravi", "Kumar", "560001");
    Assert.assertEquals(confirmationPage.getMessage(), "Thank you for your
order!");
}
```

Then implement downwards. It shows design thinking and gives you a working skeleton even if you run out of time.

4. **Narrate the trade-offs.** "I'm using an explicit wait here rather than an implicit one because..." — the commentary is most of your score.
5. **Add one negative test** if time allows. Almost every candidate automates only the happy path; adding "login with a locked-out user" shows a tester's instinct rather than a coder's.
6. **Finish with what you'd do next:** "with more time I'd externalise the test data, add the driver factory for cross-browser, and wire this into CI with a screenshot listener." That turns an unfinished exercise into a demonstration of judgement.

Practise on saucedemo.com and the-internet.herokuapp.com until you can produce a clean POM-structured login-to-checkout test in 20 minutes from a blank file, talking the whole time. That single rehearsal is worth more than another 50 questions of theory.

BONUS**Cheat sheets — print these two pages**

The last-30-minutes-before-the-interview revision material.

TASK	SELENIUM (JAVA)	PLAYWRIGHT (TS)
Launch	<code>new ChromeDriver()</code>	<code>await chromium.launch()</code>
Navigate	<code>driver.get(url)</code>	<code>await page.goto(url)</code>
Find	<code>driver.findElement(By.id("x"))</code>	<code>page.locator('#x')</code>
By text	<code>By.xpath("//*[text()='Save']")</code>	<code>page.getByText('Save')</code>
By role	—	<code>page.getByRole('button', {name: 'Save'})</code>
Click	<code>el.click()</code>	<code>await locator.click()</code>
Type	<code>el.clear(); el.sendKeys("x")</code>	<code>await locator.fill('x')</code>
Wait	<code>wait.until(ExpectedConditions...)</code>	automatic
Assert visible	<code>Assert.assertTrue(el.isDisplayed())</code>	<code>await expect(loc).toBeVisible()</code>
Dropdown	<code>new Select(el).selectByValue("IN")</code>	<code>await loc.selectOption('IN')</code>
iFrame	<code>driver.switchTo().frame(x) ... defaultContent()</code>	<code>page.frameLocator('#f').getByRole('button')</code>
New tab	<code>getWindowHandles()</code> + switch	<code>page.waitForEvent('popup')</code>
Alert	<code>driver.switchTo().alert().accept()</code>	<code>page.on('dialog', d => d.accept())</code>
Upload	<code>el.sendKeys(absPath)</code>	<code>await loc.setInputFiles(path)</code>
Screenshot	<code>((TakesScreenshot)driver).getScreenshotAs(...)</code>	<code>await page.screenshot({path: 'screenshot.png'})</code>
Execute JS	<code>((JavascriptExecutor)driver).executeScript(...)</code>	<code>await page.evaluate(() => { ... })</code>
Cookies	<code>driver.manage().addCookie(c)</code>	<code>await context.addCookies([c])</code>

Mock network	CDP (Chromium only)	<code>await page.route(url, r => r.fulfill(...))</code>
Close	<code>driver.quit()</code>	<code>await browser.close()</code>

- **Verification / Validation** — building it right / building the right thing.
- **Smoke / Sanity** — is the build testable? / is this one fix working?
- **Retest / Regression** — confirm this bug is fixed / confirm nothing else broke.
- **Severity / Priority** — technical impact (tester) / business urgency (PO).
- **Stub / Driver** — replaces the called module / replaces the calling module.
- **Alpha / Beta** — internal staff at our site / real users at their site.
- **EP / BVA** — one value per class / values at and around the edges.
- **findElement / findElements** — throws if absent / returns an empty list.
- **close() / quit()** — current window / whole session and driver process.
- **Implicit / Explicit wait** — global DOM presence / per-element condition.
- **Absolute / Relative XPath** — from `/html`, brittle / from `//`, resilient.
- **StaleElement** — the DOM node your reference pointed to was replaced.
- **ThreadLocal** — one value per thread; makes parallel Selenium safe.
- **Singleton / Factory** — one instance globally / create objects without naming the class.
- **@Parameters / @DataProvider** — config from XML / data rows from Java.
- **Hard / Soft assert** — stops at the first failure / collects all, needs `assertAll()`.
- **Surefire / Failsafe** — unit tests at `test` / integration tests at `verify`.
- **Merge / Rebase** — preserves history with a merge commit / rewrites into a linear history.
- **CI / CD** — integrate and test every commit / always deployable (or auto-deployed).
- **Locator / ElementHandle** (PW) — lazy description, never stale / bound node, can go stale.
- **BrowserContext** (PW) — an isolated incognito session created in milliseconds.
- **Web-first assertion** (PW) — an assertion that auto-retries until it passes or times out.
- **storageState** (PW) — saved cookies + localStorage, so you log in once for the whole suite.
- **Idempotent** — calling it N times has the same effect as calling it once.
- **401 / 403** — not authenticated / authenticated but not allowed.

Red-flag phrases to avoid — and what to say instead

✗ DON'T SAY	✓ SAY INSTEAD
"I use Thread.sleep when the element is not found."	"I use an explicit wait on the specific condition — clickable, visible, or a text change."
"If a test is flaky I just add a retry."	"Retries absorb infrastructure noise; genuine flakiness gets quarantined with an owner and fixed."
"I automate everything."	"I automate by ROI and risk — exploratory and usability testing stay human."
"Automation reduces the need for testers."	"Automation changes what testers spend time on — more exploratory, more risk analysis."
"100% code coverage is our goal."	"Coverage finds untested gaps; assertion quality is what I actually optimise for."
"I always use JavaScript click, it's more reliable."	"A JS click bypasses the real event model, so it can hide UI bugs — I use it only as a documented workaround."
"The developer's code was bad."	"We found an environment difference between QA and prod, and fixed the parity."
"It works on my machine, so it's a CI problem."	"CI is the source of truth — a CI-only failure is a defect in the test."
"I block the release."	"I make the risk explicit and give the PO options; the go/no-go is theirs."
"Playwright is just better than Selenium."	"Playwright is stronger for X and Y; Selenium is the right choice when Z. It depends on the team."

FINAL

Questions YOU should ask the interviewer

"Do you have any questions for us?" is a scored question. Saying "no" is the most common self-inflicted wound in interviewing.

Ask 3–4 of these. They make you look senior and they tell you whether to accept the offer.

About the work

- "What does the current automation suite look like — how many tests, how long does it take, and what's the pass rate on a typical night?" (*A vague or embarrassed answer tells you a lot.*)
- "Is the automation a gate in the pipeline, or is it informational?"
- "How is the QA team structured — embedded in feature teams, or a separate QA function?"
- "Who writes the automated tests today — QA only, or do developers contribute?"
- "What would you want the person in this role to have achieved in the first six months?"

About quality culture

- "When a bug reaches production, what does the team do afterwards?" (*Looking for: blameless post-mortem and a regression test — not blame.*)
- "How involved is QA at the requirements stage?"
- "What's the release cadence, and what's the biggest bottleneck in getting a change to production?"
- "How do you handle the trade-off when a release date is fixed and testing isn't complete?"

About growth

- "What does the growth path look like for an automation engineer here — is there a senior IC track, or is it management?"
- "Is there budget/time for learning — conferences, certifications, internal tech talks?"
- "What's the biggest technical challenge the team is facing right now?" (*Great question — the answer often reveals the real reason they're hiring.*)

Close the interview with intent: **"This sounds like exactly the kind of problem I enjoy — is there anything about my background you'd like me to expand on, or any concern I can address?"** It's a confident, professional close that occasionally lets you fix a misunderstanding on the spot.

Final checklist — the week of the interview

- ☐ Your 3-minute introduction (Q194) rehearsed out loud, at least five times.
- ☐ Your framework explanation (Q132) rehearsed with a diagram you can draw from memory.
- ☐ Three STAR stories ready: a technical win, a conflict, a mistake you learned from.
- ☐ Numbers memorised: suite size, runtime, pass rate, coverage, release frequency, team size.
- ☐ Code you can write without an IDE: explicit wait, POM class, table row XPath, data provider, TestNG listener, a Playwright test with fixtures.
- ☐ Every technology on your CV re-read — assume you'll be asked about the ones you're weakest on.
- ☐ Read the company's product; use it if it's public; look up their tech blog and job description keywords.
- ☐ Questions to ask, written down.
- ☐ Setup checked for a remote round: camera, mic, lighting, stable internet, a quiet room, an IDE and browser ready to share.
- ☐ Print or bookmark this guide, and skim the "red flag phrases" table on the morning of the interview.

Last thing. Interviewers are not looking for someone who knows every answer. They are looking for someone who thinks clearly, admits what they don't know, and shows they can learn it. If you're stuck, say: *"I haven't used that directly — my understanding is X, and here's how I'd find out."* That answer scores far better than a confident guess.